

前言

个人代码规范

笔记

计算机

基本数据类型

宏

条件编译

宏替换

宏函数

结构体

联合体（共用体）

符号

语句

关键字

数组

指针

字符串

C语言虚拟内存分区

函数

多文件编译

浮点数

笔记中使用到的Linux命令

杂项知识点

MinGW-w64目录说明

编码表（符号编解码）

英语系国家的编码表 —— ASCII

非英语系国家的编码表 —— 国标表（GB系列）、万国表（Unicode表）

国标表（国标码）

万国表（万国码）

gcc的内联汇编（了解）

注脚

前言

- 书籍的推荐先读：《高质量嵌入式 LinuxC 编程》、《深入理解计算机系统（原书第三版）》
- C语言大体的框架是统一的，但是在具体某些实现细节上不同的编译器厂家实现不同；不同操作系统和CPU架构上的相同函数，效果也会有不同 —— **不同操作系统、不同CPU、不同的编译器的C语言行为一致（效果差不多），但具体的实现细节各不相同**
- 有些知识有歧义，应该在有所区分

个人代码规范

- 代码规范
 -

○

标识符类型	推荐命名风格	别名/缩写示例	命名示例	规范说明与设计意图
局部变量 (Local Var)	小驼峰式 / 小写下划线	<code>l_</code> (Local)	<code>l_loopCounter</code> <code>adc_raw_value</code>	简短、见名知意。在嵌套较深的代码块中，加 <code>l_</code> 前缀可一眼与全局变量区分。
全局变量 (Global Var)	前缀 + 小驼峰 / 下划线	<code>g_</code> (Global)	<code>g_sysTickCount</code> <code>g_u8_app_status</code>	嵌入式开发大忌：滥用全局变量。 必须加 <code>g_</code> 前缀，警告开发者该变量有并发修改风险。
静态全局变量 (Static Global)	前缀 + 小驼峰	<code>sg_</code> (Static Global)	<code>sg_uartRxBuffer</code>	限制在当前 <code>.c</code> 文件内使用的全局变量，加 <code>sg_</code> 提示其作用域。
常量 (Constant)	全大写 + 下划线	<code>c_</code> (有时用作前缀)	<code>MAX_BUFFER_SIZE</code> <code>c_pi_value</code>	必须全大写，用于硬件缓冲区大小、数学常数等，不可更改。
普通函数 (Function)	模块名 + 动名词结合	<code>m_</code> (Module)	<code>HAL_GPIO_Write()</code> <code>Adc_ReadValue()</code>	推荐“ 模块名_动作+对象 ”结构。面向对象思想，便于在 IDE 中输入模块名时自动联想。
中断服务程序 (ISR)	触发源 + 后缀	<code>_ISR</code> / <code>_IRQHandler</code>	<code>Timer0_ISR()</code> <code>USART1_IRQHandler()</code>	必须有明确的中断后缀，绝对不能在常规代码中手动调用。
宏定义 (Macro)	全大写 + 下划线	<code>m_</code> (Macro)	<code>SET_BIT(reg, bit)</code> <code>IS_PARAM_VALID</code>	带参数的宏（如位操作）必须全大写。注意：嵌入式中能用 <code>inline</code> 函数代替宏的尽量代替。
结构体/枚举类型 (Typedef)	大驼峰 + 类型后缀	<code>_t</code> (Type) / <code>_s</code> / <code>_e</code>	<code>PidController_t</code> <code>TaskState_e</code>	别名加 <code>_t</code> 是标准 C 规范。结构体加 <code>_s</code> (Structure)，枚举加 <code>_e</code> (Enumeration)。
头文件 (Header)	小写加下划线 / 小驼峰	<code>.hpp</code> / <code>.h</code>	<code>file_system.hpp</code> <code>http_session.h</code>	现代 C++ 更倾向于用 <code>.hpp</code> 表示包含模板或内联函数的头文件，以区分纯 C 的 <code>.h</code> 。
源文件 (Source)	与头文件严格——对应	<code>.cpp</code> / <code>.cc</code>	<code>file_system.cpp</code> <code>http_session.cc</code>	后缀必须保持项目全系统统一。Google 习惯用 <code>.cc</code> ，而 Windows/多数通用项目习惯用 <code>.cpp</code> 。
测试文件 (Test)	原文件名 + 后缀	<code>_test</code> / <code>_unittest</code>	<code>file_system_test.cpp</code>	便于测试框架（如 Google Test）或构建工具（如 CMake）自动识别并排除在生产环境之外。
接口/抽象类 (Interface)	前缀 <code>i</code> 或大驼峰	<code>i_</code> / <code>I</code>	<code>i_render_engine.hpp</code> <code>IRenderEngine.h</code>	提示该文件只包含纯虚函数（抽象类），没有具体实现。

标识符类型	推荐命名风格	别名/缩写示例	命名示例	规范说明与设计意图
C++普通成员变量 (Private/Protected)	小写加下划线 尾缀 / 小驼峰 加 <code>m_</code> 前缀	<code>_</code> (尾缀) <code>m_</code> (前缀 Member)	<code>connection_count_</code> <code>m_connectionCount</code>	极重要： 加尾缀 <code>_</code> (Google 规范) 或 <code>m_</code> 可以瞬间将其与函数局部变量、函数入参区分开。
C++公有成员变量 (Public Struct)	小写加下划线 / 小驼峰	无特殊前后缀	<code>name</code> <code>age</code>	如果是纯数据结构 (struct)，通常不需要加 <code>_</code> 或 <code>m_</code> ，直接暴露访问。
C++静态成员变量 (Static Member)	前缀区分	<code>s_</code> (Static) <code>s_..._</code>	<code>s_instance_</code> <code>m_s_max_players</code>	提示该变量属于整个类而非某个实例，多线程环境下需注意并发安全。
C++类内常量 / 枚举 (Static Constant)	全大写 或 <code>k</code> 开头的驼峰	<code>k</code> (Constant)	<code>kMaxRetryTimes</code> <code>MAX_BUFFER_SIZE</code>	类内部的 <code>constexpr</code> 或 <code>const static</code> 变量。使用 <code>k</code> 前缀是 Google 风格，能有效避免宏污染。

- 写代码小思路： 指令（接收的参数、属性等） <——映射——> 行为（处理函数） 绘制成表（结构体数组）

笔记

1. 计算机

- 操作系统的如何运行软件
 - 计算机系统层次结构图



- 软件的运行 ——需要——> 机器码符合运行cpu的体系架构（指令集） + 操作系统的支撑（加载进入内存 将应用层与硬件层练习起来）

■ 操作系统

1. 提供一个CLI (command-line interface) 与用户进行交互 【图形界面并不是操作系统必须的】

2. 提供一个查找 可执行程序 的依据（关键字 字符串 语法）

3. 提供一个查找路径 通过环境变量进行查找

- 绝对路径：从根目录 或者 根盘符 开始
- 相对路径：相对在哪里开始（当前目录 或者 去PATH中寻找）
- 每次运行软件都会，读取当前的系统的全部环境变量
- Windows中环境变量
 - PATH：查找路径 提供路径前缀
 - TEMP 和 TMP：临时目录（中间文件）

■ Windos是单用户操作系统（Windos Server是多用户，但是需要购买）；Linux是多用户操作系统（指同一时间，多个用户能操作资源）

• 分类：机器语言、汇编语言、高级语言

- cpu（中央处理器 —— 负责处理指令 【二进制的规范集合 —— 机器码】）
- 高级语言 ==> 机器码 需要编译器（翻译官，解析规范与具体的cpu有关，该规范的理论由 ISO制定，具体的实现规范由具体的 编译器厂家进行规定 如：微软的MSVC编译器、开源的GNU编译器）¹

• 通过 编译器 将 高级语言 解析成 机器语言

1. 预处理：对源文件（.c）进行处理，展开头文件²、替换宏、去掉注释、条件编译。

```
gcc -E xxx.c -o xxx.i
```

预处理文件后缀名为 `.i`，在该阶段报错，通常为头文件编译器没有找到。

2. 编译：将 预处理文件 转换成 汇编文件（.s）

```
gcc -S xxx.i -o xxx.s
```

在该阶段报错，通常为语法、语句出现错误。

3. 汇编：将 汇编文件 转换成 二进制文件（.o）

```
gcc -c xxx.s -o xxx.o
```

4. 链接：将多个二进制文件，组合成可执行程序（Windows 上为 PE 格式——.exe；Linux 上为 ELF 格式——.out）

```
gcc xxx.o -o xxx
```

在该阶段报错，通常为文件之间的调用出现问题，例如：在本文件调用外部的 fun 函数，在开头没有引用其对应头文件 或者 没有用 extern 关键字声明 fun 函数为外部函数。

工程上的理解：编译阶段（预处理、编译、汇编）=》链接阶段

实际上编译器会根据文件后缀名进行执行（如：`gcc abc.s -o abc.exe` 会执行汇编、链接）

- gcc 常用选项

选项	用法	含义	工具	注意事项
-E		预处理(.i)	cpp	该阶段可以 引入宏-D 指定头文件-I
-S		生成汇编代码 (.s)	cc/cc1	编译阶段，输出 .s 文件 (cc1 负责)
-c		生成二进制代码 (.o)	as	汇编阶段，只编译生成目标文件 (.o)，不进行链接 (as 负责汇编)
-o	-o xx	链接(生成指定文件)	ld	链接阶段，将多个文件联系起来。如果没有-o 将默认输出a.exe/a.out
-O	-O	优化选项，取值范围 0~3		-O0（关闭）、-O1、-O2、-O3（最高常用）、-Ofast
-g		启动调式	gdb	
-Wall		启用所有常见的警告信息		
-Wextra		启用额外的警告信息		比 -Wall 更严格
-v		打印详细的编译过程信息		

选项	用法	含义	工具	注意事项
-D	-D <macro[=value]>	定义一个宏定义， 为定义的宏名称， [= value] 为可以定义其值。		在命令行中将宏定义传递给预处理器
-U	-U	取消定义 一个宏		
-L	-L	指定源文件目录， 源文件目录路径		路径有空格需要加""包裹
-I	-I	指定头文件目录， 头文件目录路径		
-static		生成静态链接（静态库）		生成静态可执行文件（体积大，但不需要依赖 dll）
-shared		生成动态链接（动态库）		用于生成 .dll (Windows) 或 .so (Linux)

- 编译器的 workflow

- 前提：

- 需要将 编译器的命令路径 放入到 环境变量PATH中

- 区分：

- 编辑器（只有编辑功能 vs code） 集成开发工具（内部集成多种工具 Visual Studio）

- 工作流程

- gcc -v 展示的详细信息

- 都有的信息

```

1 Using built-in specs.
2 COLLECT_GCC=E:\project\programming_environment\mingw64\bin\gcc.exe
3 COLLECT_LTO_WRAPPER=E:/project/programming_environment/mingw64/bin/./libexec/gcc/x86_64-w64-mingw32/13.2.0/lto-wrapper.exe
4 OFFLOAD_TARGET_NAMES=nvptx-none
5 Target: x86_64-w64-mingw32

```

```

6   Configured with: ../configure --
   prefix=/R/winlibs64ucrt_stage/inst_gcc-13.2.0/share/gcc --
   build=x86_64-w64-mingw32 --host=x86_64-w64-mingw32 --enable-
   offload-targets=nvptx-none --with-pkgversion='MinGW-w64 x86_64-
   ucrt-posix-seh, built by Brecht Sanders, r8' --with-
   tune=generic --enable-checking=release --enable-threads=posix -
   -disable-sjlj-exceptions --disable-libunwind-exceptions --
   disable-serial-configure --disable-bootstrap --enable-host-
   shared --enable-plugin --disable-default-ssp --disable-rpath --
   disable-libstdcxx-debug --disable-version-specific-runtime-libs
   --with-stabs --disable-symvers --enable-
   languages=c,c++,fortran,lto,objc,obj-c++ --disable-gold --
   disable-nls --disable-stage1-checking --disable-win32-registry
   --disable-multilib --enable-lto --enable-libquadmath --enable-
   libada --enable-libssp --enable-libstdcxx --enable-lto --
   enable-fully-dynamic-string --enable-libgomp --enable-graphite
   --enable-mingw-wildcard --enable-libstdcxx-time --enable-
   libstdcxx-pch --with-
   mpc=/d/Prog/winlibs64ucrt_stage/custombuilt --with-
   mpfr=/d/Prog/winlibs64ucrt_stage/custombuilt --with-
   gmp=/d/Prog/winlibs64ucrt_stage/custombuilt --with-
   isl=/d/Prog/winlibs64ucrt_stage/custombuilt --disable-
   libstdcxx-backtrace --enable-install-libiberty --enable-
   __cxa_atexit --without-included-gettext --with-diagnostics-
   color=auto --enable-clocale=generic --with-libiconv --with-
   system-zlib --with-build-sysroot=/R/winlibs64ucrt_stage/gcc-
   13.2.0/build_mingw/mingw-w64 CFLAGS='-
   I/d/Prog/winlibs64ucrt_stage/custombuilt/include/libdl-win32
   -march=nocona -msahf -mtune=generic -O2' CXXFLAGS='-wno-int-
   conversion -march=nocona -msahf -mtune=generic -O2' LDFLAGS='-
   pthread -Wl,--no-insert-timestamp -Wl,--dynamicbase -Wl,--high-
   entropy-va -Wl,--nxcompat -Wl,--tsaware'
   LD=/d/Prog/winlibs64ucrt_stage/custombuilt/share/binutils/bin/l
   d.exe
7   Thread model: posix
8   Supported LTO compression algorithms: zlib zstd
9   gcc version 13.2.0 (MinGW-w64 x86_64-ucrt-posix-seh, built by
   Brecht Sanders, r8)
10
11  # Target: x86_64-w64-mingw32    x86的cpu架构-windows64位操作系统-编
   译器厂家mingw
12  # 目标: cpu架构-操作系统-编译器厂家
13
14  # Configured with (配置)

```

■ 预处理阶段 (执行命令 `gcc -v -E -o abc.i abc.c`)

```

1   COLLECT_GCC_OPTIONS='-v' '-E' '-o' 'abc.i' '-mtune=generic' '-
   march=x86-64'

```

```

2      E:/project/programming_environment/mingw64/bin/./libexec/gcc/x
      86_64-w64-mingw32/13.2.0/cc1.exe -E -quiet -v -iprefix
      E:/project/programming_environment/mingw64/bin/./lib/gcc/x86_6
      4-w64-mingw32/13.2.0/ -D_REENTRANT abc.c -o abc.i -
      mtune=generic -march=x86-64 -dumpbase abc.c -dumpbase-ext .c
3      ignoring duplicate directory
      "E:/project/programming_environment/mingw64/lib/gcc/././lib/g
      cc/x86_64-w64-mingw32/13.2.0/include"
4      ignoring nonexistent directory
      "R:/winlibs64ucrt_stage/inst_gcc-13.2.0/share/gcc/include"
5      ignoring nonexistent directory
      "/R/winlibs64ucrt_stage/inst_gcc-13.2.0/share/gcc/include"
6      ignoring duplicate directory
      "E:/project/programming_environment/mingw64/lib/gcc/././lib/g
      cc/x86_64-w64-mingw32/13.2.0/include-fixed"
7      ignoring duplicate directory
      "E:/project/programming_environment/mingw64/lib/gcc/././lib/g
      cc/x86_64-w64-mingw32/13.2.0/./././././x86_64-w64-
      mingw32/include"
8      ignoring nonexistent directory "/mingw/include"
9      #include "..." search starts here:
10     #include <...> search starts here:
11
      E:/project/programming_environment/mingw64/bin/./lib/gcc/x86_6
      4-w64-mingw32/13.2.0/include
12
      E:/project/programming_environment/mingw64/bin/./lib/gcc/x86_6
      4-w64-mingw32/13.2.0/././././././include
13
      E:/project/programming_environment/mingw64/bin/./lib/gcc/x86_6
      4-w64-mingw32/13.2.0/include-fixed
14
      E:/project/programming_environment/mingw64/bin/./lib/gcc/x86_6
      4-w64-mingw32/13.2.0/././././././x86_64-w64-mingw32/include
15     End of search list.
16     COMPILER_PATH=E:/project/programming_environment/mingw64/bin/./
      /libexec/gcc/x86_64-w64-
      mingw32/13.2.0/;E:/project/programming_environment/mingw64/bin/
      ./libexec/gcc/;E:/project/programming_environment/mingw64/bin/
      ./lib/gcc/x86_64-w64-mingw32/13.2.0/./././././x86_64-w64-
      mingw32/bin/

```

```

17 LIBRARY_PATH=E:/project/programming_environment/mingw64/bin/./lib/gcc/x86_64-w64-
mingw32/13.2.0;E:/project/programming_environment/mingw64/bin/./lib/gcc;E:/project/programming_environment/mingw64/bin/./lib/gcc/x86_64-w64-mingw32/13.2.0/./././././x86_64-w64-
mingw32/lib/./lib;E:/project/programming_environment/mingw64/bin/./lib/gcc/x86_64-w64-
mingw32/13.2.0/./././././lib;E:/project/programming_environment/mingw64/bin/./lib/gcc/x86_64-w64-
mingw32/13.2.0/./././././x86_64-w64-
mingw32/lib;E:/project/programming_environment/mingw64/bin/./lib/gcc/x86_64-w64-mingw32/13.2.0/././././
18 COLLECT_GCC_OPTIONS='-v' '-E' '-o' 'abc.i' '-mtune=generic' '-march=x86-64' '-dumpdir' 'abc.'
19
20
21 # COLLECT_GCC_OPTIONS 设置预处理器gcc的选项
22
23 #
E:/project/programming_environment/mingw64/bin/./libexec/gcc/x86_64-w64-mingw32/13.2.0/cc1.exe -E -quiet -v -iprefix cc1.exe
间接调用cpp.exe(预处理器)

```

- 编译阶段 (执行命令 `gcc -v -S -o abc.s abc.i`)

```

1 COLLECT_GCC_OPTIONS='-v' '-S' '-o' 'abc.s' '-mtune=generic' '-
march=x86-64'
2
3 E:/project/programming_environment/mingw64/bin/./libexec/gcc/x
86_64-w64-mingw32/13.2.0/cc1.exe -fpreprocessed abc.i -quiet -
dumpbase abc.i -dumpbase-ext .i -mtune=generic -march=x86-64 -
version -o abc.s
4 GNU C17 (MinGW-w64 x86_64-ucrt-posix-seh, built by Brecht
Sanders, r8) version 13.2.0 (x86_64-w64-mingw32)
5 compiled by GNU C version 13.2.0, GMP version 6.3.0,
MPFR version 4.2.1, MPC version 1.3.1, isl version isl-0.26-GMP
6
7 GGC heuristics: --param ggc-min-expand=100 --param ggc-min-
heapsize=131072
8 Compiler executable checksum: 71996911801439ebea1892f3f3ba3826
9 COMPILER_PATH=E:/project/programming_environment/mingw64/bin/./
libexec/gcc/x86_64-w64-
mingw32/13.2.0/;E:/project/programming_environment/mingw64/bin/
./libexec/gcc/;E:/project/programming_environment/mingw64/bin/
./lib/gcc/x86_64-w64-mingw32/13.2.0/../../../../x86_64-w64-
mingw32/bin/
10 LIBRARY_PATH=E:/project/programming_environment/mingw64/bin/./
lib/gcc/x86_64-w64-
mingw32/13.2.0/;E:/project/programming_environment/mingw64/bin/
./lib/gcc/;E:/project/programming_environment/mingw64/bin/./l
ib/gcc/x86_64-w64-mingw32/13.2.0/../../../../x86_64-w64-
mingw32/lib/./lib/;E:/project/programming_environment/mingw64/
bin/./lib/gcc/x86_64-w64-
mingw32/13.2.0/../../../../lib/;E:/project/programming_environm
ent/mingw64/bin/./lib/gcc/x86_64-w64-
mingw32/13.2.0/../../../../x86_64-w64-
mingw32/lib/;E:/project/programming_environment/mingw64/bin/./
lib/gcc/x86_64-w64-mingw32/13.2.0/../../../../
11 COLLECT_GCC_OPTIONS='-v' '-S' '-o' 'abc.s' '-mtune=generic' '-
march=x86-64' '-dumpdir' 'abc.'
12
13 # cc/cc1.exe 编译器

```

- 汇编阶段 (执行命令 `gcc -v -c -o abc.o abc.s`)

```

1 COLLECT_GCC_OPTIONS='-v' '-c' '-o' 'abc.o' '-mtune=generic' '-
  march=x86-64'
2
3 E:/project/programming_environment/mingw64/bin/./lib/gcc/x86_64
  -w64-mingw32/13.2.0/./././././x86_64-w64-mingw32/bin/as.exe -v
  -o abc.o abc.s
4 GNU assembler version 2.42 (x86_64-w64-mingw32) using BFD
  version (Binutils for MingW-w64 x86_64, built by Brecht Sanders,
  r8) 2.42
5 COMPILER_PATH=E:/project/programming_environment/mingw64/bin/./
  libexec/gcc/x86_64-w64-
  mingw32/13.2.0/;E:/project/programming_environment/mingw64/bin/./
  libexec/gcc/;E:/project/programming_environment/mingw64/bin/./
  lib/gcc/x86_64-w64-mingw32/13.2.0/./././././x86_64-w64-
  mingw32/bin/
6 LIBRARY_PATH=E:/project/programming_environment/mingw64/bin/./l
  ib/gcc/x86_64-w64-
  mingw32/13.2.0/;E:/project/programming_environment/mingw64/bin/./
  lib/gcc/;E:/project/programming_environment/mingw64/bin/./lib
  /gcc/x86_64-w64-mingw32/13.2.0/./././././x86_64-w64-
  mingw32/lib/./lib/;E:/project/programming_environment/mingw64/b
  in/./lib/gcc/x86_64-w64-
  mingw32/13.2.0/./././././lib/;E:/project/programming_enviromen
  t/mingw64/bin/./lib/gcc/x86_64-w64-
  mingw32/13.2.0/./././././x86_64-w64-
  mingw32/lib/;E:/project/programming_environment/mingw64/bin/./l
  ib/gcc/x86_64-w64-mingw32/13.2.0/././././
7 COLLECT_GCC_OPTIONS='-v' '-c' '-o' 'abc.o' '-mtune=generic' '-
  march=x86-64' '-dumpdir' 'abc.'
8
9 # as.exe 汇编器 将 汇编文件 映射成 机器码
  # 当前阶段cpu可读懂，但不能执行

```

- 链接阶段 (执行命令 `gcc -v -o abc.exe abc.o`)

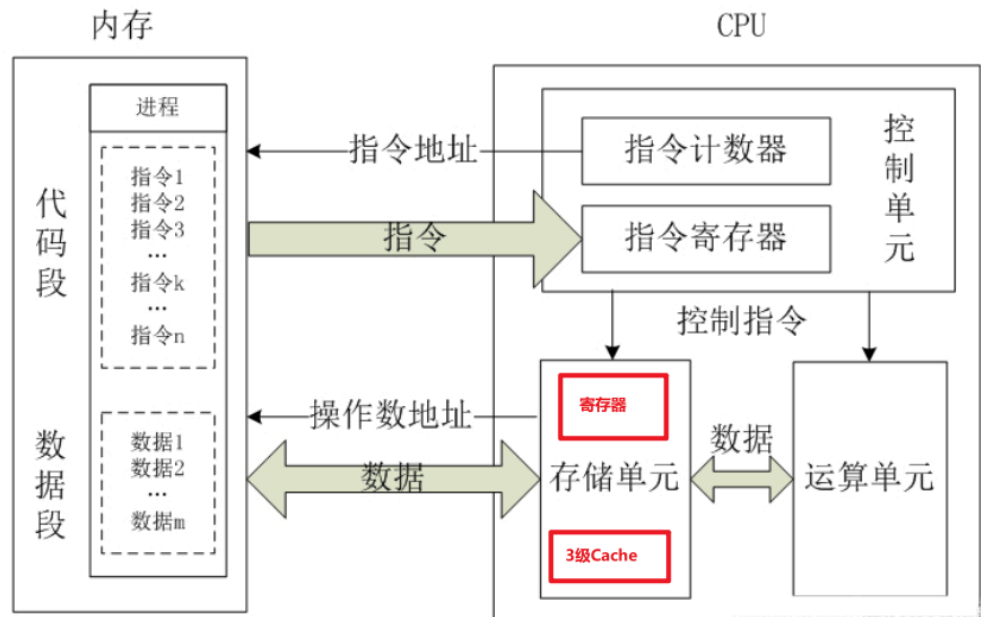
```
1 COMPILER_PATH=E:/project/programming_environment/mingw64/bin/..  
  /libexec/gcc/x86_64-w64-  
  mingw32/13.2.0/;E:/project/programming_environment/mingw64/bin/  
  ../libexec/gcc/;E:/project/programming_environment/mingw64/bin/  
  ../lib/gcc/x86_64-w64-mingw32/13.2.0/../../../../x86_64-w64-  
  mingw32/bin/  
2 LIBRARY_PATH=E:/project/programming_environment/mingw64/bin/..  
  lib/gcc/x86_64-w64-  
  mingw32/13.2.0/;E:/project/programming_environment/mingw64/bin/  
  ../lib/gcc/;E:/project/programming_environment/mingw64/bin../l  
  ib/gcc/x86_64-w64-mingw32/13.2.0/../../../../x86_64-w64-  
  mingw32/lib/./lib/;E:/project/programming_environment/mingw64/  
  bin/./lib/gcc/x86_64-w64-  
  mingw32/13.2.0/../../../../lib/;E:/project/programming_environm  
  ent/mingw64/bin../lib/gcc/x86_64-w64-  
  mingw32/13.2.0/../../../../x86_64-w64-  
  mingw32/lib/;E:/project/programming_environment/mingw64/bin/..  
  lib/gcc/x86_64-w64-mingw32/13.2.0/../../../../  
3 COLLECT_GCC_OPTIONS='-v' '-o' 'abc.exe' '-mtune=generic' '-  
  march=x86-64' '-dumpdir' 'abc.'
```

```

E:/project/programming_environment/mingw64/bin/./libexec/gcc/x
86_64-w64-mingw32/13.2.0/collect2.exe -plugin
E:/project/programming_environment/mingw64/bin/./libexec/gcc/x
86_64-w64-mingw32/13.2.0/liblto_plugin.dll -plugin-
opt=E:/project/programming_environment/mingw64/bin/./libexec/g
cc/x86_64-w64-mingw32/13.2.0/lto-wrapper.exe -plugin-opt=-
fresolution=C:\Temp\userTemp\ccdbcrs9.res -plugin-opt=-pass-
through=-lmingw32 -plugin-opt=-pass-through=-lgcc -plugin-opt=-
pass-through=-lgcc_eh -plugin-opt=-pass-through=-lmoldname -
plugin-opt=-pass-through=-lmingwex -plugin-opt=-pass-through=-
lmsvcrt -plugin-opt=-pass-through=-lkernel32 -plugin-opt=-pass-
through=-lpthread -plugin-opt=-pass-through=-ladvapi32 -plugin-
opt=-pass-through=-lshell32 -plugin-opt=-pass-through=-luser32
-plugin-opt=-pass-through=-lkernel32 -plugin-opt=-pass-
through=-lmingw32 -plugin-opt=-pass-through=-lgcc -plugin-opt=-
pass-through=-lgcc_eh -plugin-opt=-pass-through=-lmoldname -
plugin-opt=-pass-through=-lmingwex -plugin-opt=-pass-through=-
lmsvcrt -plugin-opt=-pass-through=-lkernel32 -m i386pep -
Bdynamic -o abc.exe
E:/project/programming_environment/mingw64/bin/./lib/gcc/x86_6
4-w64-mingw32/13.2.0/././././././x86_64-w64-
mingw32/lib/./lib/crt2.o
E:/project/programming_environment/mingw64/bin/./lib/gcc/x86_6
4-w64-mingw32/13.2.0/crtbegin.o -
LE:/project/programming_environment/mingw64/bin/./lib/gcc/x86_
64-w64-mingw32/13.2.0 -
LE:/project/programming_environment/mingw64/bin/./lib/gcc -
LE:/project/programming_environment/mingw64/bin/./lib/gcc/x86_
64-w64-mingw32/13.2.0/././././././x86_64-w64-mingw32/lib/./lib
-
LE:/project/programming_environment/mingw64/bin/./lib/gcc/x86_
64-w64-mingw32/13.2.0/././././././lib -
LE:/project/programming_environment/mingw64/bin/./lib/gcc/x86_
64-w64-mingw32/13.2.0/./././././x86_64-w64-mingw32/lib -
LE:/project/programming_environment/mingw64/bin/./lib/gcc/x86_
64-w64-mingw32/13.2.0/././././ abc.o -lmingw32 -lgcc -lgcc_eh -
lmoldname -lmingwex -lmsvcrt -lkernel32 -lpthread -ladvapi32 -
lshell32 -luser32 -lkernel32 -lmingw32 -lgcc -lgcc_eh -
lmoldname -lmingwex -lmsvcrt -lkernel32
E:/project/programming_environment/mingw64/bin/./lib/gcc/x86_6
4-w64-mingw32/13.2.0/crtend.o
5 COLLECT_GCC_OPTIONS='-v' '-o' 'abc.exe' '-mtune=generic' '-
march=x86-64' '-dumpdir' 'abc.'
6
7 # COMPILER_PATH 调用main函数的程序路径
8 # LIBRARY_PATH 该调用程序需要的库文件路径
9 # ld.exe 链接器 特殊原因需要包装成collect2.exe
10 # 这个阶段才确定了操作数的位置，从而cpu能知道指令中操作数的置或者地址（调
    用其他函数的指令）
11 # 将多个文件组装在一起
12 # 当前cpu可以执行

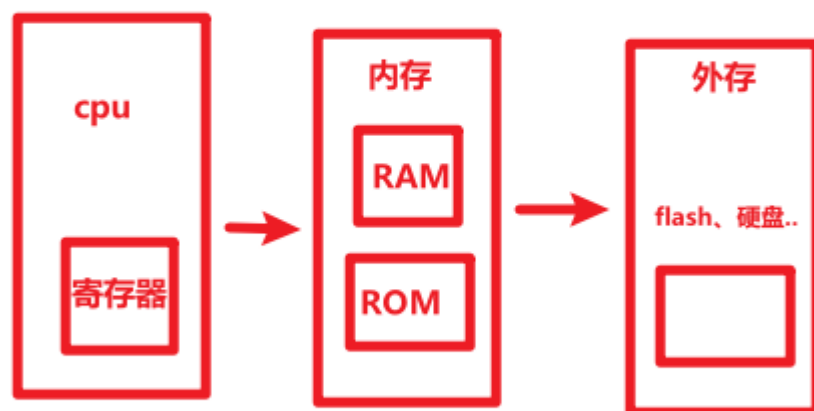
```

- 指令 = 操作码（动作） + 操作数（动作幅度）
 - CPU内部存储的是：**经常使用的操作码和操作数**，掉电会丢失
 - 大部分的操作码 和操作数存在硬盘上，当需要使用的時候，会从硬盘读到内存中
 -



- 计算机的工作方式

- 结构特点：CPU（寄存器）、内存（cpu 能直接访问。分为 ROM：掉电不易失、RAM：掉电易失）、外存（cpu 不能直接访问。掉电不丢失）
-



- 速度特点：寄存器 >> 内存 >> 外存 >> 网络
- 容量特点：寄存器 < 内存 < 外存

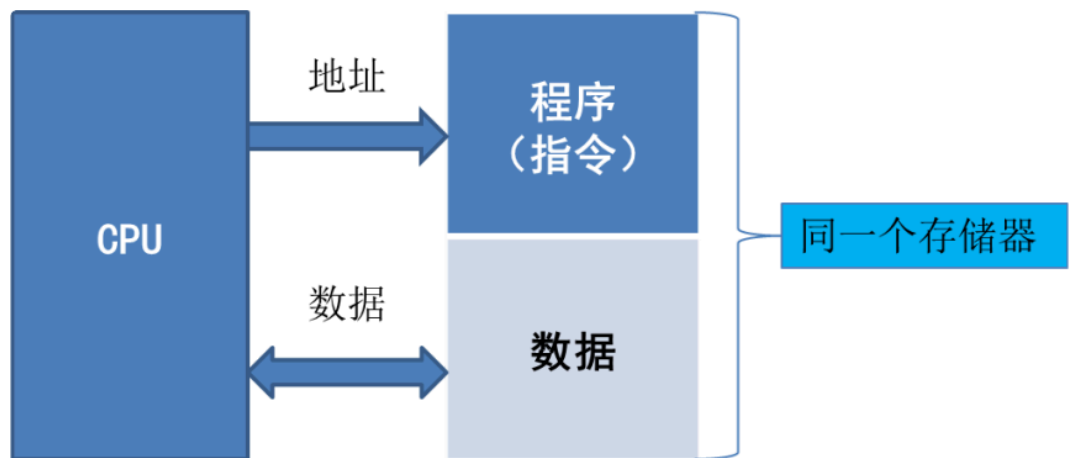
- 寻址方式：寄存器 —— 编号寻址，内存 —— 地址，外村 —— 块号（扇区号、柱面号）
- 主存中的缓存区指的是 行缓冲（并不是类似Cache那样的独立的缓存区，只是一块临时的内存区域）

维度	缓冲区 (Buffer)	缓存 (Cache)
主要目的	速度匹配：协调不同速度的设备/组件	加快访问：减少重复读取相同数据的开销
典型场景	I/O 操作（磁盘、网卡、打印机、视频播放等）	CPU 访存、网页、数据库、文件系统、分布式系统等
数据特征	数据通常 只用一次或短时间使用 （用完可丢）	数据 可能被反复使用 （热点数据、局部性原理）
管理方式	一般较简单（FIFO、循环缓冲、双缓冲、环形缓冲）	复杂策略（ LRU、LFU、FIFO、Random 等替换算法）
是否命中	一般不需要“命中”概念	有明显 命中率 (Hit Rate) 概念
是否独立存在	通常是内存中的一块临时区域 （也可硬件实现）	既有硬件实现，也有软件实现
例子	打印机缓冲区、视频播放缓冲、网卡发送/接收缓冲、管道缓冲	硬件： CPU L1/L2/L3 Cache 软件： 浏览器缓存、OS Page Cache、Redis、Nginx 缓存

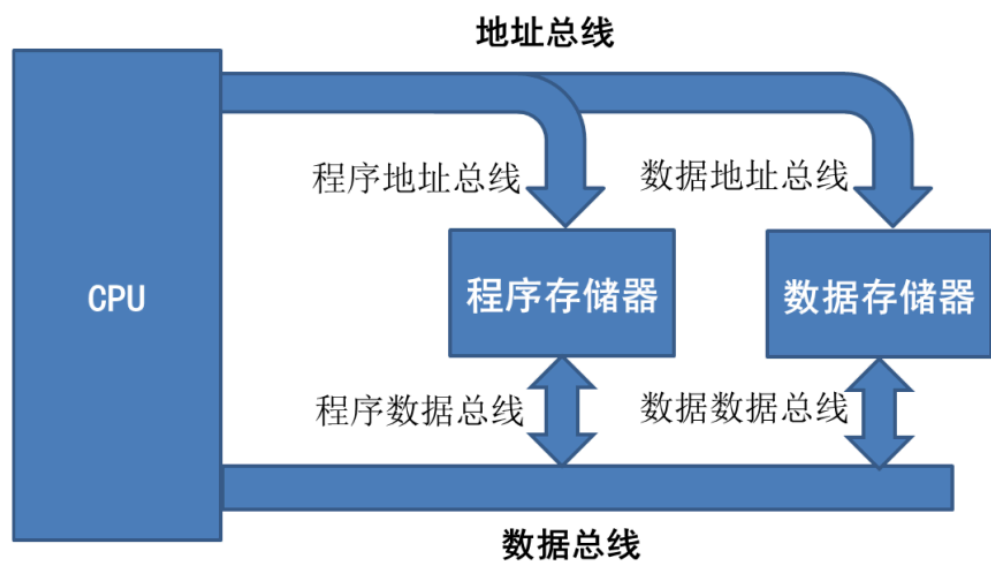
- 体系结构

	特点	指令执行
冯诺依曼结构	指令和数据放在同一片存储器中共用一套总线	串行执行
哈佛结构	指令和数据分开存放 指令总线 and 数据总线分开	并行执行（流水线技术）

■

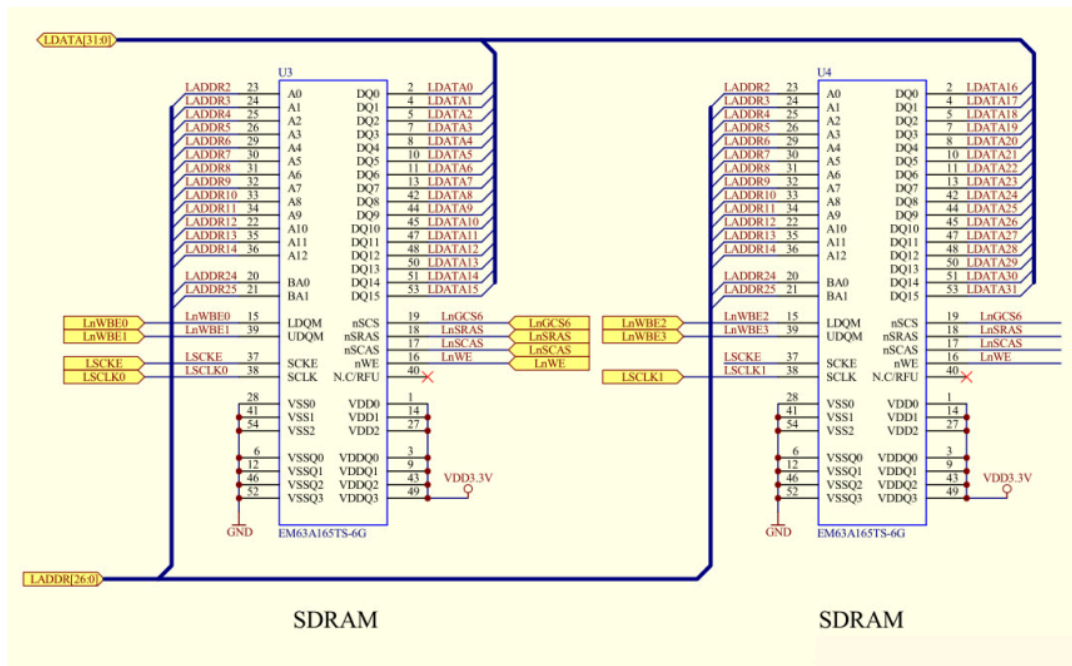


冯诺依曼结构



改进型哈佛结构

- 现代的计算机在宏观上使用冯诺依曼结构（指令数据放到同一块内存空间上——硬盘，使用系统总线通信），微观上使用哈佛结构（Cache中有指令Cache和数据Cache）



- 1 上图为什么CPU放出的[26:0]的地址线，从LADDR2开与SDRAM相接
- 2 cpu（本身是按找一个字节处理）发出
- 3 C程序有int a;
- 4 假如a的地址刚好是0x100，那么cpu就希望0x100~0x103都是a的内存空间
- 5 那么发送到SDRAM取值时，就应该以4个字节偏移，所有需要舍弃LADDR0和
- 6 LADDR1，而一片SDARM里面接收到的地址是0x100 >> 2 = 0x40，从这个地址上读2个字节，两个SDARM的结果组合成4B
- 7 0x100
- 8 0x101
- 9 0x102
- 10 0x103
- 11
- 12 cpu只想在发出0x104时，SDRAM才返回4B
- 13 0x104
- 14
- 15
- 16 如果想要cpu想要读取char类型，SDRAM依然返回4B，只是cpu只读最低一位的字节，存入cpu
- 17 如果想要cpu读取long long型，会发送两次地址，SDRAM返回两次结果

◦ CPU —— 按字节为单位

• C 语言的本质操作是，操作内存空间（盒子）

◦ 8 比特(bit) = 1 字节(byte，简称为 B) 比特是最小的数据单位

◦ $1\text{GB} = 2^{10}\text{MB} = 2^{20}\text{KB} = 2^{30}\text{B} = 2^{30} * 8\text{bit}$

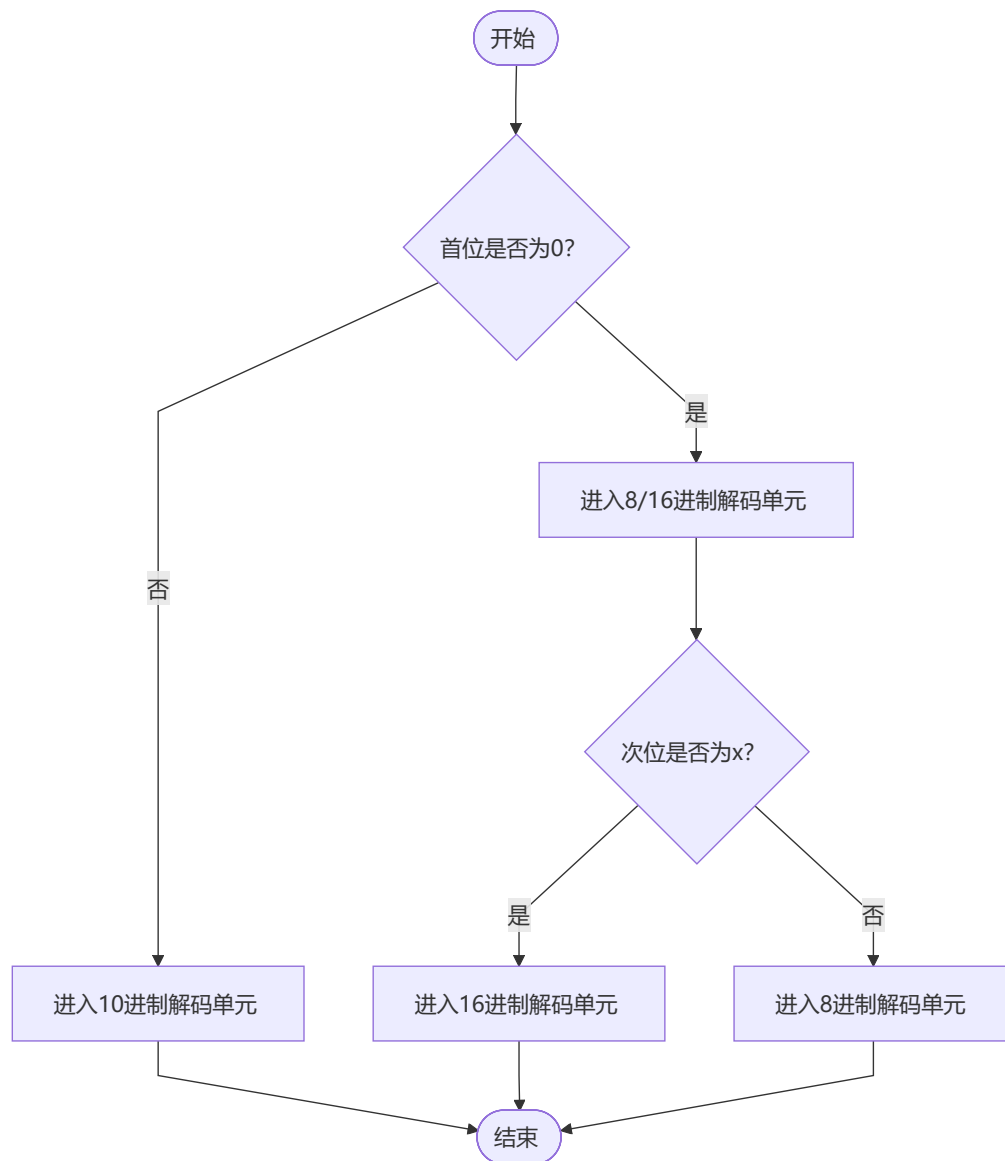
• 计算机中的数据表示

◦ 二进制 表示（无论是 数值型数据 还是 非数据值型数据）

• 进制转换

◦ 前导符（标识符）

- C对二进制解码规则



-
- 0b 是C14标准才加入的
-

前导符	代表的进制
0b	二进制
0	八进制
0x	十六进制

- 8421 方法 (简单常用)

二进制表示	数值
0x0000 0001	$2^0 = 1$
0x0000 0010	$2^1 = 2$
0x0000 0100	$2^2 = 4$

二进制表示	数值
0x0000 1000	$2^3 = 8$
0x0001 0000	$2^4 = 16$
0x0010 0000	$2^5 = 32$
0x0100 0000	$2^6 = 64$
0x1000 0000	$2^7 = 128$
0x0001 0000 0000	$2^8 = 256$
0x0010 0000 0000	$2^9 = 512$
0x0100 0000 0000	$2^{10} = 1024$
0x1000 0000 0000	$2^{11} = 2048$
0x0001 0000 0000 0000	$2^{12} = 4096$

- 将 100 转换成二进制: $100 = 64 + 32 + 4 \rightarrow 0b0110\ 0100$
 将 $0b0000\ 1010$ 转换成十进制: $0b0000\ 1010 = 0b0000\ 1000 + 0b0000\ 0010 = 8 + 2 \rightarrow 10$
 - 一位八进制对应三位二进制 互相转换也可以用 8421 方法
 - 一位十六进制对应四位二进制 互相转换也可以用 8421 方法
 - 其余方法: n 进制转二进制 —— 辗转相除法 二进制转 n 进制 —— 按位 $\times$ 权重, 再相加**
 - 16进制、8进制、2进制 单纯表示数据 (无负数)
 - 10进制 表示数字 (有数学意义 —— 有正负)
- 计算机中数据的存储
 - 计算机中存储都是数据的补码³
 - 正数与 0: 原码 = 反码 = 补码
 - 负数: 原码——最高位为 1 (符号位), 其余与正数原码相同; 反码——符号位不变, 其余按位取反; 补码——反码+1
 - 5 的原、反、补码: $0b\ 0000\ 0101$; -5 的原: $0b\ 1000\ 0101$ 、反: $0b1111\ 1010$ 、补: $0b1111\ 1011$ (该例子中原反补都为 8 位大小) (快速将原码 $\rightarrow$ 补码⁴)
 - $-(-5)$: -5 的补码, $0b1111\ 1011$; 在取它的补码 (取反 + 1), $0b0000\ 0101 = 5$
 - 非数值型的数据表示
 - 通过 [ASCII 码表](#), 将符号与数字对应 (a 为 97, A 为 65)
 - 其他字符编码集 (utf-8、GBK)
- 计算机中乘除运算
 - 对于 2 的乘数 ($\times 2$ 、 $\times 4$ 、 $\times 8 \dots$) 或者被除数 ($/2$ 、 $/4$ 、 $/8 \dots$): 计算机本质是通过 **移位** 完成。(乘法左移、除法右移)

- 对于非 2 的乘数 (×3、×5、×6...) 或者被除数 (/3、/5、/6...)：乘法本质是通过 移位+加法；除法是通过 硬件除法器 计算。
- 移位
 - 左移：有无符号位，右边空位补 0（会出现溢出情况）

```

1  #include <stdio.h>
2
3  int main()
4  {
5      char a = -65;
6      printf("%hhx %d\n",a,a);
7
8      a = a * 2;          // 左移溢出 会先将-65 * 2 变成int型计算，在变会
                           char型，实际的值为-（256-126） = -130 < -128 （低于最低的-128）
9
10     printf("%hhx %d\n",a,a);
11
12     return 0;
13 }
14
15 // 结果：
16 /*
17 bf  -65
18 7e  126
19 */

```

- 右移：有符号位，左边补符号位；无符号位，左边补 0

2. 基本数据类型

- 数据类型的本质是：大小和行为 ==> 在内存上划分多大的盒子来存储数据，其中是否需要特殊位（符号位）
=> 分配内存空间 以及 大小 运行之前就明确了空间大小
- 数据类型 ==> 分配空间的大小和行为（有无符号） ==> 分配的盒子多大，以及是否将最高位当作符号位。
- 基本数据类型的大小关系：
 - char < short <= int <= long < long long
 - 大小：1B 2B 2B/4B 不小于 4B 8B
 - 不同的操作系统中，char 一定为 1B，long long 一定为 8B
 - short 是 short int 简写，long 与 long long 分别为 long int 与 long long int 的简写
- 占位符
 - %为占位符，后面的字母表示使用的解码格式（printf会根据解码格式显示 相当于使用这种方式看内存中的数据）

```

1  // %d 以有符号十进制的形式查看（补码解码进行解析 转换成字符数字），默认大小为4B
2  // %u 以无符号十进制的形式查看（正权值解码进行解析），默认大小为4B
3  // %x 输出无符号整数的十六进制，字母使用小写(a-f),默认int 大小 , 4B
4  // %X 输出无符号整数的十六进制，字母使用大写(A-F)

```

```

5
6 // h 为half（一半）的简写
7 // %hhx 1B 16进制显示
8 // %hx 2B 16进制显示
9 // %hd 2B 10进制显示
10
11 // %c 根据字节，查找ASCII表，根据字体文件找到对应的位图，交给显示模块 char
12 // %s 输出字符串 通过传入的首地址，按照字符解码的方式，逐个解码，遇到0值（0值有 各种进制的0、'\0'）结束打印
13
14 // %zd 可变大小，查看size_t 类型，z表示size_t类型，d表示以4B大小的十进制显示 也常用sizeof的返回值
15 // %lu 以long u 的形式，即无符号整型的方式输出，具体大小由操作系统决定
16 // %llu 以long long u 的形式，即64位无符号整型的方式输出
17 // %llx 表示以十六进制形式打印一个长长整型（通常是 64 位）
18
19 // %p 以十六进制形式输出地址
20
21 // %5~ 宽度为5
22 // %05~ 以零填充

```

o

```

1 #include <stdio.h>
2 #include <stdint.h>
3
4 int main()
5 {
6     printf("sizeof(char): %d ",sizeof(char));
7     printf("sizeof(short): %d ",sizeof(short));
8     printf("sizeof(int): %d ",sizeof(int));
9     printf("sizeof(long): %d ",sizeof(long));
10    printf("sizeof(long long): %d\n",sizeof(long long));
11
12    // 使用size_t型,需要包含 #include <stdint.h>
13    printf("sizeof(int8_t): %zd ",sizeof(int8_t));
14    printf("sizeof(int16_t): %zd ",sizeof(int16_t));
15    printf("sizeof(int32_t): %zd ",sizeof(int32_t));
16    printf("sizeof(int64_t): %zd\n",sizeof(int64_t));
17
18    return 0;
19 }
20
21 // 结果:
22 /*
23 sizeof(char): 1 sizeof(short): 2 sizeof(int): 4 sizeof(long): 4
24 sizeof(long long): 8
25 sizeof(int8_t): 1 sizeof(int16_t): 2 sizeof(int32_t): 4 sizeof(int64_t): 8
26 */

```

- 防止不同操作系统中 基本数据类型的大小不一样，封装了 <stdint.h>，定义了固定大小的数据类型⁵

- 初始化本质

- ```

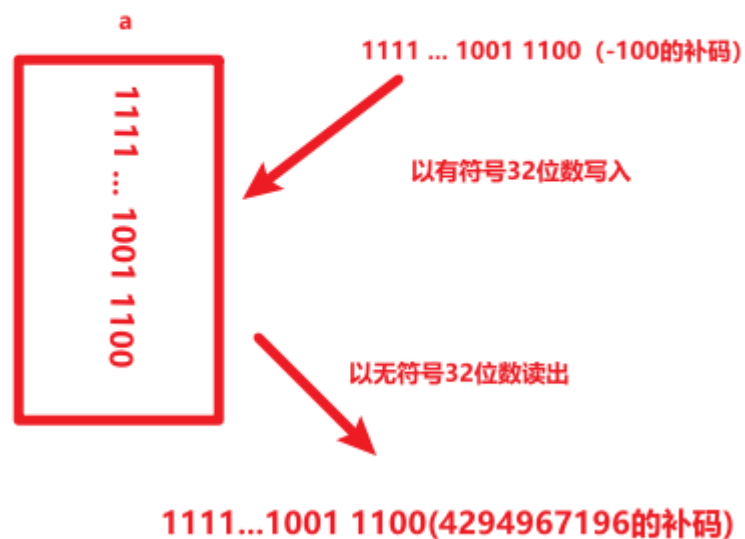
1 char a = -100;
2 // 本质上：程序先将 -100 以32位大小的二进制嵌入到存储区 .rodata （常量区）或直接作为指令立即数（指令中的一个固定数值--嵌入代码区.text对应的指令），在开辟一个char（8bit）大小的空间命名为a，将-100的32位二进制补码的低8位拷贝到a空间中。
3 // 100（常量）默认表示为int型
4
5 // 字面常量（100、3.14...）默认以int大小嵌入程序中，可以通过 字面常量+（L/LL/UL...）修改嵌入大小。
6 // eg. char a = -100LL; （-100以64位有符号二进制嵌入程序中）

```
- 程序中的存数据和取数据不会相互影响，影响的只有存或者取时的行为（以有无符号、多少位进行存取）

- ```

1 int a = -100;
2 printf("%u",a); // 使用%u的眼镜观察a空间
3
4 // 打印出的结果为4,294,967,196，但a存储的仍然为-100的补码，只是读取a的内容时，将-100的补码看成没有符号的数值以整型输出。

```



- 大容量拷贝给小容量：小容量从大容量的低位开始 **截断** 自己的长度。（大容量：指代数据类型大小比较大的变量；小容量：指代数据类型大小比较小的变量）
- 小容量拷贝给大容量：大容量多余空间 **补齐** ——小容量有符号，填充符号位的数值；无符号，填充 0（根据小空间的约束 来补全大空间的空位置）

- ```

1 #include <stdio.h>
2
3 int main()
4 {

```

```

5 // 大容量拷贝给小容量
6 char a = 257; // 257以32位形式嵌入程序（大容量），a只有8位大小
7
8 printf("a:%d\n",a); // a中存储的是1
9
10 // 小容量拷贝给大容量
11 // 小容量有符号
12 // 小容量有符号
13 char a1 = 1; // a1、a2只有8位大小
14 char a2 = -1;
15 unsigned int b = a1; // b、c有32位大小
16 unsigned int c = a2; //
17
18 printf("b:%u\n",b); // 以无符号整型查看，b获取的数值依然为
19 1 b中空余位置填充了符号位0
20 printf("c:%u\n",c); // 以无符号整型查看，c获取的数值为
21 4294967295 c中空余位置填充了符号位1
22
23 // 小容量无符号
24 unsigned char a3 = 1;
25 unsigned int d = a3;
26
27 printf("d:%u\n",d); // 以无符号整型查看，d的值为1，d中空余
28 位置填充了0
29
30 return 0;
31 }
32
33 // 结果：
34 /*
35 a:1
36 b:1
37 c:4294967295
38 d:1
39 */

```

o unsigned 无符号类型；signed 有符号

- 本质：unsigned => 使用正权值进行解码 signed => 使用补码进行解码

- ```

1 int a; // 划分了一个4B的空间，用a来标识它。如何操作这32bit的空间 用补码
  解码方式来操作
2
3 unsigned int b; // 划分了一个4B的空间，用b来标识它。如何操作这32bit的空间
  用全正权值方式来操作
4
5 a > b // 比较运算符 <、>、<= ... 会按照空间约束(转换成无符号类型)进行比较

```

■

- 一个有符号与一个无符号相运算时，编译器会将有符号转换成无符号进行计算。（本质向大容量转换）

- C语言大空间 和 小空间 比较或者计算，都是转换成大空间
 - 当内存空间当作数字（数学含义），就用signed
 - 当内存空间当作数据（没有数学含义），就用unsigned
- 字符：' '单引号包裹，编译器才能识别成字符，否则会当成变量名。" "双引号包裹字符串。
- 字符串中有一个隐藏的'\0'，标识该字符串结束
 - "abc"—> "abc\0"

```

1  #include <stdio.h>
2
3  int main()
4  {
5      char a = '7';
6
7      printf("%x\n",a);           // a实际存储的数据: 0x37 对应字符'7'
8      printf("%d\n",a);           // 将a的数据以整型显示为55
9      printf("%d\n",a - '0');      // 显示出数字7，减去字符'0'的偏移
10
11     return 0;
12 }
13
14 // 字符数字 与 数字并不一样

```

- 转义字符\：改变原本字符的意义
- 自定义数据类型
 - 内存累加（有序）—— 结构体
 - 内存共享 —— 共用体
 - 枚举 —— 内部是普通的整型类型 比较鸡肋 赋值时可以乱写
- 常量表达式 (3600 * 24 * 365)ULL 在编译时就计算完成，ULL表示unsigned long long类型
- 定义与初始化区别
 1. 定义：需要分配的内存空间大小和行为（有无符号）
 2. 初始化：定义 + 赋予初始化值（在分配空间的同时，将数据放入这个空间）
- 变量取名规范
 - `_t`：已经被typedef定义，从普通数据类型变成有一定意义的类型
 - `uint8_t`：无符号的字节类型，数据空间的单位（1B）
 - `size_t`：内存字节单位 B
 - `time_t`：时间数据类型

3. 宏

- 预处理阶段 时编译器进行处理，“#”开头的均表示预处理命令

3.1 条件编译

- 条件编译
 - `#if ... #else ... #endif`
 - `#if` 用来判断一个条件表达式是否为真，如果为真，则编译相应的代码块。
 - 如果使用 `#elif` (else if 的意思)，则可以为条件提供多个分支。
 - `#ifdef` 和 `#ifndef`
 - `#ifdef`：用于判断一个宏是否已经被定义。
 - `#ifndef` 用于判断一个宏是否没有被定义
 - `#define` 和 `#undef`
 - `#define`：用于定义宏。
 - `#undef`：用于取消宏定义。
- 运用场景：不修改代码，区分debug版本和release版本

```
1  #include <stdio.h>
2
3  int main(){
4      #ifdef DEBUG
5          printf("这是debug版本，需要控制台输出");
6      #else
7          printf("这是release版本，没有测试代码");
8      #endif
9
10     return 0;
11 }
```

```
1  gcc test.c -o test.exe -DDEBUG # 这是debug版本
2  gcc test.c -o test.exe # 这是发行版本
```

◦

3.2 宏替换

- 宏定义 (`#define`) 本质上是 **文本替换**。不占用内存，编译器不会检查
 - C 语言的全局常量，用 `const` 关键字，且要占内存、在编译阶段会检查
 - 全局变量，编译阶段确认其值；
 - 局部变量。运行时确认其值
- 作用：**文本替换**、**选择性的添加代码**（将 `printf` 等调式语句隐藏）

3.3 宏函数

- 不带参数：`#define` 宏名 **【字符串 或者 常量】**
- 带参数：`#define` 宏名(形参) 替换文本 **替换文本中的形参应该使用()包裹起来**

```
1  #include <stdio.h>
```

```

2
3 #define MAX(a,b) ((a) > (b) ? (a) : (b)    // 带参数的
4 // 不带参数 #define MYNUM 1
5
6 int main()
7 {
8
9     #ifdef MYNUM                                // 如果定义了MYNUM，则打印MYNUM的值
10         printf("MYNUM:%d\n",MYNUM);
11     #endif
12
13     #ifndef MYNUM                                // 如果没有定义了MYNUM，则打印NO MYNUM
14         printf("NO MYNUM\n");
15     #endif
16
17     #if MYNUM == 1                                // MYNUM等于1，则打印a>b
18         printf("a>b\n");
19     #endif
20
21     int a = 20;
22     int b = 10;
23     printf("MAX:%d\n",MAX(a,b));
24
25     return 0;
26 }
27
28 /*
29 执行结果
30 NO MYNUM
31 MAX:20
32 */

```

- ```

1 > gcc .\day02\test03.c -o .\day02\test03
2 > .\day02\test03.exe
3 NO MYNUM
4 MAX:20
5
6 # 通过-D 宏定义MYNUM
7 > gcc .\day02\test03.c -D MYNUM=1 -o .\day02\test03
8 > .\day02\test03.exe
9 MYNUM:1
10 a>b
11 MAX:20
12
13 # 该效果经常用于调式，在打印的语句前面添加#ifdef ，通过-D来选择是否显示 或者 使用
 #define与#undef来选择

```

- 宏中的# 与 ##:

- “#” 将输入的形参变成 字符串
- “##” 将形参拼接

- ```

1 #include <stdio.h>
2 #include <string.h>

```

```

3
4 #define AAA(b) #b
5 #define BBB(a,b) a ## b
6 #define CCC(a,b,c) c ## b ## a
7
8 int main()
9 {
10
11     int num = 1;
12
13     // #
14     printf(" #: %s\n",AAA(aaa));           // 将aaa转换成字符串"aaa"
15     printf(" #: %s\n",AAA(num));           // 将num转换成字符串"num"
16
17     // ##
18     printf(" ##: %d\n",BBB(1,2));           // 将1和2拼接到一起，组成12
19     printf(" ##: %d\n",BBB(n,um));         // 将u和um拼接到一起，组成num，
    且具有原本num的属性，即会打印1
20
21     printf(" ##: %d\n",CCC(m,u,n));         // 拼接会根据宏定义语序进行拼接，
22     // printf(" ##: %d\n",CCC(m,m,n));     // 如果拼接的结果不是常量或者已经
    有的变量，则在编译阶段出现错误
23
24 }
25
26 /* 结果
27  #: aaa
28  #: num
29  ##: 12
30  ##: 1
31  ##: 1
32 */

```

4. 结构体

- 语法:

```

1 // 声明
2 struct 结构体名 {
3     数据类型1 成员名1;
4     数据类型2 成员名2;
5     ...
6 };
7
8
9 // 定义
10 struct 结构体名 结构体变量名;
11
12 // 使用
13 // 普通变量
14 结构体变量名.成员名;

```

```

15 // 结构体指针变量
16 结构体指针变量名->成员名; // 成员地址: 起始地址 (结构体指针变量 存的内容) + 偏移量
17

```

```

o 1 // 1.
2   struct people{
3       int m_age;
4       char *m_name;
5   }my_people; // 声明struct people的变量 my_people 如果需要声明
                其他变量需要使用 struct people 变量名;
6
7   // 2.
8   struct {
9       int m_age;
10      char *m_name;
11  }my_people; // 声明了一个匿名结构体的变量

```

```

o 1 typedef struct stu{
2     int id;
3     char *name;
4     char sex;
5 } Stu;
6
7 /* 定义 */
8 struct stu mystu1; // 标准定义一个结构体变量 使用 struct 结构体名 */
9 mystu1.id = 1;
10 mystu1.name = "你好";
11 mystu1.sex = 'b';
12
13 /* 初始化1 */
14 Stu mystu2 = { // 使用别名进行定义 */
15     2,
16     "aa",
17     'g'
18 }; // 这个方式必须按照它的变量顺序进行初始化 */
19
20 /* 初始化2 */
21 Stu mystu3 = { // 这是GNU的g++提供的初始化方式 */
22     .id = 2, // 未初始化的变量, 默认为0 */
23     .name = "bb"
24 };

```

- o
- 结构体开辟内存大小
 - o 结构体对齐原则:
 1. 结构体的首地址 = 第一个成员的首地址
 2. 成员对齐: 每个成员的首地址 = 结构体起始地址 + 偏移量 (min(自身对齐值, 指定对齐值))
 3. 总体对齐: 结构体总体大小 必须 能够被 min(所以成员中最大的自身对齐值, 指定对齐值)整除
 - 注意:

min(自身对齐值, 指定对齐值): 自身对齐值与指定对齐值中最小的一个值。

自身对齐值: 该成员的数据类型大小。

指定对齐值:

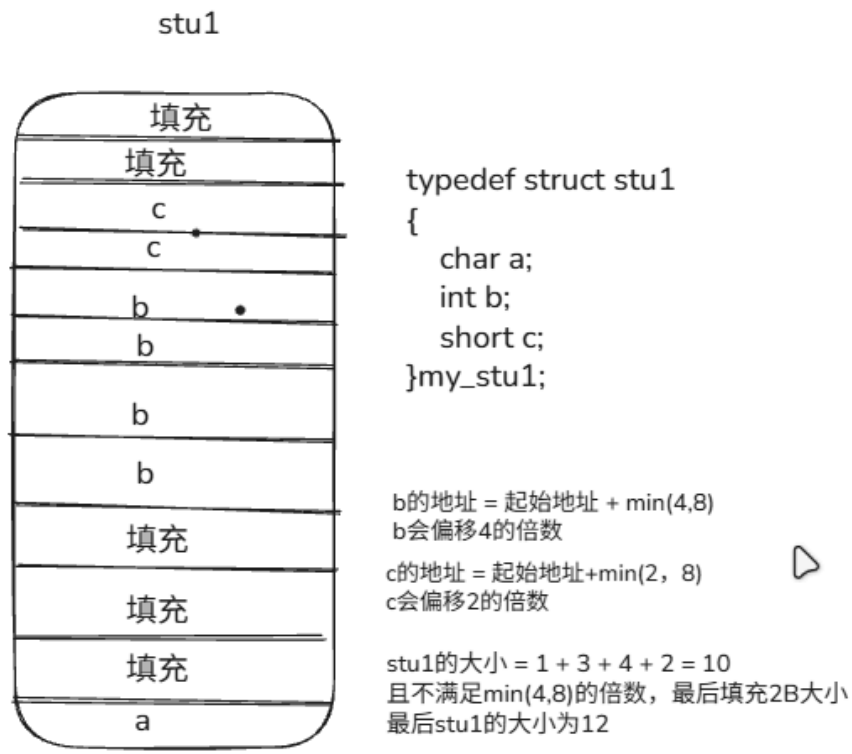
默认 32 位 Linux 的指定对齐值为 4; 64 位的指定对齐值为 8; ARM CPU 上指定对齐值默认为 8; 32/64 位 windows 默认 8 字节;

可以通过宏 `#pragma pack(N)` 设定, $N = 2^n$ (n 为 1, 2, 3....)

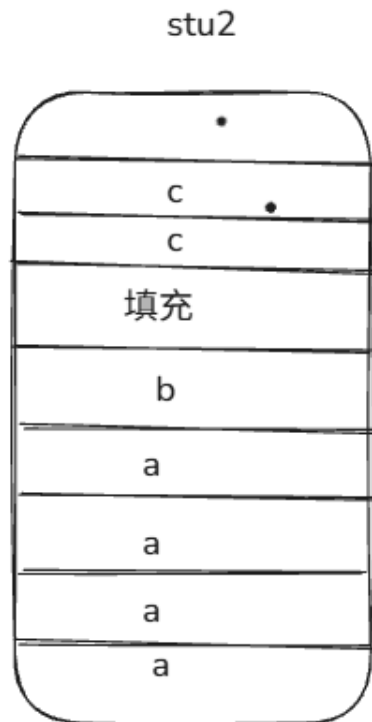
```
1  #include <stdio.h>
2
3  // 结构体对齐方式
4  // stu1 的总体大小为 12
5  typedef struct stu1
6  {
7      char a;
8      int b;
9      short c;
10 }my_stu1;
11
12 // stu2 的总体大小为 8
13 typedef struct stu2
14 {
15     int a;
16     char b;
17     short c;
18 }my_stu2;
19
20 // stu3 的总体大小为 12
21 typedef struct stu3
22 {
23     short a;
24     int b;
25     char c;
26 }my_stu3;
27
28 int main()
29 {
30     my_stu1 a1;
31     my_stu2 a2;
32     my_stu3 a3;
33
34     // 结构体名 总体大小 第一成员的首地址 第二成员的首地址 第三成员的首地址
35     printf("name\t\t size\t\t 1\t\t 2\t\t 3\t\t\n");
36     printf("stu1\t\t %d\t\t %x\t\t %x\t\t %x\t\t\n", sizeof(a1), &a1.a, &a1.b, &a1.c);
37     printf("stu2\t\t %d\t\t %x\t\t %x\t\t %x\t\t\n", sizeof(a2), &a2.a, &a2.b, &a2.c);
38     printf("stu3\t\t %d\t\t %x\t\t %x\t\t %x\t\t\n", sizeof(a3), &a3.a, &a3.b, &a3.c);
39
40     return 0;
```

```
41 }
42
43 // 结果
44 /*
45 name          size          1          2
46   3
47 stu1          12          7b9ffbe4
48 7b9ffbe8          7b9ffbec
49 stu2          8          7b9ffbec
50 7b9ffbe0          7b9ffbe2
51 stu3          12          7b9ffbd0
52 7b9ffbd4          7b9ffbd8
53 */
```

■ 内存框图 (windows 默认指定对齐值为 8) :



■

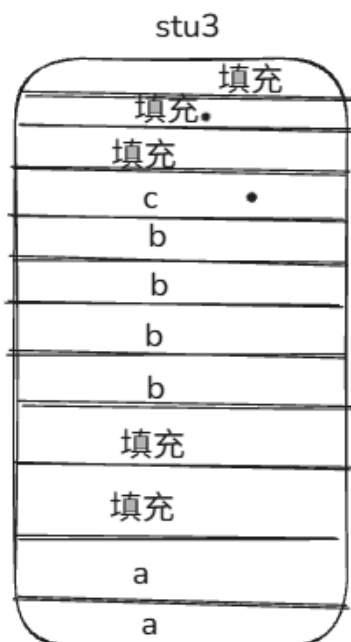


```
typedef struct stu2
{
    int a;
    char b;
    short c;
}my_stu2;
```

b的地址 = 起始地址 + min(1,8)
b会偏移1的倍数

c的地址 = 起始地址+min(2, 8)
c会偏移2的倍数

stu2的大小 = 4 + 1 + 1 + 2 = 8
且满足min(4,8)的倍数,
最后stu2的大小为8



```
typedef struct stu3
{
    short a;
    int b;
    char c;
}my_stu3;
```

b的地址 = 起始地址 + min(4,8)
b会偏移4的倍数

c的地址 = 起始地址+min(1, 8)
c会偏移1的倍数

stu3的大小 = 2 + 2 + 4 + 1 = 9
不满足min(4,8)的倍数, 最后填充3B大小
最后stu3的大小为12

■ #pragma pack(N)使用:

```
1  #include <stdio.h>
2
3  // 结构体对齐方式
```

```

4 // stu1 的总体大小为 7
5 #pragma pack(1)
6 typedef struct stu1
7 {
8     char a;
9     int b;
10    short c;
11 }my_stu1;
12 #pragma pack() // 恢复默认的指定对齐值
13
14 // stu2 的总体大小为 8
15 #pragma pack(2)
16 typedef struct stu2 // 指定对齐值只会影响首地址，
17                     // 其实际开辟的内存空间还是数据类型决定
18 {
19     char a;
20     int b;
21     short c;
22 }my_stu2;
23 #pragma pack()
24
25 // stu3 的总体大小为 12
26 typedef struct stu3
27 {
28     char a;
29     int b;
30     short c;
31 }my_stu3;
32
33 int main()
34 {
35     my_stu1 a1;
36     my_stu2 a2;
37     my_stu3 a3;
38
39     // 结构体名 总体大小 第一成员的首地址 第二成员的首地址 第三成员
40     // 的首地址
41     printf("name\t\t size\t\t 1\t\t 2\t\t 3\t\t\t\n");
42     printf("stu1\t\t %d\t\t %x\t\t %x\t\t %x\t\t\t\n", sizeof(a1), &a1.a, &a1.b, &a1.c);
43     printf("stu2\t\t %d\t\t %x\t\t %x\t\t %x\t\t\t\n", sizeof(a2), &a2.a, &a2.b, &a2.c);
44     printf("stu3\t\t %d\t\t %x\t\t %x\t\t %x\t\t\t\n", sizeof(a3), &a3.a, &a3.b, &a3.c);
45
46     return 0;
47 }
48
49 // 结果
50 /*
51 name          size          1          2
52          3

```

50	stu1	7	98fff699
	98fff69a		98fff69e
51	stu2	8	98fff690
	98fff692		98fff696
52	stu3	12	98fff684
	98fff688		98fff68c
53	*/		

- 位域

- 用于给结构体 / 联合体的成员分配特定 **bit** 来存储数据。**节省内存**，不能对位域变量取地址
- 使用场景：协议（增加可读性）
- 语法：数据类型 变量名:分配bit位数； （尽量使用无符号数）
- 对位域变量进行取地址操作

```

1  #include <stdio.h>
2  #include <string.h>
3
4  // #define ABC
5
6  // 尽量使用无符号的数据类型进行操作，有符号的数据类型会默认占用一个符号位，从而有
   溢出现象(截断)
7  #ifdef ABC
8  typedef struct work{
9      int a : 2;      // a的大小为1bit
10     int b : 2;      // b的大小为3bit
11     int c : 4;      // c的大小为4bit
12 }my_work;
13 #endif
14
15 #ifndef ABC
16 typedef struct work{
17     unsigned int a : 4;      // a的大小为4bit
18     unsigned int b : 4;      // b的大小为4bit
19 }my_work;
20 #endif
21
22 int main()
23 {
24     #ifdef ABC
25     my_work w = {1,1,7};
26     printf("%d %d %d\n",w.a,w.b,w.c);
27     #endif
28
29     #ifndef ABC
30     my_work w;
31     // printf("%x,%x,%x,%x",&w.a,&w.b,&w.c,&w.d);      // 位段不能进行
   取地址操作
32     memset(&w,0,sizeof(w));      // 将该结构体内
   存初始化为0（包括那些填充的区域）
33
   // 该函数头文件
   为#include <string.h>
34     printf("%x %x\n",w.a,w.b);

```

```

35     w.a = 0x74;
36     printf("%x %x\n",w.a,w.b);           // 实际上两个的
    空间完全隔离，即 w.a = 0x4 ,w.b = 0
37     #endif
38
39     return 0;
40 }
41
42 // 结果
43 /*
44 0,0
45 5,0
46 */
47
48 /*
49 #pragma pack(n)           // 设置全局对齐值为 n (1,2,4,8,16)
50 #pragma pack()           // 恢复默认对齐值
51 #pragma pack(push, n)    // 保存当前对齐值，并设置为 n
52 #pragma pack(pop)        // 恢复最近保存的对齐值
53 #pragma pack(show)       // 编译时显示当前对齐值 (MSVC)
54 */

```

5. 联合体（共用体）

- 共用体里的成员：共用同一块内存。共用体占用空间大小为最大的成员变量所需的空间
- 使用：与结构体用法一样

```

1 // 声明
2 union 共用体名 {
3     数据类型1  成员名1;
4     数据类型2  成员名2;
5     ...
6 };
7
8 // 定义
9 union 共用体名 共用体变量名;
10
11 // 使用
12 // 普通变量
13 共用体变量名.成员名;
14 // 共用体指针变量
15 共用体指针变量名->成员名; // 成员地址：起始地址（共用体指针变量 存的内容） + 偏移量
16

```

- 共用体的实际使用(判断cpu的大小端)

```

1 #include <stdio.h>
2 #include <string.h>
3
4 // 判断cpu为大端模式（Big endian）还是小端模式（Little endian）
5 // 大端模式：高地址存数据的低位，低地址存数据的高位；

```

```

6 // 小端模式：高地址存数据的高位，低地址存数据的低位；
7 typedef union {
8     int a;
9     char b;
10 }my_endianUnion;
11
12
13 int main()
14 {
15     my_endianUnion m;
16     memset(&m,0,sizeof(m)); // 初始化共存体内存
17     m.a = 1; // 通过a写入1个字节
18
19
20     if(m.b == 1) // b(1字节)从中读取低地址的值
21         printf("Little endian!\n");
22     else
23         printf("Big endian!\n");
24     printf("%d",sizeof(m)); // 内存大小为 int的大小 (4)
25
26     return 0;
27 }
28 // 结果
29 /*
30 Little endian!
31 4
32 */

```

6. 符号

- 常用分类
 - 一目（元）运算符：~ 操作数
 - 二目运算符：+、-、*、...
 - 三目运算符：判断语句 ? 结果1 : 结果2
- 优先级（简单记法）：!(逻辑非→判断真假) > 算术运算符 (+、/、*、-、%、++、--) > 关系运算符 (>、<、>=、==、<=、!=) > && > || > 赋值运算符 (=、&=、|=、%=、>=...)
- 异或 运用加密
 - 本质： $a = a \oplus 0$;
 - 加密数据 = 原始数据 $\oplus$ 密钥（自己规定的数字 或者 字符）
 - 原始数据 = 加密数据 $\oplus$ 密钥 = 原始数据 $\oplus$ 密钥 $\oplus$ 密钥 = 原始数据 $\oplus 0$
- 基本运算：
 - / (除法)：结果是取商。 $5 / 2 === \rangle 2$
 - % (取模)：结果是取余。 $5 \% 2 === \rangle 1$
 - ++ / -- (自增/自减)

```

1  #include <stdio.h>
2
3  int main()
4  {
5      int num = 1;
6      int a,b;
7
8      // ++num 先++,后返回数值
9      // num = num + 1;
10     // a = num;
11     a = ++num;      // a = 2,num = 2
12     printf("a=%d,num=%d\n",a,num);
13
14     // num++ 先返回数值,后++
15     // b = num;
16     // num += 1; <==> num = num + 1;
17     b = num++;      // b = 2,num = 3
18     printf("b=%d,num=%d\n",b,num);
19
20     // num--与--num 同理
21     return 0;
22 }
23
24 // 结果
25 /*
26 a=2,num=2
27 b=2,num=3
28 */

```

- 存地址的变量可以用 `++ --` ; 数组名不能 `++ --` (数组名是不可修改的值) (常量不能写在等号左边)

- 位运算:

- 芯片的物理接口(交流的通道)是由引脚构成(1根引脚为1位或者1bit) 程序是通过寄存器(某一位引脚通常依附于一个寄存器,该寄存器大小通常不小于1B)控制引脚
- 计算机内存一般是按字节编址,所以一个bit位没有地址,如果需要取bit的值,需要位运算
- ~ (取反——按位取反)、& (位与——全真为真)、| (位非——全假为假)、<< (左移)、>> (右移)、^ (异或——不同为真)、- (取相反数,不够需要处理空间上限)

- 小技巧

1. 置0用& 置1用|
2. &: 常用于提取某些位,屏蔽某些位,某位置0
3. |: 常用于某位置1

- 某个引脚置为1

```

1  /* 对a变量的5bit的位设置为1 */
2  int a;
3  a |= (0x1 << 5);

```

- 某个引脚置为0

- ```

1 /* 对a变量的5bit的位设置为0 */
2 int a;
3 /* a &= 0xFFDF; // 可移植性较差 */
4 a &= ~(0x1 << 5);

```

- 某几个连续的引脚设置为 某个特定的值

- ```

1  /* 对[8:4]位设置成 10110 其他位不变 */
2  int a;
3  a &= ~(0x1F << 4); /* 先清零 */
4  a |= (0x16 << 4); /* 在置1 [8:4]已经全是0的情况下 */

```

- 多组连续的引脚设置为特定值

- ```

1 /* 对[10: 8]置为100 , [4:3]置为10 */
2 int data;
3
4 data &= ~(0x7 << 8 | 0x3 << 3); /* 清零 */
5 data |= (0x4 << 8 | 0x2 << 3);

```

- 设备中收到了一个int大小的数据data，判断第6bit是否为1

- ```

1  int data;
2  /* 数据的第6位是否为1 */
3  /* 不能写成 data & (0x1 << 6) == 1 */
4  if(data & (0x1 << 6)){
5      /* 处理代码 */
6  }
7
8  /* 数据的第6位是否为0 */
9  if(!(data & (0x1 << 6))){
10     /* 处理代码 */
11 }

```

- 示例:

- ```

1 #include <stdio.h>
2
3 int main()
4 {
5 int a = 0b10011010;
6
7 /* 将a的第3位bit置为0。 置0 选择& 0b1001 1010 => 0b1111 0111位与 =>
 ~(0b0000 1000) => ~(0b0000 0001 << 3)
8 int a1 = a & ~(0x1 << 3);
9 printf("a1:%hhx\n",a1); // a1=0x92 0b1001 0010
10

```

```

11 // 将a的第5位bit置为1. 置1 选择| 0b1001 1010 => 0b 0010 0000位或 =>
 0b0000 0001 << 5
12 int a2 = a | (0x1 << 5);
13 printf("a2:%hhx\n",a2); // a2=0xba 0b1011 1010
14
15 // 将a的第3位到第5位置成101. 多位置位 先把目标部分置0, 再置1
16 int a3 = (a & ~(0x7 << 3)) | (0x5 << 3);
17 printf("a3:%hhx\n",a3); // a3=0xaa 0b1010 1010
18 return 0;
19 }
20
21 // 结果
22 /*
23 a1:92
24 a2:ba
25 a3:aa
26 */
27
28 // 多位置位 先把目标部分置0, 再按需求置1

```

#### ○ C语言的形参, 从右向左运算

```

1 #include <stdio.h>
2
3 int main(){
4 int a = 0x1;
5 int b = 0x1;
6
7 printf("b = %d\n",b); // b = 1
8
9 // C语言中的传参, 是从右往左进行的
10 printf("%d %d\n",a & b, a && ++b); // 错误印象: 先 (a &
11 printf("b = %d\n",b); // 实际上是: 先 (a &&
12 ++b) 后 (a & b) ==> 0 1 b = 2
13
14 printf("%d %d\n",a | b, a || b++); //先 (a || b++) 后
15 (a | b) ==> 3 1 b = 2 其中a || b++ 执行到a就结束了
16 printf("b = %d\n",b);
17 printf("%d %d\n", b , ++b);
18
19 a = 0x1,b =0x1;
20 printf("b = %d\n",b); // b = 1
21 printf("%d\n",a & b++); // 1
22 printf("b = %d\n",b); // b = 2
23 printf("%d\n",a & ++b); // 1
24 printf("b = %d\n",b); // b = 3
25
26 return 0;
27 }
28
29 // 结果
30 /*

```

```

30 b = 1
31 0 1
32 b = 2
33 3 1
34 b = 2
35 b = 1
36 1
37 b = 2
38 1
39 b = 3
40 */

```

○ 三种求空间中bit1的个数

1. 与 0x1 位与，判断是否为0

```

1 #include <stdio.h>
2
3 // 求指定内存中bit 1的个数
4 // 1.与 0x1 位与，判断是否为0
5 int main(){
6 int num = 0x89763432;
7 int cut = 0;
8
9 for(int i = 0; i < sizeof(num) * 8; i++){
10 if((num & (0x1 << i)) != 0){
11 cut++;
12 }
13
14 // 错误写法：错误原因 (num & (0x1 << i))除了第一次可能为1以
 外，其余结果都是2的倍数
15 // if((num & (0x1 << i)) == 1){
16 // cut++;
17 // }
18
19 // 修改：
20 // if((num & (0x1 << i)) == (0x1 << i)){
21 // cut++;
22 // }
23 }
24 printf("%d",cut);
25
26 }
27
28 // 结果
29 // 14

```

2. 原始数据 右移 后与 0x1位与，判断是否为1

- 使用时，应该先放入 无符号的空间 中进行右移

```

1 #include <stdio.h>
2

```

```

3 // 求指定内存中bit 1的个数
4 // 2.原始数据 右移 后与 0x1位与, 判断是否为1
5 int main(){
6 int num = 0x89763432;
7 // int num1 = num;
8 unsigned int num1 = num;
9 int cut = 0;
10
11 // 使用int num1 进行处理, 可能会出现死循环 num1每次右移后, 填充的
 位符号位, 如果符号位为1, 则死循环
12 // 修改: 使用unsigned int num1接受数据
13 while (num1 != 0){
14 if((num1 & 0x1) == 1){
15 cut++;
16 }
17 num1 = num1 >> 1;
18 }
19
20 printf("%d",cut);
21
22 }
23 // 结果
24 // 14

```

3. 减法（相减不足会借位），进入循环默认不是0（存在1），每次-1与自身位1,是为了让自身变成0, 退出循环

```

1 #include <stdio.h>
2
3 // 求指定内存中bit 1的个数
4 // 3.减法（相减不足会借位），进入循环默认不是0（存在1），每次-1结果与自身
 位与,如果自身变成0就表示没有1了, 退出循环
5 int main(){
6 int num = 0x89763432;
7 int cut = 0;
8
9 while (num){
10 cut++;
11 num = num - 1 & num; // 每次消去一个bit1, 与自身位与
 （消除一个原本的一个1）
12 }
13
14 printf("%d",cut);
15
16 }
17 // 结果
18 // 14
19 // 0b 1000 0001
20 // -1 ==> 0b 1000 0000
21 // &自身 ==> 0b 1000 0000
22 // -1 ==> 0b 0111 1111
23 // &自身 ==> 0b 0000 0000 ==> 2个1

```

- 逻辑运算符

- 0 0x0 '\0' 都能表示假
- && (与) : A && B。两真全真 编译器再运行时, 如果A为真, 执行B; 如果A为假, 不执行B, 直接结束。A && B 不等价 B && A。有先后关系的执行
- || (或) : A || B。一真全真 编译器再运行时, 如果A为假, 执行B; 如果A为真, 不执行B, 直接结束。预防性编程 (没有发生, 应该怎么处理)

```

1 #include <stdio.h>
2
3 int main()
4 {
5 int a = 1, b = 2, c = 3, d = 4;
6
7 printf("c=%d,d=%d\n", c, d); // c = 3, d = 4
8 // &&
9 // 如果A为假, 不执行B
10 (c = a > b) && (d += 1);
11 printf("c=%d,d=%d\n", c, d); // c = 0, d = 4 只执行了(c = a
> b)
12 // 如果A为真, 执行B;
13 (c = a < b) && (d -= 1);
14 printf("c=%d,d=%d\n", c, d); // c = 1, d = 3 (c = a < b) 与
(d -= 1)都执行了
15
16 // ||
17 // 如果A为假, 执行B
18 (c = a > b) || (d += 1);
19 printf("c=%d,d=%d\n", c, d); // c = 0, d = 4 执行了(c = a >
b)、(d += 1)
20 // 如果A为真, 不执行B
21 (c = a < b) || (d -= 1);
22 printf("c=%d,d=%d\n", c, d); // c = 1, d = 4 只执行了(c = a
< b)
23
24 return 0;
25 }
26
27 // 结果
28 /*
29 c=3,d=4
30 c=0,d=4
31 c=1,d=3
32 c=0,d=4
33 c=1,d=4
34 */

```

•

| 运算符 | 作用                           | 说明                                                                                                |
|-----|------------------------------|---------------------------------------------------------------------------------------------------|
| =   | 赋值                           | 等号右边空间的内容 按一定约束( <a href="#">大小容量转换</a> ) 逐位 拷贝到左边的空间中 初始化, 在生成空间时, 放入空间数据 (只执行一次 <sup>6</sup> )  |
| ==  | 逻辑相等 (表达式)                   |                                                                                                   |
| ~   | 按位取反                         | 当成纯数据进行取反 (即全部位 一起取反)                                                                             |
| `   |                              | C语言中没有定义该符号                                                                                       |
| !   | 逻辑取反 (表达式)                   | 只有真 或者 假                                                                                          |
| @   |                              | C语言中没有定义该符号                                                                                       |
| #   | 预处理前缀符号                      |                                                                                                   |
| \$  |                              | C语言中没有定义该符号                                                                                       |
| %   | 取模 (取余)                      | 常用于 得到循环数;<br>限制范围 $\%n \Rightarrow (0 \sim n - 1)$                                               |
| ^   | 异或                           | 常用于 加密解密;<br>2个数交换, 不使用第三个数 $a = a \wedge b; b = a \wedge b; a = a \wedge b;$<br>异或链表, 省内存 (但极少用) |
| &   | 按位与 (2个操作数)、<br>取地址符 (1个操作数) | 取地址符: 不能大小端 (只会影响存数据的方式), 取到的地址都是低位地址 (首地址)                                                       |
| &&  | 逻辑与 (表达式)                    | 当第一个表达式为假时, 不会对第二个表达式进行判断                                                                         |
| *   | 乘法、指针定义符 (类型提升符)、指针取用符       | 类型提升符: 完成通过指针里保存的地址, 间接访问其他空间的内容<br>区别类型提升符、指针取用符, 声明 (类型提升符) —— 有数据类型; 使用 (指针取用符) —— 没有数据类型      |
| ()  | 优先级符号、函数输入参数的定义、函数调用         |                                                                                                   |
| -   |                              | C语言中没有定义该符号                                                                                       |
| -   | 减法                           | 有时需要取相反数                                                                                          |
| +   | 加法                           |                                                                                                   |

| 运算符 | 作用                    | 说明                                                                                                                                                        |
|-----|-----------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------|
| ++  | 自增                    | 行为上的区别：<br>a++（把a当成表达式的结果，先把a返回，再改变a）<br>++a（先改变a，在把a当成表达式结果，返回其值）<br>性能区别：<br>a++ 会先在内存中备份一次就数据，再在a上自增，最后返回备份的旧数据<br>++a 直接自增，返回a的值。<br>所以a++开销和内存比 ++a大 |
| --  | 自减                    | 同上                                                                                                                                                        |
| { } | 函数体、结构体的成员列表、数组空间的初始化 | 在内存上，这是一块空间                                                                                                                                               |
| [ ] | 数组下标、空间范围符（对于指针）      |                                                                                                                                                           |
|     | 按位或                   |                                                                                                                                                           |
|     | 逻辑或（表达式）              | 当第一个表达式为真时，就不会执行第二个表达式                                                                                                                                    |
| \   | 转义字符、续航符              | 一般\后面直接跟数字都是八进制表示；\x后面跟着数字表示十六进制                                                                                                                          |
| :   | 标签标识、位域（结构体中语法）、三目运算符 | 当成标签时在内存中不会占空间，编译时也不会变成指令                                                                                                                                 |
| ;   | 语义分隔符                 |                                                                                                                                                           |
| ""  | 字符串常量                 | 在内存中是一块连续空间，且末尾编译器会自动生成一个\0的结束标志<br>本质就是一个地址（首地址）标签                                                                                                       |
| ''  | 一个字符（占1B）             |                                                                                                                                                           |
| ,   | 逗号表达式、配合{}进行初始化       | int a = 10, 20; 最后a = 20; (逗号表达式)<br>{1,2,3}                                                                                                              |
| <   | 逻辑小于                  |                                                                                                                                                           |
| >   | 逻辑大于                  |                                                                                                                                                           |
| ?   | 三目运算符                 | 简化的if -else                                                                                                                                               |
| /   | 除法                    |                                                                                                                                                           |
| <<  | 位运算 左移                |                                                                                                                                                           |
| >>  | 位运算 右移                |                                                                                                                                                           |
| .   | 结构体变量访问成员             |                                                                                                                                                           |

| 运算符 | 作用           | 说明                                   |
|-----|--------------|--------------------------------------|
| ->  | 结构体指针访问成员    |                                      |
| ... | 不定参数（可变参数列表） | 只能在函数的形参列表中使用，且在形参列表的位置是最后，后面不能跟其他形参 |

- `+ - * /`
  - `+` `-`:cpu通过硬件电路实现，（减法会变成加法，将负数转换成补码）
  - `*` `/`:便宜的cpu没有乘法器和除法器的硬件电路。
    - 如果想要实现乘法和除法，软件模拟，软件乘法和软件除法，通过 `+` `-` 的循环、移位 来模拟

## 7. 语句

- C的分支循环语句（if-else、while、do-while、for、switch...case...default、goto）
- 分支语句
  - if-else

- ```

1  if (a > b)
2  // 执行一条语句
3
4
5  if (a > b) {
6
7  }
8
9
10
11 if (a > b) {
12
13 } else {
14
15 }
16
17
18
19 if () {
20
21 } else if () {
22
23 } else {
24
25 }
```

- switch-case-default

- ```
1 switch(变量){ // 变量类型，只能是整数类型（char int
longlong long short）
2 case 常量表达式1:
3 代码块；
4 break; // 如果没有这个，程序会继续执行下面代码；添加之
后才会跳出循环
5 case 常量表达式2:
6 代码块；
7 break; // 如果没有这个，程序会继续执行下面代码；添加之
后才会跳出循环
8 ...
9 default:
10 break;
11 }
12
13 // 变量 == 常量表达式n 从case 常量表达式n:开始执行代码。 变量 != 常量表达
式n 从default开始执行
```

- 循环语句

- while

- ```
1  // 先判断语句，为真执行代码块，执行完之后再次对语句进行判断；为假不进入循环
2  while(语句){
3      代码块；
4  }
5
6  // 先执行代码块，再判断语句，为真继续执行代码块，然后继续判断；为假退出循环
7  do{
8      代码块；
9  }while(语句);
```

- for

- ```
1 // 先执行表达式1
2 // 判断表达式2，为真执行代码块，执行表达式3，再次判断表达式2；为假退出循环
3 // 表达式3 一般推荐使用++i（会少一点性能消耗）
4 for(表达式1;表达式2;表达式3){
5 代码块；
6 }
```

- 跳转语句

- goto

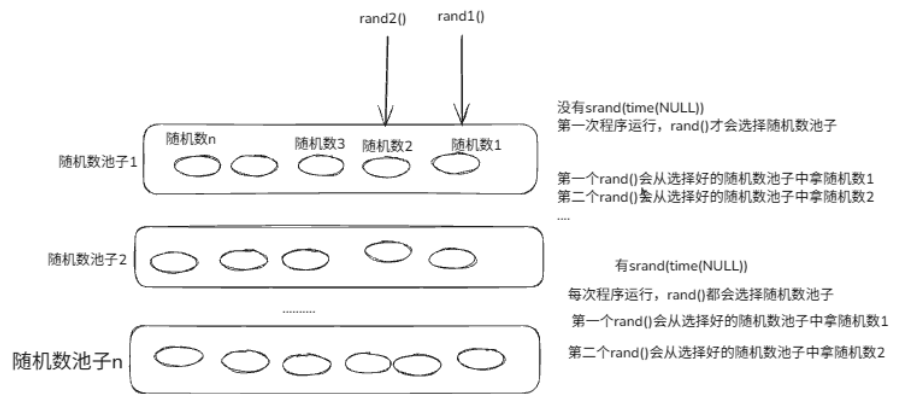
- ```
1  // 跳转语句 跳过中间代码，执行标记名下方的代码
2  goto 标记名；
3      ...
4  标记名：
5      ...
6
```

```

7 // goto 作用范围为 当前函数中
8 // 经常用于重复执行代码
9 // eg. 资源申请
10 int state = 0; // 结果状态
11 申请资源
12 int ref = 申请资源函数();
13 // 申请资源失败
14 if(ref < 0){
15     state = 1; // 申请资源失败状态
16     goto out1;
17 }
18 打开设备
19 int ref = 打开设备函数();
20 // 打开设备失败
21 if(ref < 0){
22     state = 2; // 打开设备失败状态
23     goto out2;
24 }
25 关闭设备
26 out2:
27     关闭设备函数()
28 释放资源
29 out1:
30     释放资源函数()
31     return state;

```

- break: 退出当前这层循环
- continue : 跳过当前循环的剩余代码, 进入下一次循环
- C语言必须使用整数类型 (char、int、short、long、long long) 的地方: % 和 switch()
- 小技巧
 - 使用while语句的情况: 不知道具体循环次数时, 利用其条件进行循环。
 - 使用for语句的情况: 明确知道循环次数时, 利用其循环次数进行循环。
- 练习代码 —— 猜数字程序
 - 随机数函数
 - rand() ——头文件#include <stdlib.h>
 - 生成的随机数很大, 需要取模获取相应的随机数范围。
 1. 目标范围[left,right]
 2. rand() % (right - left + 1) 获取[0,right-left], right - left + 1为总的大小
 3. (rand() % (right - left + 1)) + left 获得目标范围[left,right]
 - 会从多个随机数池子中的一个池子 获取随机数, 默认运行第一次就固定了随机数池子 (后续再运行也是取第一次运行的随机数)
 - srand(time(NULL)); 通过时间的秒数改变随机数池子 time()头文件#include <time.h>



■ 使用方法

```

1  #include <stdlib.h>
2  #include <time.h>
3
4  int left = 2, right = 100;
5  srand(time(NULL));
6  int num = rand() % (right - left + 1) + left; // 获取[2,100]随机数

```

○ scanf():

■ 获取键盘的输入

```
scanf("占位符",&变量名); // scanf("%d",&num);
```

```

1  #include <stdio.h>
2  #include <stdlib.h>
3  #include <time.h>
4  #include <windows.h>
5
6  /*
7  猜数字
8  1. 随机一个数字[100,200]
9  2. 用户输入数字猜测数字
10  3. 用户只有5次猜测机会
11  4. 提示用户每次猜测结果
12  */
13
14  int main(){
15      SetConsoleOutputCP(65001);
16
17      // 数字范围[100,200]
18      int left = 100, right = 200;
19
20      int num = 0;
21      srand(time(NULL)); // 使用时间来获取随机数池子
22
23      num = rand() % (right - left + 1) + left;

```

```

24  #ifdef MY_DEBUG
25      printf("num=%d\n",num);
26  #endif
27
28      int user_num = 0;
29
30      // 循环获取5次
31      for(int i = 0; i < 5; i++){
32          input1:
33              // 获取数字
34              printf("可以猜测次数:%d/5,请输入猜测数字 (100~200): \n", i + 1);
35              int ref = scanf("%d",&user_num);
36              // 处理非法数字和字符
37              if(ref != 1 || user_num < 100 || user_num > 200){
38                  while(getchar() != '\n');
39                  goto input1;
40              }
41
42              // 比较数字
43              if(user_num < num){
44                  printf("猜小了!\n");
45              }else if(user_num > num){
46                  printf("猜大了!\n");
47              }else{
48                  printf("猜对了!\n");
49                  break;                // 退出循环
50              }
51      }
52
53      printf("结束游戏! ");
54
55      return 0;
56  }

```

o

8. 关键字

- 编译器解析行为：
 - 对 源码文件⁷ 按行 (c/c++以分号为一行) 解析、识别 字符串
 - 字符串 (符号)：每个标点符号 都有默认的运算规则 c++有运算符的重载 来扩展规则
 - 字符串 (单词)：
 - 编译器已知单词 => 关键字
 - 编译器扩展单词(未知) => 变量
 - 文本文件：按照一个特定符号编码 显示出符号
 - 二进制文件：特定的解码规则

C 语言中的关键字

auto	break	case	char
const	continue	default	do
double	else	enum	extern
float	for	goto	if
int	long	register	return
short	signed	sizeof	static
struct	switch	typedef	union
unsigned	void	volatile	while

- float与double：用于存储小数（内存中小数分成符号位+指数位+有效数字位）。**float的大小：4B。double的大小：8B。编译器默认小数使用double进行嵌入**，例如：float num = 3.14f; 让3.14在运行之前以float形式嵌入指令。
- void ——> void * 【万能指针，能接收任何指针，但是不能直接解引用（* p），相应解引用需要强制类型转换（*（int）p）】
- auto：在C语言里 auto **没有实际意义**，局部变量默认就是auto关键字。
- enum：枚举类型。用法同结构体。枚举中定义的成员的默认初始值，在前一个的初始值基础上+1

```
1  #include <stdio.h>
2
3  typedef enum aa{
4      A,                                // 没有初始值，默认第一个成员初始值为0，后续成员
      的初始值逐个+1
5      B,
6      C = 5,                            // 有初始值，后续成员的初始值在次基础上逐个+1
7      D,
8      E
9  }AAA;                                // 0, 1, 5, 6, 7
10
11 int main(){
12
13     AAA a = A;
14     AAA b = B;
15     AAA c = C;
16     AAA d = D;
17     AAA e = E;
18
19     printf("%d %d %d %d %d",a,b,c,d,e);
20
21     return 0;
22 }
23
24 // 结果
25 // 0 1 5 6 7
```

- register：register 数据类型 变量名 会将标记的变量放入寄存器中 常用于 经常使用的数据 **该关键**

字标记的变量没有地址，不能取地址

o typedef

- 作用：起别名
- typedef 普通用法：typedef 原型 别名；

```
1 // 1.
2 typedef struct people{
3     int m_age;
4     char *m_name;
5 }my_people;           // 现在my_people等价于struct people,
                        // 这个可以用 my_people或者struct people声明变量
6
7 // 2.
8 typedef long byte8_t; // 现在byte8_t等价于long
9
10 // 3.
11 typedef struct {
12     int m_age;
13     char *m_name;
14 }my_people;           // 这个只能用 my_people声明变量
15
16
17 // 注意：
18 /*
19 这是一个匿名结构体
20 struct {
21     数据类型1  成员名1;
22     数据类型2  成员名2;
23 };
24 */
```

- 对函数指针起别名：typedef 返回值类型 (*函数别名)(形参列表);

```
1 // 4.
2 typedef int (*fun)(int a,int b);
3 // fun为函数指针别名
4
5 int add(int a,int b)
6 {
7     return a + b;
8 }
9
10 int main()
11 {
12     // 使用时将指向需要的函数的地址
13     fun fp = add;
14     int a = fp(1,2);
15
16     return 0;
17 }
```

- 与#define 的区别：
 - #define 是单纯的文本替换；
 - typedef 是起别名（它有作用范围）

```

1  /* 区别1:对普通情况 */
2  // typedef 是类型别名，编译器会进行类型检查
3  // #define 文本替换，编译器不会检查
4
5  /* 区别2:对指针 */
6  typedef char* char_ptr;
7  #define CHAR_PTR char *
8
9  char_ptr a,b; /* a和b都是char指针 */
10 CHAR_PTR c,d; /* c是char指针，d是char类型 */
11
12 /* 区别3:对复杂的类型 */
13 typedef char char_a[10];
14 char_a a; /* a是char[10]数组 */
15
16 #define CHAR_A char[10];
17 CHAR_A b; /* 错误，文本替换后 不符合C语法 char[10]
18 b; 编译阶段会报错 */
19
20 /* 区别4:作用域 */
21 void func() {
22     typedef int MY_INT; // 只在 func 内有效
23     MY_INT a = 10; // 正确
24 }
25 MY_INT b = 20; // 报错：MY_INT 未定义
26
27 void func2() {
28     #define MY_INT int // 从这里开始，后面所有代码都替换
29     MY_INT a = 10; // 正确
30 }
31 MY_INT b = 20; // 正确！#define 穿透了函数作用域
32
33 /* 区别5:调式 */
34 // typedef 能被调式器识别
35 // #define 调式器无法看见它（预处理就消失了）

```

- **sizeof**: 计算大小，**编译阶段完成计算**。sizeof不是函数。
 - **volatile**: 防止编译器优化。（编译器为了方便会将常用且当前范围没有修改的变量放入寄存器中，这样会导致其他范围修改了该值以后（内存中值变了），编译器依然从寄存器中获取该变量。使用了volatile后，会强制编译器从内存中读取该变量）

- 使用场景

1. 并行设备的硬件寄存器（如：状态寄存器）
2. 一个中断服务字程序中会访问到的非自动变量（Non - automatic variables）
3. 多线程应用中被几个任务共享的变量

- **extern**: 声明, 全局可用 (任何位置的文件可用)
- **const**: 用const修饰的变量, 不能通过当前变量修改里面的值。可以通过指针获取该变量地址后, 使用指针修改其值。 (**直接不能改, 指针间接改**)
- **static**:
 - **修饰函数、全局变量, 让其只能在本文中使用的**
 - **修饰局部变量** (不会改变作用范围)
 1. 将修饰的局部变量放到静态数据区 (没有修饰的局部变量一般放到栈区)。
 2. 该局部变量只会初始化一次。
 3. 该局部变量的内存释放是在程序结束时 (生命周期从当前函数变到整个程序)。

```
1 // test05_main
2 #include <stdio.h>
3 #include <stdlib.h>
4 #include <time.h>
5 #include <windows.h>
6
7 #include "test05_fun.h"
8
9 /*
10 1. 编写一个函数, 每调用一次, 就输出当前是第几次调用。
11 2. 无论调用多少次, 计数必须连续累加, 不能重置为 0。
12 3. 计数变量不能是全局变量, 只能定义在函数内部。
13 4. 程序结束后, 计数变量才释放内存。
14 */
15
16 int main(){
17     SetConsoleOutputCP(65001);
18
19     srand(time(NULL));
20     int num = (rand() % 10) + 1;
21
22     while(num){
23         num--;
24         fun();
25     }
26
27
28
29     return 0;
30 }
31
32 // 结果(结果不一定相同)
33 /*
34 调用次数1
35 调用次数2
36 调用次数3
37 调用次数4
38 */
```

```
1 // test05_fun.h
2 #ifndef TEST05_FUN_H
3 #define TEST05_FUN_H
4
5 extern void fun();
6
7 #endif
8
9 // test05_fun.c
10 #include <stdio.h>
11
12 void fun(){
13     static int cut = 0;           // 调用计次 static修饰，一次初始化，当前文件能用
14
15     cut++;
16
17     printf("调用次数%d\n",cut);
18 }
```

关键字	作用	使用	补充
sizeof	返回 数据类型 或者 变量 所占的字节 大小	sizeof(类型)	sizeof 变量 可以这么写，但是无法区分(关键字与关键字 即无法识别 sizeof int),所有使用了()。sizeof的返回值大小并不是确定的（每个平台的数据类型大小不一样）使用 %zd 可变大小 （printf("%zd", sizeof(long));) sizeof一般都是在编译阶段就计算出结果（将创建空间时的大小，复制过来）
typedef	起别名	typedef 原名 别名;	后期常用于对指针、结构体起外号
struct	结构体声明 (自定义类型 声明)	用法	
union	共用体声明 (自定义类型 声明)	用法	
enum	枚举	同结构体	比较鸡肋，赋值时没有范围要求（可以写不是枚举类型的值）
const	建议修饰的 变量不能修改	const 变量;	C语言const修饰并非强制性的；只是不同直接修改，可以通过指针间接修改该空间的内容
extern	全局声明	extern 函数;	可以不写，默认就是extern

关键字	作用	使用	补充
static	静态变量; 本文件有效	static 变量名; static 函数;	提升变量生命周期; 降低函数、全局变量的作用范围

9. 数组

- 数组本身是指针的语法糖，是一种数据结构（有限的线性表），不是数据类型，所以数据不会检查越界
 - 在编译时，编译器认可数组，编译时将所有非运行时的结果（包括数组分配的空间），在编译时期计算完成
 - 一旦开始运行，就没有数组概念，所有的数组名，在编译时，数组名 = 空间首地址
- 数组定义：数据类型 变量名[个数]。
 - 本质上：开辟了一个连续的空间，在这个空间上均分出相同的对象（盒子）
- 数组名 ==> 数组的首地址 ==> 等价于常量，不能被赋值

```

1  #include <stdio.h>
2
3  int main(){
4      int arr[5] = {1,2,3,4,5};
5      int (*p)[5] = &arr;
6      // &arr 存储的是整个数组的首地址
7      // arr,&arr[0]存储的是数组第一个成员的首地址
8      // 二者数值相等，意义不同
9      printf("%p %p %p\n",arr,&arr[0],p);
10
11     // 前两个都是偏移 一个int大小 后面一个偏移了一个数组大小（5个int）
12     printf("%p %p %p\n",arr+1,&arr[0] + 1,p + 1);
13
14     return 0;
15 }
16
17
18
19 // 现象
20 /*
21 000000cb633ffc00 000000cb633ffc00 000000cb633ffc00
22 000000cb633ffc04 000000cb633ffc04 000000cb633ffc14
23 */

```

- 数组的使用：
 - 数组名[偏移单位];
 - 数组名本质是一段连续空间的标签，所以数组名必须在编译阶段编译器就需要知道大小，因此数组名不能被改变（常量）——数组名不能被赋值（出现在等号的左边）。

- 数组名遇到sizeof关键字时，才会被当做这个连续空间；其他时候只是做为整个连续空间的首地址
- 数组的处理
 1. 只要声明时初始化，可以整体赋值
 2. 一旦定义了，再使用时，处理思路是**逐个处理**。（未赋值的部分值为随机值，所以尽量初始化，初始化会将未赋值的部分用0填充）
- 数组的注意事项（裸机开发可能会遇到）

```

1  char data[] = "hello";
2
3  // 等号左边是一个连续的存放char的空间
4  // 等号右边是一个字符串的首地址
5  // 数组 != 地址
6  // 但是在C语言内部，会自动调用memcpy函数或者strcpy函数（逐个拷贝）实现data空间的初始化
7  // 在裸机开发时，会出现开发编译不通过，原因是，不同体系结构的裸机缺少对应的调用函数，所以需要自己手动实现函数（如：memcpy()、strcpy()...）

```

- 编译器会在首地址上 + 偏移单位（偏移单位 ==> 偏移 (+/-) 数据类型的大小）

```

1  #include <stdio.h>
2
3  #define MAX 5
4
5  int main(){
6
7      int arr[MAX] = {1,2,3,4,5};
8      char arr1[MAX] = {'a','b','c','d','e'};
9
10     // 读取数据
11     for(int i = 0; i < MAX; i++){
12         // arr[1] 取出arr空间上的第二个盒子的内容
13         printf("%d %c\n",arr[i],arr1[i]);
14     }
15     printf("\n");
16     // 查看相应地址
17     for(int i = 0; i < MAX; i++){
18         // &arr[1] 取出arr空间上的第二个盒子的首地址，两个相邻首地址相差的大小为数据类型大小
19         // &arr[0] 与&arr[1]相差4B   &arr1[0]与&arr1[1]相差1B
20         printf("%p %p\n",&arr[i],&arr1[i]);
21     }
22
23     return 0;
24 }
25
26 // 现象
27 /*
28 1 a
29 2 b
30 3 c

```

```

31     4 d
32     5 e
33
34     477b1ffba0 477b1ffb9b
35     477b1ffba4 477b1ffb9c
36     477b1ffba8 477b1ffb9d
37     477b1ffbac 477b1ffb9e
38     477b1ffbb0 477b1ffb9f
39     */
40

```

■ 数组未初始化的成员，用0填充

```

1  #include <stdio.h>
2
3  int main(){
4
5      int a[5] = {1,2,3};
6      char b[5] = {'A','B','C'};
7      float c[5] = {1.0,2.1,3.2};
8
9      // 没有初始化的数组成员，全部默认使用0填充
10     for(int i = 0; i < 5; i++){
11         printf("%d,%c,%f\n",a[i],b[i],c[i]);
12     }
13
14
15     char b1 = '\0';
16     char b2 = '0';
17
18     printf("%c,%c\n",b1,b2);           // 字符0 ASIIC码为
0x30; ASIIC码为0对应'\0'（字符串结束标志符）
19     printf("%llx,%llx\n",b1,b2);
20
21     return 0;
22 }
23
24 // 现象
25 /*
26 1,A,1.000000
27 2,B,2.100000
28 3,C,3.200000
29 0,,0.000000
30 0,,0.000000
31 ,0
32 0,30
33 */

```

• 数组越界

- 使用超出盒子规定个数的大小，进行读取数据（下标 >= 定义的大小）
- c语言编译器能编译通过，但是不合法（即会警告，但不报错）

```

1  #include <stdio.h>
2

```

```

3   #define MAX 5
4
5   int main(){
6       int arr[MAX] = {1,2,3,4,5};
7
8       // 合法访问
9       printf("%d %d\n",arr[0],arr[4]);
10
11      // 不合法访问
12      // -1和5都已经超过了这个数组的规定访问（0~4）
13      // 会出现的不确定的结果（那个内存是否能读写）
14      printf("%d %d\n",arr[-1],arr[5]);
15
16      return 0;
17  }
18  // 现象
19  /*
20  1 5
21  0 516
22  */

```

- sizeof与数组

- sizeof(数组名) == 整个连续空间（数组）的大小
- sizeof(数组名[1]) == 第二个数组上盒子的大小

- ```

1 #include <stdio.h>
2
3 int main(){
4
5 int arr[5] = {1,2,3,4,5};
6
7 printf("%d\n",sizeof(arr)); // 20B 整个数组的大小
8 printf("%d\n",sizeof(arr[0])); // 4B 数组中一个成员的
大小
9 printf("%d\n",sizeof(arr) / sizeof(arr[0])); // 5个 数组中的成员个数
10
11 return 0;
12 }
13 //现象
14 /*
15 20
16 4
17 5
18 */

```

- 二维数组（空间连续）

- 二维数组的定义，是告诉编译器如何访问内存索引的计算方式
- 一维的内存映射成二维的行列
  - 目的：为了方便程序员阅读代码，计算坐标系统集合、线性代数（矩阵运算）
  - 在实际的开发过程中，还是关注逻辑坐标，进行编写代码。逻辑转换成内存空间，由编译器完成

- 数据类型 变量名[行数][列数] (列数不能省略)
- 二维数组遍历：
  - 第一层循环一个坐标系
  - 第二次循环一个坐标系
- 二维数组访问不同方式的时间
  - 不同方式的时间差异与cpu体系架构有关：缓存（Cache）访问主存是，会将一片连续空间拷贝到Cache中，但是Cache空间有限，没有命中（没有找到）就拷贝新的一段空间覆盖原本的
  - 内存中按照一行一行读取的情况
    - 先按行遍历，在遍历列，内存会连续访问，Cache一次访问内存，能读出更多有效数据
    - 先按列遍历，在遍历行，内存访问不连续，访问时间更久

```

1 /* 测试2：二维数组不同访问方式的时间 */
2 void test02(){
3
4
5 /* 初始化 */
6 for(int row = 0; row < ROW; ++row){
7 for(int col = 0; col < COLUMN; ++col){
8 image[row][col] = COLUMN * row + col + 1;
9 }
10 }
11
12 /* 按行读取 */
13 unsigned long long sum = 0;
14 clock_t strat = clock();
15 /* 记录起始cpu已经消耗的计时单元数 */
16 for(int row = 0; row < ROW; ++row){
17 for(int col = 0; col < COLUMN; ++col){
18 sum += image[row][col];
19 }
20 }
21 clock_t end = clock();
22 /* 记录结束cpu消耗的计时单元数 */
23 printf("%.12f \n", ((double)end - (double)strat) /
24 CLOCKS_PER_SEC); /* 换算成秒 */
25
26 sum = 0;
27 strat = clock();
28 for(int col = 0; col < COLUMN; ++col){
29 for(int row = 0; row < ROW; ++row){
30 sum += image[row][col];
31 }
32 }
33 end = clock();
34
35 printf("%.12f \n", ((double)end - (double)strat) /
36 CLOCKS_PER_SEC);
37 }

```

```

34
35 // clock(); // 记录当前的计算机的节拍数 单位次数
36
37 /*
38 结果:
39 0.148000000000
40 0.506000000000
41 */

```

- o `*(num + 1) <==> num[1]    *(* (num + 1) + 1) <==> num[1][1] <==> *(num[1] + 1)` 加  
\* 看出进盒子内部; 加&看出退出盒子

- 定义一个指针变量, 来保存二维数组、三维数组名

- `int arr1[m][n]; int (*p)[n] = arr1;`
- `int arr2[x][m][n]; int (*p)[m][n] = arr2;`

```

o 1 #include <stdio.h>
2
3 int main(){
4 // 多维数组 : int arr[][] = {{1,2,3},{4,5,6}}; 这样写会报错
5 int arr1[2][3] = {{1,2,3},{4,5,6}};
6
7 // 正常取数据
8 for(int i = 0; i < sizeof(arr1) / sizeof(arr1[0]); i++){
9 for(int j = 0; j < sizeof(arr1[0]) / sizeof(arr1[0][0]); j++){
10 printf("%d ", arr1[i][j]);
11 }
12 }
13
14 printf("\n");
15
16 // 数组各个层大小
17 printf("sizeof(arr1):%d\n", sizeof(arr1)); // 数组总体大小
18 24B
19 printf("sizeof(arr1[0]):%d\n", sizeof(arr1[0])); // 数组内层大小
20 12B
21 printf("sizeof(arr1[0][0]):%d\n", sizeof(arr1[0][0])); // 单个成员大小
22 4B
23
24 printf("\n ----- \n");
25
26 // 地址
27 printf("%p\n", &arr1); // 数组首地
28 址
29 printf("&arr1[0]:%p %p\n", arr1, &arr1[0]); // 数组行首
30 地址
31 printf("&arr1[0][0]:%p %p\n", &arr1[0][0], arr1[0]); // 数组列首
32 地址(单个成员)
33
34 printf("\n --- \n");
35
36 // 地址偏移

```

```

31 printf("%p\n",&arr1 + 1); //
 数组地址偏移 + 24B
32 printf("&arr1[0]:%p %p\n",&arr1[0] + 1,arr1 + 1); //
 数组行首地址 + 12B
33 printf("&arr1[0][0]:%p %p\n",&arr1[0][0] + 1,arr1[0] + 1); //
 数组列首地址(单个成员) + 4B
34
35 return 0;
36 }
37
38 // 现象
39 /*
40 1 2 3 4 5 6
41 sizeof(arr1):24
42 sizeof(arr1[0]):12
43 sizeof(arr1[0][0]):4
44 -----
45 000000bcb65ffa70
46 &arr1[0]:000000bcb65ffa70 000000bcb65ffa70
47 &arr1[0][0]:000000bcb65ffa70 000000bcb65ffa70
48 ---
49 000000bcb65ffa88
50 &arr1[0]:000000bcb65ffa7c 000000bcb65ffa7c
51 &arr1[0][0]:000000bcb65ffa74 000000bcb65ffa74
52 */

```

## • 数组指针 和 指针数组

- 数组指针：重点是指针 `数据类型 (*变量名)[];` 指向数组的指针
- 指针数组：重点是数组 `数据类型 *变量名[];` 存储指针（地址）的数组

```

◦ 1 #include <stdio.h>
2
3 int main(){
4 int arr1[2][3] = {{1,2,3},{4,5,6}};
5 int arr2[3][3] = {{1,2,3},{4,5,6},{7,8,9}};
6
7 // 数组指针
8 int (*p1)[3] = arr1; // 传行首地址 p1走一次需要跨越3
 个int的大小
9 // int (*p2)[3] = &arr1; // 传数组首地址 有警告 会变成逻辑
 上的 三维数组
10 int (*p2)[3] = arr2;
11 // 一个 int (*p1)[3] 可以指向 行数任意列数为3的二维数组
12
13 // 指针数组
14 int *p3 = NULL,*p4 = NULL;
15 int * arrp[] = {p3,p4};
16 int (*arrp1[][3]) = {p1,p2}; // 存放二维数组的行指针
17
18
19 // 大小
20 printf("%d %d %d\n",sizeof(p1),sizeof(p2),sizeof(arrp)); // 8 8 16

```

```

21 printf("%d %d\n", sizeof(*p1), sizeof(**p1)); // *p1是行
 元素 **p1是列元素
22
23 // 取值 同二维数组
24 // 地址
25 printf("%p,%p\n", p1, *p1);
26 // 地址偏移
27 printf("%p,%p,%p\n", p1 + 1, *p1 + 1, *(p1 + 1)); // 行偏移 列
 偏移*(p1 + 0) + 1 列偏移
28
29 return 0;
30 }
31 // 现象
32 /*
33 8 8 16
34 12 4
35 000000c6839ffad0, 000000c6839ffad0
36 000000c6839ffadc, 000000c6839ffad4, 000000c6839ffadc
37 */
38
39 /*
40 *p + 1 <==> &a[0][1]
41 *(p + 1) <==> &a[1][0]
42 */

```

- n维数组的数组指针：数据类型 (\*p)[][][]..[] = 数组名; 就是用(\*变量名) 替换 数组名[]

```

o 1 #include <stdio.h>
2
3 int main(){
4
5 // n维数组的数组指针： 数据类型 (*p)[][][]..[] = 数组名; 就是用(*变量名) 替
 换 数组名[]
6 // a[1][2]; => (*p)[2] 用*p替换外层的a[1]
7
8 int a1[3] = {0};
9 int *p1 = a1;
10 int (*p2)[3] = &a1; // 这个就是二维数组; 需要一维数组的整个数组首地
 址
11
12 int a2[3][3] = {0};
13 int (*p3)[3] = a2;
14 int (*p4)[3][3] = &a2; // 这个就是三维数组; 需要二维数组的整个数组首地
 址
15
16 int a3[3][3][3] = {0};
17 int (*p5)[3][3] = a3;
18 int (*p6)[3][3][3] = &a3; // 这个就是四维数组; 需要三维数组的整个数组首地
 址
19
20 return 0;
21 }

```

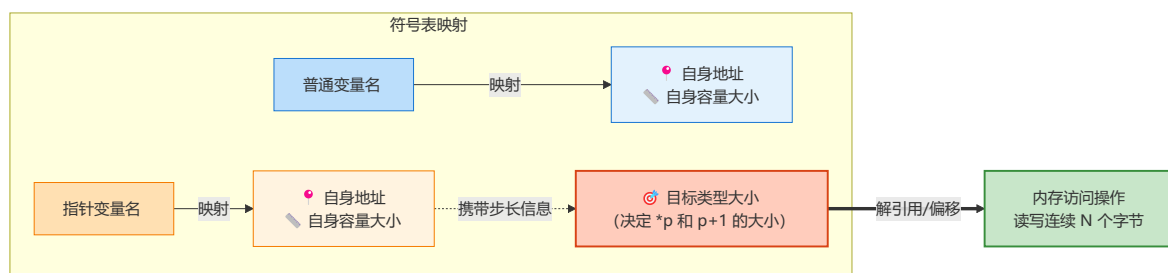
- 指针数组 和 数组指针判断

- 指针数组：（看到指针数组需要思考：数组的大小；存什么数据）
  1. 找变量名
  2. 向变量名的右边看是 [
- 数组指针：（看到数组指针需要思考：如何修饰指针（存的什么地址（函数；数组；其他指针；普通变量））
  1. 找变量名
  2. 向变量名右边看是 ) , 在向左边看是 \*
- 常量不能取地址（执行时没有意义）。如果对其取地址，他的含义会被编译器重新定义
  - 数组名：数组空间首地址
  - &数组名：编译器自行的行为，地址表示一个数组一个数组大小的访问，也就是当成了二维数组

```
1 | int a[10];
2 | &a;
```

## 10. 指针

- C语言分配与访问内存的两种方式：
  1. 普通变量：分配空间（容量大小）+ 定义空间的行为（有符号、无符号）
  2. 数组、指针：分配空间（容量大小）+ 定义空间的行为（解码形式）+ 相关的对象的大小限制（指向的空间或者内部划分的空间）
- C语言作为高级语言，更关注内存对象（空间）



- 存储**二进制信息** —— 基本数据类型
- 存储**地址信息**（需要关注下面两个信息） —— 指针类型
  - **自身的容量**：编译器来决定（gcc -v中的 target 能查看到x86\_64，也就是64bit 对应 8B，32bit 对应4B）
  - **存的地址能否访问**，如何访问改空间（修饰该空间的类型，读多大，以有无符号读）

|      | 什么样的容器           | 运算                   |
|------|------------------|----------------------|
| 普通变量 | 一个 保存基本数据类型大小的容器 | 实现数学规则， 支持位、逻辑、算术、运算 |

|      | 什么样的容器        | 运算                                                                   |
|------|---------------|----------------------------------------------------------------------|
| 指针变量 | 一个存储地址信息大小的容器 | 如何访问下一个对象 $\Rightarrow \pm 1$ : 地址向上/向下一个对象(空间)大小的偏移 而对于其他运算(乘除)没有意义 |

- 指针的本质是“存储内存地址的变量”(如何处理内存编号上盒子的数据), 其大小等于CPU的地址总线宽度
  - 32位CPU: 地址总线宽度32位  $\rightarrow$  指针占4字节(能表示  $0 \sim 2^{32}-1$  的内存地址);
  - 64位CPU: 地址总线宽度64位  $\rightarrow$  指针占8字节(能表示  $0 \sim 2^{64}-1$  的内存地址);
  - 关键: 指针大小与指向的类型无关 (`char*`/`int*`/`double*` 大小完全一致), 只和CPU架构有关。
  - 对于指针, 编译器必须在编译阶段等到2个问题的答案, 才能工作

1. 指针容器大小

2. 通过这个容器间接访问的内存的行为和大小

- 地址总线 = 有多少个对象(空间); 数据总线 = 每个对象多大; 控制总线  $\Rightarrow$  控制读/写、片选
  - 芯片容量 = 地址线(有多少个) \* 数据线(每个多大) 如:  $4G \times 16$  地址线有32根 ( $2^{32}GB$ ), 数据线有16根

- “某CPU有16根地址线(A0-A15), 8根数据线(D0-D7), 读写控制线为  $WE \sim \overline{WE}$ 。现有容量为  $4K \times 4$  位的RAM芯片。

1. 需要多少片这样的芯片?

2. 画出CPU与这些芯片的连线图(或问: CPU总共需要引出多少根线连接到存储器?)”

**总连线数 = 地址线根数 + 数据线根数 + 控制线根数**

- 离CPU越远, 内存容量越大, 速度越慢, 需要的地址线多(为了解决过多的线, 导致芯片空间过大, 通常使用多个时间周期来获取大容量内存中的数据)
- 32位的系统, 一个字(word) = 4个字节; 64位系统, 1word = 8byte; 16位的系统, 一个字 = 2个字节 —— 一般字都是形容cpu一次能高效处理的数据宽度, 一周期按字处理。区分内存的编址能力, 按字节编址

- 数据类型 `*(&变量名);`

- &: 取出变量的地址, 拿该盒子中最低的内存编号(首地址)
- \*: 指针标志(表示地址)。
- 数据类型: 通过什么形式取出数据  $\rightarrow$  `*`

- 指针定义: `数据类型 *变量名`

- 定义一个专门用于存储变量地址的盒子。使用该盒子之前, 需要告知以什么形式取出该地址中数据
- 指大小: 与操作系统的地址总线相关。eg. 32位操作系统  $\rightarrow$  32根地址总线  $\rightarrow$  指针大小为4B
- 行为: 如何处理该内存中的数据
  1. 当作一个数据(普通指针)。eg. `char`、`long`、`int`
  2. 当作一个地址(多级指针)。eg. `char*`、`long*`、`int*`
  3. 当作一段代码(函数指针), 存的是代码的入口。eg. `int (*p)(int a, int b);` ——  $\rightarrow$  返回值类型(`*`变量名)(形参列表)
- 指针既可以像数组一样同下标访问(`[]`), 也可以通过(`*`)

```

1 #include <stdio.h>
2
3 int fun(int a,int b){
4 return a + b;
5 }
6
7 int main(){
8
9 int arr[5] = {1,2,3,4,5};
10 int *p = arr; // 等价于 int *p = &arr;
11 int **pp = &p; // 二级指针，pp存储的是p的地址
12 int (*funP)(int,int) = fun; // 函数指针，funP存储的是fun的首地址
13
14 // 无论指针前面的数据类型是什么，指针大小都只和地址总线关
15 printf("%d %d %d\n\n",sizeof(p),sizeof(pp),sizeof(funP)); // 8 8 8
 现在是64位操作系统
16
17 // 二级指针
18 printf("%d %d %d\n",arr[0],*p,**pp); // 1 1 1 数组第一个成员的
 数据
19 printf("%p %p %p\n",&arr[0],p,*pp); // 数组第一个成员地址
20 printf("%p %p\n",&p,pp); // 指针p的地址
21 printf("%p\n\n",&pp); // 指针pp的地址
22
23 // 函数指针
24 printf("%d\n",funP(1,2)); // 3 相当于
 printf("%d",fun(1,2));
25 printf("%p %p\n",funP,fun); // fun的地址
26 printf("%p\n\n",&funP); // funP的地址
27
28 // 指针两种访问形式
29 printf("%d %d\n",*p,p[0]); // 1 1
30
31 return 0;
32 }
33 // 现象
34 /*
35 8 8 8
36
37 1 1 1
38 975ffb80 975ffb80 975ffb80
39 975ffb78 975ffb78
40 975ffb70
41
42 3
43 163614a4 163614a4
44 975ffb68
45
46 1 1
47 */

```

- 变量名：存的谁的内存编号 （p中是arr的首地址）

- \* 变量名：指代谁（\*p是arr中第一个成员）
- **注意**：能拿到地址的方法（指针、数组名）都要很大的权力，只要有了地址，那么你能访问任何一块空间（没有操作系统的情况）。

- C语言编译器不能对指针的访问，进行越界检查，执行时，有可能会被操作系统杀死

- ```

1  int a[10] = {0};
2  int *p = a;
3  a[100] = 10;    /* 能访问,编译器不报错(但是操作系统会检查,如果访问的区域刚好是不能访问的区域,操作系统会杀死进程),但是不合法 */
4  p[100] = 10;    /* 同上 这种写法等价于 *(p + 100) */
      
```

- 地址相关的打印尽量使用%p
- 初始化字符空间 和 非字符空间

- ```

1 // 字符空间
2 // 1.
3 char arr1[] = "hello";
4 // 编译器同调用函数（memcpy或者strcpy），逐个将字符放入arr1空间,0值也会放入arr1空间
5
6 // 2.
7 char arr2[] = {"hello"};
8
9 // 非字符空间
10 // 1.与上面2个的区别是，arr3中没有0值的放入，所以使用%s、strlen等字符串相关函数，会出现不可预测的结果
11 char arr3[] = {'h','e','l','l','o'};

```

- 

- (p+1)和\*p + 1区别

- ```

1  #include <stdio.h>
2
3  int main(){
4      int arr[5] = {1,2,3,4,5};
5      int *p = arr;
6      // int *p = (int *)0x12456778;    // int * 的作用就是告诉编译器，这是一个地址，指向地址内容是整型。如果不加编译器会有警告
7
8      printf("%d\n",*p + 1);              // 2  其实是取出第一个成员1，然后相加
9      printf("%d %d\n",*(p + 1),p[1]);    // 2 2  *(p + 1) == p[1] 本质都是在arr的首地址 + 偏移一个单位（4B）
10     printf("%p %p\n", (p + 1), &p[1]);  // (p + 1) == &p[1]
11
12     return 0;
13 }
14
15 // 现象
16 /*
17 2
18 2 2
      
```

```
19 0000004b2b7ff8d4 0000004b2b7ff8d4
20 */
```

- 相同类型指针做差，结果为二者在内存中的距离

- 两个相同类型的指针相减（两个指针位于同一片连续空间中）= 相差的对象的个数（不是字节数）
- $[x,y] \Rightarrow y - x \Rightarrow$ 和实际个数相同
- $[x,y] \Rightarrow y - x \Rightarrow$ 没有统计y，比实际少统计一个

```
1 #include <stdio.h>
2
3 int main(){
4     int arr[] = {1,2,3,4,5};
5     int *p1 = &arr[0];
6     int *p2 = &arr[sizeof(arr) / sizeof(arr[0]) - 1];    // 指向arr的最后
    一位
7
8     printf("%d", p2 - p1);    // 4 二者之差为 两者相差的盒子个数 编译器会自
    动除以一个sizeof(int)
9
10
11     return 0;
12 }
13
14 // 现象
15 // 4
```

- 指针常量 和 常量指针（以英文为主，中文翻译有多种）

- 指针常量（pointer to constant）（重点是“常量”）：const int *p; 或者 int const *p; 指向的盒子中内容不能用这个变量修改（指向变量的值不能变）
- 常量指针（constant pointer）（重点是“指针”）：int * const p; 指向的方向不能用这个变量修改（存的地址不能变）
- const int * const p; 固定方向，固定数值（指向的地址内容与存的地址都不能）
- 判断方法：
 - 去掉数据类型
 - 看const修饰谁 谁就无法改变（可以使用其他变量获取其地址来改变内容）

```
1 #include <stdio.h>
2
3 int main(){
4
5     // 普通变量
6     int num1 = 10;
7     int num2 = 100;
8
9     printf("num1 = %d, num2 = %d\n", num1, num2);    // num1 = 10,
    num2 = 100
10
11     // 常量指针 不变的是常量（指向的盒子内容）
12     int const *p1 = &num1;    // 即num1中的值无法通过p1进行改变
```

```

13 // *p1 = 20; // 会有报错提示
14 printf("num1 = %d, num2 = %d, *p1 = %d\n", num1, num2, *p1); // num1 =
15 10, num2 = 100, *p1 = 10
16 p1 = &num2; // 可以改变指向的方向（指向另一个变量）
17 printf("num1 = %d, num2 = %d, *p1 = %d\n", num1, num2, *p1); // num1 =
18 10, num2 = 100, *p1 = 100
19 num2 = 200;
20 printf("num1 = %d, num2 = %d, *p1 = %d\n", num1, num2, *p1); // num1 =
21 10, num2 = 200, *p1 = 200
22
23 // 指针常量 不变的是指针（指向方向）
24 int * const p2 = &num1; // 即p2永远指向num1
25 // p2 = &num2; // 会报错
26 printf("num1 = %d, num2 = %d, *p2 = %d\n", num1, num2, *p2); // num1 =
27 10, num2 = 200, *p2 = 10
28 *p2 = 20; // 可以修改盒子中的值
29 printf("num1 = %d, num2 = %d, *p2 = %d\n", num1, num2, *p2); // num1 =
30 20, num2 = 200, *p2 = 20
31 int ** const p = (int ** const)&p2; // &p2的类型是 int * const * 可以通
32 过其他变量来改变
33 *p = &num2;
34 printf("num1 = %d, num2 = %d, *p2 = %d\n", num1, num2, *p2); // num1 =
35 20, num2 = 200, *p2 = 200
36
37 return 0;
38 }
39 // 现象
40 /*
41 num1 = 10, num2 = 100
42 num1 = 10, num2 = 100, *p1 = 10
43 num1 = 10, num2 = 100, *p1 = 100
44 num1 = 10, num2 = 200, *p1 = 200
45 num1 = 10, num2 = 200, *p2 = 10
46 num1 = 20, num2 = 200, *p2 = 20
47 num1 = 20, num2 = 200, *p2 = 200
48 */

```

- 指针常量主要用途：函数传参时保护原内容

● 指针判读技巧

1. 以 变量名 (编译器不认识的值) 为中心，**先右后左**，依次回答问题
2. 遇到 * 升级成指针变量，容量为8B，那么如何访问对象？
3. 以 *变量名 为中心，**先右后左**，依次回答问题
4. 遇到什么，那么访问的对象大小、行为就确定了（遇到 ） 就往反方向看）

- 1 int (*f)(void);
2 // 以f为中心，先右后左，遇到)，反方向
3 // 反方向遇到 *，f提升为指针，容量为8B，如何访问对象？
4 // 以*f为中心，先右后左，遇到(void)，那么对象就是函数，返回值为int，形参为void
5 // 所以这是一个函数指针
6
7 void (*signal(int, void (*)(int)))(int);

```

8 // 以signal为中心, 先右后左, 遇到(int,void*)(int)),所以为signal为函数, 输入参数
  为int 和 函数指针, 那么它的返回值是什么?
9 // 以signal(int,void*)(int))为中心, 先右后左, 遇到),反方向
10 // 遇到*, 返回值是一个指针,那么是什么样的指针呢?
11 // 以(*signal(int,void*)(int))为中心, 先右后左, 遇到(int),所以返回值是一个函数指
  针
12 // signal是一个输入int和函数指针, 返回值为函数指针的函数
13 // 可以改写成
14 typedef void (*func)(int);
15 func signal(int, func);

```

- 多维指针

- 常使用的多维指针 —— 二维指针
- 本质: 将内存中不连续的多个离散空间, 在逻辑上变成线性结构
- 二维指针指向的一个一位数组空间, 只时这个数组里的每个元素是另外空间的首地址
 - 一个连续钥匙串, 每个环上挂一把钥匙
 - 物理上不连续的空间, 逻辑上要求连续
 - 常见: main函数里的形参 `char *argv[]`
- 二维指针可以优化switch分支, 维护性更好, 将离散空间统一到一个连续空间中

```

1  /* 测试3: 二维指针 */
2  /* 使用场景 ---- 每周的每天做一件事情(函数),但是每个周事情可能会变,怎么拓展才能
   不动该代码,实现事情的变更 */
3  typedef void (*work)(void);
4
5
6  /**
7   * @brief 每天工作函数
8   *
9   * @param work_arr 传入一周7天的工作内容的数组
10  * @param num 设置了一周的几天
11  */
12 void test03DayWork(work *work_arr, int num){
13     if(work_arr == NULL){ return; }
14
15     /* 1. 获取当前星期 */
16     time_t now_time = time(NULL); /* 获取当前时间戳
17
18     */
19     struct tm *tm_struct = localtime(&now_time); /* 转换成时间结构体
   (包含了星期) */
20     int day = tm_struct->tm_wday; /* 获取星期
   对应的数字(0~6) 0表示星期天 */
21     printf("%d\n",day);
22
23     /* 2. 当前星期匹配对应的工作 */
24     if((day + 1) > num){ return; }
25     work_arr[day]();
26 }
27
28 void func1(){ printf("day1"); }
29 void func2(){ printf("day2"); }

```

```

28 void func3(){ printf("day3"); }
29 void func4(){ printf("day4"); }
30 void func5(){ printf("day5"); }
31 void func6(){ printf("day6"); }
32 void func7(){ printf("day7"); }
33
34 void test03(){
35     work work_arr[] = {
36         func7,
37         func1,
38         func2,
39         func3,
40         func4,
41         func5,
42         func6
43     };
44
45     test03DayWork(work_arr, sizeof(work_arr) / sizeof(work_arr[0]));
46 }

```

■

11. 字符串

- 二进制空间

- 按照数值解释区分

1. 数据空间（无符号空间）：使用unsigned修饰，单纯表示数量、大小、地址偏移、位标志等，没有负数
2. 数值空间（有符号空间）：使用signed修饰，有正负、数学运算有意义，如：温度、速度

- 按照字符区别

1. 字符空间

- C语言：多个字节的空间里，如果把0值当做结束标志的空间，称为字符空间（可读可写），特殊字符空间——字符常量（只读）
- 最小单位 1B
- 存放字符

2. 非字符空间

- 0值有意义
- 有外部变量约束这个空间的大小
- 存放二进制数据

- C语言描述字符空间

- ```

1 /* 指向一个int空间的首地址 */
2 int a = 0;
3 int *p = &a;

```

```

4
5 /* 指向一片int的数组空间（连续空间） */
6 /* p和5共同描述这片空间，描述复杂空间时，指针和个数可以封装成结构体 */
7 int arr[5] = {0};
8 int *p = arr;
9
10 /* 指向一个char空间的首地址 */
11 char a = 0;
12 char *p = &a;
13
14 /* 指向一片char的数组空间（连续空间） */
15 char *p = "hello"; /* 约定了0值为结束标志 —— 字符空间 */
16 char data[2] = {'a','b'}; /* 有个数限制 —— 非字符空间 */
17
18 // 部分编译器会认为unsigned char 和 char 是一样的，都是描述数据的（无符号）
19 // 后期项目开发描述一个传感器采集数据集的数据空间（1B为一个采集单元）推荐使用uint8_t代
 替unsigned char,理由是：可读性高

```

- 字符空间的访问 —— while遍历（终止条件为 遇到0值）
- 非字符空间的访问 —— for遍历（终止条件为 空间尽头）
- 字符串的本质
  - 字符串通常以 '\0' 结尾（\0：结束标志），所以字符串长度比看到的字符多一个'\0'
  - char \*p = "xxx"; "xxx"本质上是字符串的首地址

```

1 #include <stdio.h>
2
3 int main(){
4 // char p[] = "ni hao";
5 char *p = "ni hao";
6 char arr[100] = {0};
7
8 printf("%s\n",arr); // 空白 即'\0'
9
10 // 字符串的大小
11 printf("%d\n",sizeof(p)); // 8 指针大小 如果p是数
 组, 那么大小为7
12 printf("%d\n",sizeof("ni hao")); // 7 比ni hao多以一个\0
13 printf("%d\n", sizeof(*p)); // 1 char类型
14
15 // "xxx"的本质
16 printf("%p %p\n",&("ni hao"),p);
17
18 int i = 0;
19 do{
20 arr[i] = *p;
21 i++;
22 p++;
23 }while(*p != '\0');
24
25 printf("%s\t%c",arr,arr[6]);
26
27 return 0;

```

```

28 }
29
30 // 现象
31 /*
32
33 8
34 7
35 00007ff6c0f19000 00007ff6c0f19000
36 ni hao
37 */

```

- 字符串的赋值

- 字符数组的大小

1. 看[]中的数字
2. []中没有数字，看初始值的数量

- `char str[] = 'xxx';`:

- 编译器会将“xxx”转换成{'x','x','x','\0'};
- “xxx”存储在字符常量区 程序运行时将“xxx”从常量区拷贝给栈区的str

- `sizeof(str);`

- str为变量名 / 数据类型：变量的大小;
- str为数组名：数组总空间的大小;
- str为字符串：字符串的大小（包括'\0'）。      字符串长度（不包括'\0'）

- `printf()`中的占位符: %s 将后续变量当成字符串地址，将其中的内容以一个字符一个字符的打印出来。

```

1 #include <stdio.h>
2
3 int main(){
4
5 int num1 = 0x0061626364;
6 printf("%s\n",&num1); // 去num1的盒子中以一个字节一个字节的读取打印
7 printf("%s\n",num1); // 去0x0061626364中读取打印（会报段错误，内
 存中的这个片区域可能没有读取权限）
8
9 return 0;
10 }
11
12 // 现象
13 /*
14 PS E:\project\cProject> .\tt.exe
15 dcbaHp蹦?
16 PS E:\project\cProject> echo $?
17 False
18 */

```

- 段错误：访问的内存没有对应权限（例如：修改只读区的内容）

- 字符空间的行为（拷贝、拼接、统计）—— 相关的函数

- 相关函数头文件 `#include <string.h>`

o `strlen()`;

- 统计字符串长度，不包括'\0'（从传入的首地址开始，统计字符个数，直到遇到0值为止）
- `strlen() - 1` 后期项目中，常用于处理将回车符替换成0值

```
1 // 函数原型
2 /**
3 * strlen - Find the length of a string
4 * @s: The string to be sized
5 */
6 size_t strlen(const char *s)
7 {
8 const char *sc;
9
10 for (sc = s; *sc != '\0'; ++sc) // 使用这个sc =
s;while((*sc ++) != '\0'); sc的指针在'\0'之后，就是计算了'\0'
11 /* nothing */;
12 return sc - s;
13 }
14
15 // 自己的统计字符串长度
16 int my_strlen(char const *str){
17 int cnt = 0;
18 while(*str != '\0'){
19 str++;
20 cnt++;
21 }
22 return cnt;
23 }
24
```

o `strcpy()` 和 `strncpy()`

- 拷贝字符串（`strcpy()`：完整拷贝; `strncpy()`：部分拷贝）
- `strcpy`：不会考虑目标区域能不能接收完源数据（会存在越界可能）。—— **不推荐使用**，一旦超出会修改临界区域的值 <= 扣钱函数

■ `strncpy`

| 源空间与输入的<br>count (拷贝长度) 关系           | 结果                                                                                   | 注意                                              |
|--------------------------------------|--------------------------------------------------------------------------------------|-------------------------------------------------|
| <code>strlen(src) &gt; count</code>  | 部分拷贝，不会将0值拷贝到目标空间                                                                    | 后续需要留0值的位置，所以这种情况需要sizeof(dest) - 1，后续手动添加0值到末尾 |
| <code>strlen(src) &lt;= count</code> | 全部拷贝，剩余的空间会用0值进行填充<br>有些编译器的厂家实现的<br><code>strncpy</code> 会在src结束之后直接退出（只拷贝src长度的字符） | 如果是直接退出，也需要手动添加0值                               |

```

1 // 函数原型
2 /**
3 * strcpy - Copy a %NUL terminated string
4 * @dest: Where to copy the string to
5 * @src: Where to copy the string from
6 */
7 char *strcpy(char *dest, const char *src)
8 {
9 char *tmp = dest;
10
11 while ((*dest++ = *src++) != '\0') // 先用完，在加加 *dest
12 = *src; *dest != '\0'; *dest++;*src++
13 /* nothing */;
14 return tmp;
15 }
16 /**
17 * strncpy - Copy a length-limited, C-string
18 * @dest: Where to copy the string to
19 * @src: Where to copy the string from
20 * @count: The maximum number of bytes to copy
21 *
22 * The result is not %NUL-terminated if the source exceeds
23 * @count bytes.
24 *
25 * In the case where the length of @src is less than that of
26 * count, the remainder of @dest will be padded with %NUL.
27 */
28 char *strncpy(char *dest, const char *src, size_t count)
29 {
30 char *tmp = dest;
31
32 while (count) {
33 if ((*tmp = *src) != 0)
34 src++;
35 tmp++;
36 count--;
37 }
38 return dest;
39 }
40
41 // 完整拷贝
42 char * my_strcpy(char *s1, char const *s2){
43 char *t = s1; // 保留首地址
44 char t1 = 0; // 用于比较
45
46 do{
47 *s1 = *s2;
48 t1 = *s1;
49 s1++;
50 s2++;
51 }

```

```

52 }while(t1 != '\0');
53
54 return t;
55 }
56
57 // 不完整拷贝
58 // 如果 字符串长度 >= 截取的数量 只用截取需要的部分;
59 // 如果 字符串长度 < 截取的数量 需要将剩下的填充'\0'
60 char * my_strncpy(char *s1,char const *s2,int num){
61 char *t = s1; // 保留首地址
62 char t1 = *s2; // 用于比较
63
64 // 先部分拷贝
65 while(num && t1 != '\0'){
66 *s1 = *s2;
67 num--;
68 s1++;
69 s2++;
70 t1 = *s2;
71 }
72
73 // 剩余填充
74 while(num--){
75 s1++;
76 *s1 = '\0';
77 }
78
79 return t;
80 }
81

```

#### o strcat()

- 将字符串拼接在一起

```

1 // 函数原型
2 /**
3 * strcat - Append one %NUL-terminated string to another
4 * @dest: The string to be appended to
5 * @src: The string to append to it
6 */
7 char *strcat(char *dest, const char *src)
8 {
9 char *tmp = dest;
10
11 while (*dest)
12 dest++;
13 while ((*dest++ = *src++) != '\0')
14 ;
15 return tmp;
16 }
17
18 // 字符串拼接
19 char *my_strcat(char *dest,char const *src){

```

```

20 char *t = dest; // 保留dest的首地址
21
22 while(*dest != '\0'){
23 dest++; // 将指针dest移动到字符串末尾
24 }
25
26 do{
27 *dest = *src;
28 dest++;
29 }while(*src++ != '\0'); // src++写在里面会漏拷贝'\0'
30
31 return t;
32 }
33

```

○ `strcmp()`、`strncmp()` 和 `strcasecmp()`

- 比较字符串是否相等（遇到'\0'就会停下）
- `strcmp()`：完全相等
- `strncmp()`：部分相等
- `strcasecmp()`：忽略大小写的完全相等

```

1 // 函数原型
2 /**
3 * strcmp - Compare two strings
4 * @cs: One string
5 * @ct: Another string
6 */
7 int strcmp(const char *cs, const char *ct)
8 {
9 unsigned char c1, c2; //使用unsigned char 跨平
10 台兼容，以及对负数值的处理
11 while (1) {
12 c1 = *cs++;
13 c2 = *ct++;
14 if (c1 != c2)
15 return c1 < c2 ? -1 : 1;
16 if (!c1)
17 break;
18 }
19 return 0;
20 }
21
22 /**
23 * strncmp - Compare two length-limited strings
24 * @cs: One string
25 * @ct: Another string
26 * @count: The maximum number of bytes to compare
27 */
28 int strncmp(const char *cs, const char *ct, size_t count)
29 {
30 unsigned char c1, c2;

```

```

31
32 while (count) {
33 c1 = *cs++;
34 c2 = *ct++;
35 if (c1 != c2)
36 return c1 < c2 ? -1 : 1;
37 if (!c1)
38 break;
39 count--;
40 }
41 return 0;
42 }
43
44 int strcasecmp(const char *s1, const char *s2)
45 {
46 int c1, c2;
47
48 do {
49 c1 = tolower(*s1++);
50 c2 = tolower(*s2++);
51 } while (c1 == c2 && c1 != 0);
52 return c1 - c2;
53 }
54
55 /*
56 // 大写字母二进制第 6 位是 0 小写字母二进制第 6 位是 1
57 int tolower(int c)
58 {
59 if ((unsigned int)c - 'A' < 26)
60 return c | ('a' - 'A'); // 位运算更快
61 return c;
62 }
63 */
64
65
66 // 字符串对比 相等输出0，大于输出1，小于输出-1
67 // 完全相等
68 int my_strcmp(char const *p1, char const *p2){
69 while(*p1 != '\0' && *p2 != '\0' && *p1 == *p2){
70 p1++;
71 p2++;
72 }
73
74 return *p1 - *p2; // 相等输出0，大于输出正数，小于输出负数
75 }
76
77 // 在限制范围中比对
78 int my_strncmp(char const *p1, char const *p2, int cout){
79 while(cout != 0 && *p1 != '\0' && *p2 != '\0' && *p1 == *p2){
80 p1++;
81 p2++;
82 cout--;
83 }

```

```

84 if(cout == 0){
85 return 0;
86 }else{
87 return *p1 - *p2;
88 }
89 }
90
91
92 // 不区分大小写的 完全比对 使用tolower() 将字符中的字母转换成小写
93 int my_strcasecmp(char const *p1, char const *p2){
94 while(*p1 != '\0' && *p2 != '\0' && tolower(*p1) ==
95 tolower(*p2)){
96 p1++;
97 p2++;
98 }
99 return tolower(*p1) - tolower(*p2); // 相等输出0, 大于输出
100 1, 小于输出-1
101 }
102

```

◦ `sprintf()` 和 `snprintf()`

- 格式化字符串的写入数组，都会自动拷贝0值
- `sprintf()` 与 `strcpy()` 一样不安全，不推荐使用（扣钱函数）
- **推荐使用** `snprintf`：内部会自动添加0值，所以实际拷贝的字符个数为  $n - 1$
- 具体实现，[见github](#)

```

1 /* 测试6: sprintf 和 snprintf */
2 void test06(){
3 int a = 0x00313233;
4 char data1[10] = {0};
5 char data2[10] = {0};
6 printf("%s\n", (char *)&a);
7
8 /* 危险拷贝 */
9 sprintf(data1, "hello , %s", "world!");
10
11 printf("%s \n", data1);
12
13 printf("%s\n", (char *)&a);
14
15 /* 限制拷贝 */
16 snprintf(data2, sizeof(data2), "hello , %s", "world!");
17 printf("%s \n", data2);
18 }

```

◦ `strstr()`：从一个字符空间找另一个字符空间一样的内容 长串中找短串

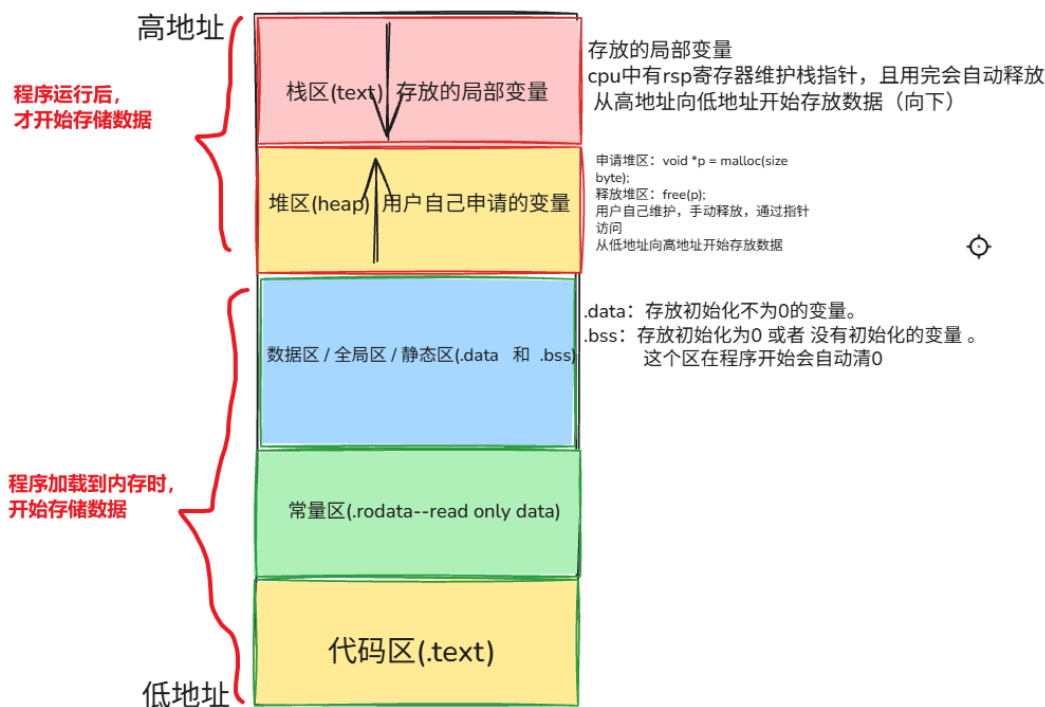
- `memcpy()`：内存拷贝，能拷贝0值，字节拷贝
- `memset()`：内存区域设置某值 不能做到连续空间中每个对象设置成相同数（int 4B） 常用于清零 字节设置

## 12. C语言虚拟内存分区

- 32位操作系统，理论内存大小有 $2^{32}B = 4GB$ ； 64位操作系统，理论内存大小有 $2^{64}B = 2^{34}GB$
- 内存分段管理（大概概念上，当前层出现解决不了的问题就在这层基础上加一层）
  - 操作系统（OS）为了保证系统的运行稳定性，每个程序运行时的环境（内存空间）进行分段管理
    - 将 程序访问的资源 映射成 CPU能够寻址到的空间
    - 操作系统提供虚拟地址（统一地址，让每个程序都以为自己使用了全部的内存），然后将虚拟地址 映射成 物理地址（物理空间不统一）
    - 虚拟地址的大小 == OS的位数有关
      - 物理内存 不能超过 虚拟内存
      - 64位的操作系统 真实寻址能力为48位
- 虚拟内存：一种抽象的内存，让每个进程都以为自己使用的是全部的主存。（目的：为了解决主存的容量问题）
- 虚拟地址：理论的寻址空间（理论内存大小）。——硬件MMU映射——> 实际的物理地址（自己装的内存条。如：16G内存条）
- 程序中的所说的地址都是虚拟地址，其理论需要多少根数据线/地址线，实际的可能比理论少
- C程序的虚拟内存空间

○

点击并拖动，完成可拖拽



- 代码区的起始地址不是0x0，而是大于0x0的地址，操作系统将0x0~代码区的起始地址保留了。
- 有些分法将常量区合并到了数据区，有存在将常量区合并到代码区的
- Linux的全部的虚拟内存空间（（4G大小的虚拟空间为例进行分段 —— 现代操作系统多为分页）

○

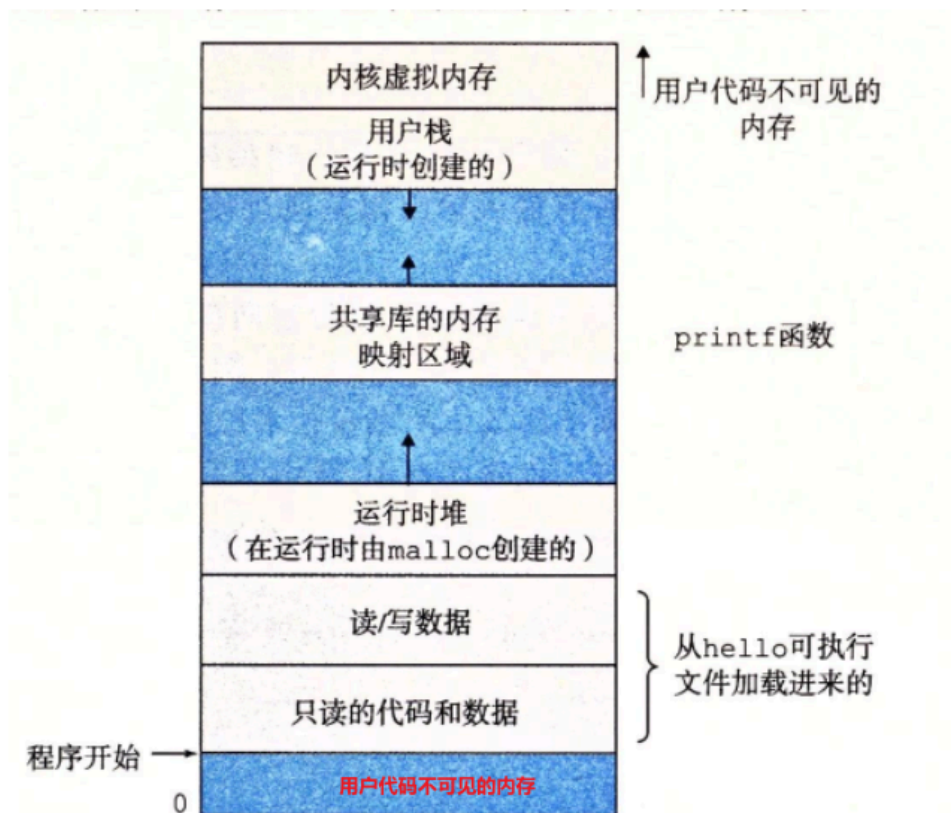


图 1-13 进程的虚拟地址空间

○ 分段

- 管理时机：编译器 在链接时 确定 每个目标文件（所有的.c文件，第三方库）在内存所在的位置
- 操作系统存在一个分段分区的标准，来约束不同编译器使用内存（虚拟内存）
- 面向硬件编程 没有OS，但是编译器默认有 分段行为 每个段所在的基地址 结合 具体硬件 进行配置

○ 段错误（segment fault）是操作系统报的错误

○ 用户代码不可见的内存 —— 内核空间（操作系统的内存）应用程序读写都报错

| 分段名                         | 功能        | 生命周期                            | 内部变量创建时机 |
|-----------------------------|-----------|---------------------------------|----------|
| 栈段                          | 局部变量      | 编译器来维护生命周期，不使用就释放（函数调用 —— 函数退出） | 运行时      |
| 堆段                          | 动态申请的空间   | 程序员来维护，手动释放                     | 运行时      |
| 数据段(.data) —— 只读区 (.rodata) | 常量（只读）    | 程序开始 —— 程序退出                    | 编译时      |
| 数据段 —— 全局区、静态区 (.data)      | 全局变量、静态变量 | 程序开始 —— 程序退出                    | 编译时      |

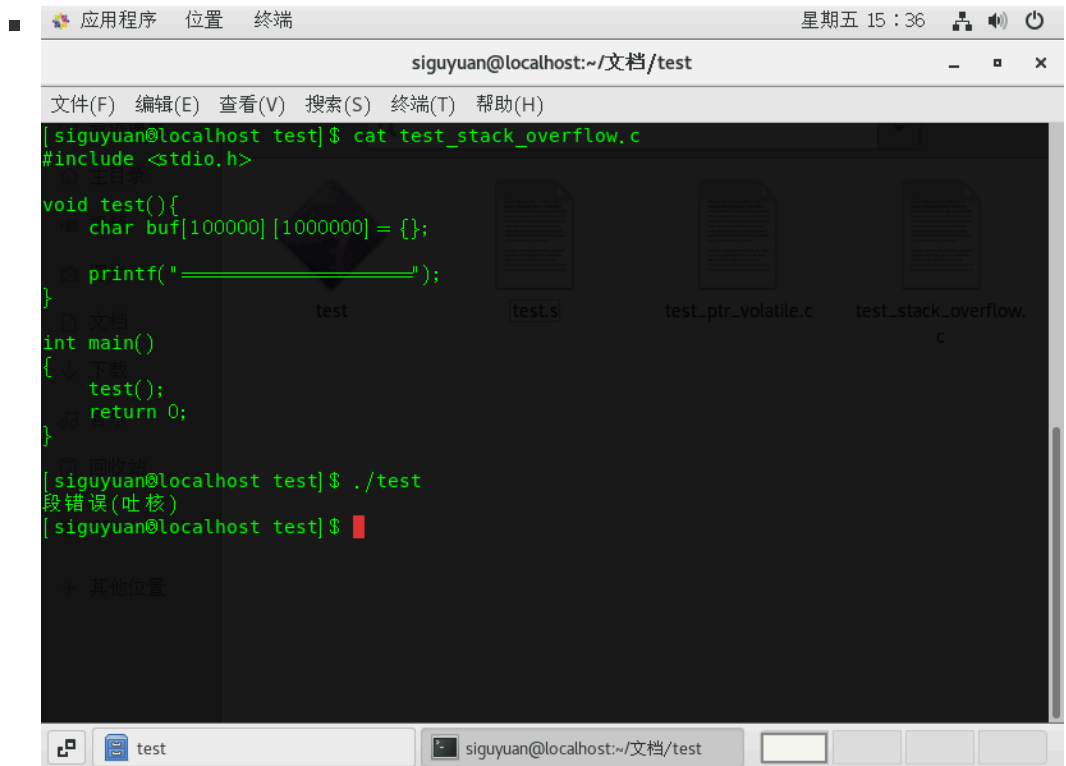
○

| 分段名                 | 功能                              | 生命周期         | 内部变量创建时机 |
|---------------------|---------------------------------|--------------|----------|
| 数据段 —— 未初始化区 (.bss) | 未初始化全局变量、静态变量（后续自动初始化，利用0进行占位）  | 程序开始 —— 程序退出 | 编译时      |
| 代码段(.text)          | 所有的函数（指令）——按指令解析（只读）——可以通过反汇编查看 | 程序开始 → 程序退出  | 编译时      |

- 程序的编译、链接后的可执行文件大小 ≈ 代码段 + 数据段（静态段）
- 一般定义任何空间时，可以通过一些方法指定该空间在链接时，分配到哪个段中（通常使用默认分段）
- 

| 管理者        | 操作系统 | 编译器 | 程序员 |
|------------|------|-----|-----|
| 管理区域       |      |     |     |
| .text      | 管理   |     |     |
| .rodata    | 管理   |     |     |
| .bss/.data |      | 管理  |     |
| 栈          |      | 管理  |     |
| 堆          |      |     | 管理  |

- 堆区与栈区
  - 栈区
    - 无法动态申请空间
    - 根据声明情况（编译期），调整 栈指针 实现申请空间，函数返回时，调整栈指针 释放空间（数据还在）
    - 栈底的地址是高地址，栈顶的地址是低地址，数据从栈底→栈顶存（高地址->低地址）。效率高
    - 栈区存的数据其地址一定连续；
    - 栈区的大小限制：希望整个栈区能够为更多的子函数提供服务
      - 每个子函数的局部变量 有一个大小限制
      - 以后的交叉编译（PC上写代码 开发板 上 运行）PC上写的代码默认以PC为基础分配栈，移植到更小的开发板，会导致闪退 =》引发栈溢出 =》所以局部变量尽量少
      - 递归函数 =》容易引发栈溢出
      - 栈溢出 == 栈装不下了



```
siguyuan@localhost: ~/文档/test
文件(F) 编辑(E) 查看(V) 搜索(S) 终端(T) 帮助(H)

[siguyuan@localhost test]$ cat test_stack_overflow.c
#include <stdio.h>
void test(){
 char buf[100000] [1000000] = {};
 printf("====");
}
int main()
{
 test();
 return 0;
}
[siguyuan@localhost test]$./test
段错误 (吐核)
[siguyuan@localhost test]$
```

堆区

- 运行时，向操作系统申请 **可变空间（动态空间）**；操作系统维护 哪些空间被使用；向操作系统提交注销空间
- 堆区：堆底是低地址，堆顶是高地址，数据从堆底—>堆顶存（底地址->高地址）
- 堆区存的数据其地址不一定连续
- 操作系统 申请了一个空闲空间（运行时才能知道空间大小），返回空间的首地址（匿名空间）—— malloc（不同操作系统 调用的接口不同）—— 用一个指针接收

```
1 xx *p = malloc(单位是字节 sizeof(每个元素的数据类型) * 个数);
2 if(p == NULL) {...}
3 free(p); // 不能重复释放
4 p = NULL; // 防止重复释放导致系统杀死 free(0 / NULL); 不做任何操作
```

```
1 #include <stdlib.h>
2 #include <stdio.h>
3 #include <string.h>
4
5 void test(){
6 char *p = (char *)malloc(sizeof(char) * 0);
7 if(p == NULL){
8 printf("null\n");
9 }else{
10 printf("no null\n");
11 }
12
13 printf("%p\n", p);
14 printf("%c\n", *p);
15
16 free(p);
17 printf("+++++\n");
```

```

18 }
19
20 int main(){
21
22 test();
23
24 return 0;
25 }
26 /*
27 no null
28 0000015957F8ADE0
29 ?
30 ++++++++
31
32 这个的结果有两种
33 堆区内内存耗尽会返回 NULL
34 堆区内内存还要会返回一个地址
35 */

```

- 堆溢出 == 一个堆上的盒子存放了超出盒子容量的数据 堆内存耗尽 == 堆装不下了
  - 内存泄漏：匿名空间存在，但找不到了（没有释放） 程序继续能运行，但是一旦过多内存泄漏，系统内存会被占满（一般存在操作系统，操作系统会自动检测，并杀死这种程序）

```

siguyuan@localhost:~/文档/test
[siguyuan@localhost test]$ cat test_memory_leak.c
#include <stdio.h>
#include <stdlib.h>

int main() {
 printf("开始无限分配内存...\n");
 while (1) {
 void* p = malloc(10 * 1024 * 1024); // 每次分配 10MB
 if (p == NULL) {
 printf("malloc 失败, 内存不足!\n");
 sleep(1);
 } else {
 // 不 free, 制造泄漏; 也可以 memset 写入数据加速消耗
 memset(p, 0xAA, 10 * 1024 * 1024);
 }
 }
 return 0;
}

[siguyuan@localhost test]$./test
开始无限分配内存...
已杀死
[siguyuan@localhost test]$

```

- double free：多次释放堆区，程序会被OS杀死

```

[siguyuan@localhost test]$ cat test_heap_overflow.c
#include <libio.h>
#include <stdio.h>
#include <stdlib.h>

void test(){
 char *p = (char *)malloc(sizeof(char));
 if(p == NULL){ return; }
 free(p);
 free(p);
}

int main()
{
 test();
 return 0;
}

[siguyuan@localhost test]$./test
*** Error in `./test': double free or corruption (fasttop): 0x0000000000a3a010 ***
===== Backtrace: =====
/lib64/libc.so.6(+0x81329)[0x7f5f73b91329]
./test[0x4005b4]

```

```

1 #include <stdio.h>
2 #include <stdlib.h>
3
4 int main()
5 {
6 int *a = malloc(4); // 开辟了4B大小的空间（一个int的空间）
7 // int *b = malloc(4);
8 int *b = a + 1;
9 char *c = malloc(4); // 开辟了4B大小的空间（四个char的空间）
10
11 a[0] = 10;
12 // b[0] = 1;
13 *b = 1;
14 c[0] = 100;
15 printf("%d %d %d\n",a[0],b[0],c[0]); // 10 1 100
16
17 a[1] = 20;
18 /*
19 a越界（溢出）问题：
20 a如果越界，会修改b指向空间的内容 10 20 20 100
21 a如果没越界，不会对b有任何变动 10 20 1 100
22
23 使用int *b = malloc(4);
24 会出现两种结果：
25 a与b刚好分配到连续空间上 a越界了（使用了b的空间） 所以会打印 10 20 20 100
26 a与b没有分配到连续空间上 a越界了（使用了其他变量的空间，对b无影响） 所以会打印
27 10 20 1 100
28 */
29 printf("%d %d %d %d\n",a[0],a[1],b[0],c[0]); // 10 20 2
30
31 printf("%d %d\n",sizeof(a),sizeof(*a));
32
33 free(a);
34 // free(b); // free 只能释放 malloc/calloc/realloc 返回的地址
35 free(c);
36
37 return 0;

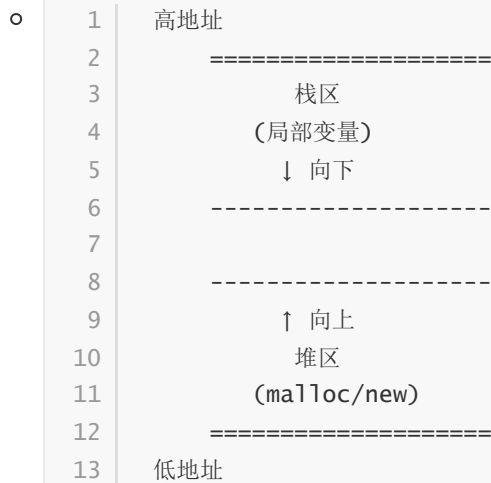
```

```

37 }
38
39 // 结果
40 /*
41 10 1 100
42 10 20 20 100
43 8 4
44 */

```

- `free(str)` 只是修改 `str` 指向的空间中的内容，没有修改 `str` 的指向。



- 数据区

- 静态区 (static 关键字)
  - 修饰的这个空间 分配到数据区 (分配到生命周期很长的区域 —— 程序结束死亡)
  - 与全局变量 区别
    - 全局变量 所有子函数，都能访问 (提升作用范围)
    - 静态变量 不会改变变量的作用范围，常驻内存，只初始化一次 (如果是赋值，则是多次)

## 13. 函数

- 函数

- 函数名
  - 等价于 **标签** (常量地址 —— 编译器编译时临时使用，一旦生成了可执行文件，常量地址运行时不占用空间，常量地址嵌入到指令中) 数组名同理

- ```

1     /* 函数名 -- 不占用内存空间 */
2     /* main.c */
3     #include <stdio.h>
4
5     extern int fun();
6
7     void test(){
8         fun();
9     }

```

```

10
11     int main(){
12         test();
13         return 0;
14     }
15
16     /* func.c 与main.c同一级目录 */
17     int fun(){
18         return 0;
19     }

```

■ `gcc -c -o test_fun_name.o .\test_fun_name.c objdump -d .\test_fun_name.o`

■

■

```

0000000000000000 <test>:
   0:  55                push    %rbp
   1:  48 89 e5          mov     %rsp,%rbp
   4:  48 83 ec 20       sub     $0x20,%rsp
   8:  e8 00 00 00 00    call    d <test+0xd>
  d:  90                nop
  e:  48 83 c4 20       add     $0x20,%rsp
 12:  5d                pop     %rbp
 13:  c3                ret

```

■ `gcc -o test_fun_name .\test_fun_name_fun.c .\test_fun_name.c objdump -d`

`.\test_fun_name.exe > test_fun_name.txt` 【图中的调整 改为 跳转】

```

00000001400016e0 <test>:
1400016e0:  55                push    %rbp
1400016e1:  48 89 e5          mov     %rsp,%rbp
1400016e4:  48 83 ec 20       sub     $0x20,%rsp
1400016e8:  e8 e3 ff ff ff    call    1400016d0 <fun>
1400016ed:  90                nop
1400016ee:  48 83 c4 20       add     $0x20,%rsp
1400016f2:  5d                pop     %rbp
1400016f3:  c3                ret

```

两次的地址不一样，说明将函数偏移地址嵌入到指令中

o

```

1  #include <stdio.h>
2
3  int fun(int const num1, int const num2){
4      return num1 + num2;
5  }
6
7  int main(){
8
9      printf("-- fun name --\n");
10
11      // 函数名 ==> 函数的首地址（入口地址）
12      printf("%p\n",fun);
13      printf("%p\n",&fun);
14

```

```

15     printf("-- size --\n");
16
17     // 大小  函数是代码，不是数据，所以大小无意义，sizeof无法函数真实的函数大小，返回1
    的状态
18     printf("%d\n",sizeof(fun));
19     fun(100,200);
20     printf("%d\n",sizeof(fun));
21
22     printf("-- offset --\n");
23
24     // 偏移  偏移都是指针指向类型的偏移 char类型
25     // 函数地址的运算 没有实际的意义，不同编译器有不同的结果（描述行为）
26     printf("%p\n",fun + 1);
27     printf("%p\n",&fun + 1);
28
29     return 0;
30 }
31 // 现象
32 /*
33 -- fun name --
34 00007ff725bd14a4
35 00007ff725bd14a4
36 -- size --
37 1
38 1
39 -- offset --
40 00007ff725bd14a5
41 00007ff725bd14a5
42 */

```

- 函数的输入参数 —— 接收信息

- 函数的声明（定义）本质是告诉编译器这个名字表示什么意义。之后的函数使用前的声明，主要在头文件中声明
- 形参列表：调用时给形参申请局部空间，由函数调式时，传递数据（赋值）
 - 值传递、地址传递本质就是拷贝，只是一个拷贝内容的副本，一个拷贝地址的副本
- 函数的调用者，传递实参，**依然遵守大小容量的转换（截断、补齐）**
- 函数中，如何接收任何空间
 1. 独立空间，一个数据类型
 - 基本数据类型的变量已经存在，直接拷贝给函数形参
 - 结构体/共用体：是数据类型。支持拷贝，一般结构体都是多个成员组成，拷贝结构体的时间和空间开销大，一般传地址
 - 数组：不是数据类型，不能拷贝
 2. 连续空间（如果是只查看——用const修饰，如果需要修改——不使用const）
 3. 字符连续空间
 - 隐式的结束标志：把0值做为结束标志的空间的首地址传递到函数形参中

- ```

1 // 例如: printf就只需要查看, 不需要修改
2 int printf(const char * _Format,...);
3 // snprintf的format(源空间)只读, __stream(目标空间)需要写
4 int snprintf (char * __stream, size_t __n, const char *
 __format, ...);

```

#### 4. 非字符连续空间

- 数字空间/特殊空间

- 没有结束标志, 一般会传递个数做为结束约束
- 如果是特殊空间, 还需要明确知道一个连续空间有多大

- ```

1 void f3(const struct vip_info *table, int num); // 读取
  num个vip_info的表单, 只读
2 void f3(struct vip_info *table, int num); // 更新
  vip_info里的信息, 最多更新num个, 可改
3
4 // 例子
5 // 二维数组
6 // 进行图像的按行均值化处理
7 int blur(char (*image)[600], int row);

```

- 纯数据空间

- 需要考虑如何在函数形参中表示任意空间 —— void *
- void *: 不能做为变量的声明, 不能做为实参, 只能在形参中接收一个任意空间的首地址 (万能指针)
- 只能在形参中出现, 收到任意空间首地址后, **一般按照字节为单位进行逐个处理**
声明时, 还需要限制以字节为单位的个数, 需要进行强制转换成 char *

- ```

1 void voice_handle(const void *, size_t num); // 只读
2 void voice_handle(void *, size_t num); // 可修改
3
4 // 他的可变参数, 就需要空间地址
5 int scanf(const char * _Format,...);
6 scanf("%d", &c);

```

- 函数的返回值 —— 反馈信息

- 调用者有一块空间, 调用函数之后, 这块空间中的内容改变
- 返回形式

##### 1. 通过函数输入参数返回。以下为条件

- 调用者必须传递要更新空间的首地址。&
- 被调用者里, 必须访问这个地址指向空间, 并进行改变 (放在等号左边)
  - 二维指针应用场景
    - 出现在函数形参时, 可能表示更新一维指针空间
    - 出现在函数形参时, 可能表示一个数组空间, 只是这个空间里保存其他空间的首地址 (让物理上不连续的空间, 逻辑上变得连续)

- 不仅仅有二维指针，而且还要一个数量来约束这个空间
- 有些会约定NULL (0值) 为结束标志
- `strtol()`: 字符转换成数字，常用。to表示转换；ul表示转换成unsigned long

```

1 /**
2 * @brief 字符串转换成数字
3 *
4 * @param _Str 需要转换的字符串
5 * @param _EndPtr 返回转换结束时的指针位置，通过这个判读是否完全转换
6 * @param _Radix 转换的进制
7 * @return unsigned 转换后的数字
8 */
9 unsigned long strtoul(const char * _Str, char ** _EndPtr, int
 _Radix);
10
11 void test02(){
12 char *s = "123@123.com";
13 char *end = NULL;
14 unsigned long num = 0;
15 num = strtoul(s, &end, 10);
16
17 printf("%lu, %c\n", num, *end);
18 }
19
20 /**
21 123, @
22 */

```

## 2. 通过函数返回值返回

- 逐位拷贝，一旦返回值是指针，指向的空间，必须保证函数返回后，依然有效
  - 返回指针时，可返回以下地址
    - .rodata区：常量（不常用）
    - .data区：全局变量（不常用）、静态变量
    - 堆区
      - 区别堆区和.data区
        - 返回堆区，一般接口是成对出现（有创建、有销毁 —— 都是创建者提供，销毁时尽量封装，不要让使用者来销毁，因为使用者不一定知道具体数据结构）
    - .data区：只用一个接口

• 函数指针：函数返回值类型 (\*指针变量名)(函数形参的数据类型) = 函数名；

- 调用：(\*指针变量名)(); `<==>` 指针变量名();
- 保存函数名的指针变量

|          | 函数指针(回调函数、钩子函数) | 普通指针 |
|----------|-----------------|------|
| 占用地址大小容量 | 4/8B            | 4/8B |

|     | 函数指针(回调函数、钩子函数)                  | 普通指针            |
|-----|----------------------------------|-----------------|
| 运算  | $\pm$ 没有意义                       | $\pm 1$         |
| 取内容 | <code>(*func)() == func()</code> | <code>*p</code> |

○ 函数指针使用场景

1. 把物理上离散的函数名，变成逻辑上连续的 —— 函数指针数组
2. 跨平台设计时、设计时使用（统一接口指向了不同驱动的具体实现函数） —— 解耦（让各个模块独立）

```

1 /* 不同身份客户，使用不同功能（VIP调用不同的函数，普通调用另一个函数） */
2 /* 不同类型的用户显示不同的效果 */
3 void showVIP1(char *s){
4 printf("VIP1, %s\n", s);
5 }
6 void showVIP2(char *s){
7 printf("VIP2, %s\n", s);
8 }
9 void showVIP3(char *s){
10 printf("VIP3, %s\n", s);
11 }
12
13 typedef struct client_s{
14 void (*showVIP)(char *); /* 客户身份显示 */
15 /* 其他客户信息 */
16 }Client_t ;
17
18 void test01(){
19 Client_t client1 = {
20 .showVIP = showVIP1
21 };
22 client1.showVIP("你好");
23
24 Client_t client2 = {showVIP2};
25 (*client2.showVIP)("你好");
26 }
27
28 /*
29 VIP1, 你好
30 VIP2, 你好
31 */

```

- 
- 有趣的应用（网络安全方面）：将函数（如：病毒函数）的二进制封装到数组中，再通过函数指针将数组当成函数使用。（需要申请内存的可执行权限，尽量函数不要调用其他函数——其他函数调用是通过当前函数的地址进行偏移）

```

o 1 E:\project\cProject> gcc .\day09\test01.c -o tt.o -c
2 E:\project\cProject> objdump -d tt.exe | Select-String "fun" -Context 0,50
3 > 000000001400014a4 <fun>:
4 1400014a4: 55 push %rbp
5 1400014a5: 48 89 e5 mov %rsp,%rbp
6 1400014a8: 89 4d 10 mov %ecx,0x10(%rbp)
7 1400014ab: 89 55 18 mov %edx,0x18(%rbp)
8 1400014ae: 8b 55 10 mov 0x10(%rbp),%edx
9 1400014b1: 8b 45 18 mov 0x18(%rbp),%eax
10 1400014b4: 01 d0 add %edx,%eax
11 1400014b6: 5d pop %rbp
12 1400014b7: c3 ret

```

```

o 1 #include <stdio.h>
2 #include <windows.h>
3
4 // 使用 objdump -d tt.exe | Select-String "fun" -Context 0,50
 将.\day09\test01.c中的可执行程序反汇编，同时将fun过滤出来
5 // 注意：使用这个方法执行程序时，最好函数中不要调用其他函数。函数中调用其他函数，地址的转
 换是通过当前函数的地址偏移过去的，所以使用了这种方法会找不大其他函数。
6 /*
7
8 int fun(int const num1, int const num2){
9 return num1 + num2;
10 }
11
12 > 000000001400014a4 <fun>:
13 1400014a4: 55 push %rbp
14 1400014a5: 48 89 e5 mov %rsp,%rbp
15 1400014a8: 89 4d 10 mov %ecx,0x10(%rbp)
16 1400014ab: 89 55 18 mov %edx,0x18(%rbp)
17 1400014ae: 8b 55 10 mov 0x10(%rbp),%edx
18 1400014b1: 8b 45 18 mov 0x18(%rbp),%eax
19 1400014b4: 01 d0 add %edx,%eax
20 1400014b6: 5d pop %rbp
21 1400014b7: c3 ret
22 */
23
24
25 // 存储fun函数的可执行的二进制编码
26 unsigned char buf[] = {
27 0x55,
28 0x48, 0x89, 0xe5,
29 0x89, 0x4d, 0x10,
30 0x89, 0x55, 0x18,
31 0x8b, 0x55, 0x10,
32 0x8b, 0x45, 0x18,
33 0x01, 0xd0,
34 0x5d,
35 0xc3,
36
37 };

```

```

38
39 int main(){
40
41 // 因为windows下不能执行数据区的代码，所以需要使用VirtualAlloc()申请一块可执行的
 内存
42 unsigned char *buf1 = VirtualAlloc(
43 NULL, // 1. 让系统自动选地址
44 sizeof(buf), // 2. 要申请多大内存
45 MEM_COMMIT | MEM_RESERVE, // 3. 内存分配方式
46 PAGE_EXECUTE_READWRITE // 4. 权限：可执行+可读+可写
47);
48
49 // 将buf的内存中内容拷贝到 buf1中
50 memcpy(buf1, buf, sizeof(buf));
51 // 为什么不是strncpy()
52 // memcpy()是内存数据的拷贝，他会将'\0'（或者 0）也一起拷贝过去。能接受 任何类型
 的指针
53 // strncpy()是字符拷贝，遇到'\0'会停止拷贝 只能接受 char*
54
55 // 使用函数指针 执行这个块区域
56 int (*p)(int const, int const) = (int (*)(int const, int const))buf1;
57
58 printf("%d", p(1, 2));
59
60 return 0;
61 }
62 // 现象
63 /*
64 3
65 */

```

- main函数的形参

- 官方只有两种写法：

- `int main();`
    - `int main(int argc, char **argv);` <==等价于==> `int main(int argc, char *argv[]);`

- argc

- 命令行参数个数 自动统计

- argv

- 指向的是命令行中的第一个字符串的首地址（命令行中的字符串以空格为分隔，且会被分配到一段连续的空间中）
    - 本质是 指针 而不是数组 形成一片连续的空间，将每个的参数首地址记录到这片空间中
    - `**argv` <====> `*argv[]` 在形参列表中，每一个[]都可以看成一个\*

- 使用： `.\可执行程序 参数1 参数2 参数3...`

- 本质上是将 可执行程序名和参数 当作字符串进行处理，使用argv存储每个字符串的首地址，同时告诉argc有几个参数
    - 外面传进的参数会进行修改。如 `1\0231` => `1\\0231` 所以打印了5个字符

- o ping命令本质也是 一段可执行程序 + 参数

```
o 1 #include <stdio.h>
2 #include <stdlib.h>
3
4 // 转换成数字
5 int my_atoi(char const *p){
6 int i = 0;
7 while(*p != '\0' && (*p - '0') >= 0 && (*p - '0') < 10){
8 i = i * 10 + (*p - '0');
9 p++;
10 }
11
12 if(*p != '\0'){
13 return -1;
14 }
15
16 return i;
17 }
18
19 // 等价于 int main(int argc, char *argv[])
20 int main(int argc, char **argv){
21 // 传入的参数 .\tt.exe 01\0123 abc 123
22 // 外面传入的参数会进行一定的修改
23 // 如: 01\0123 -> 01\\0123 让其能正常打印\
24
25 char **p = argv;
26
27 // 两种打印参数的办法
28 // 使用argc
29 for(int i = 0; i < argc; i++){
30 printf("%s ",argv[i]);
31 }
32
33 printf("\n");
34
35 // 不使用argc
36 while(*argv != NULL){
37 printf("%s ",*argv);
38 argv++;
39 }
40
41 printf("\n---");
42
43 // 如果需要使用外面传输, 需要转换成对应的类型, 进行计算
44 // int sum = 200 + atoi(p[3]); // atoi()将字符串转换成数字
45 int sum = 200 + my_atoi(p[3]); // my_atoi()将字符串转换成数字
46 // 如果使用int sum = 200 + atoi(argv[3]); 程序不会有结果, 因为指针argv移动到了 "123"之后的位置
47
48 printf("%d",sum);
49
50 return 0;
```

```
51 }
52
```

- 1 PS E:\project\cProject> gcc .\day08\test01.c -o tt
- 2 PS E:\project\cProject> .\tt.exe 01\0123 abc 123
- 3 E:\project\cProject\tt.exe 01\0123 abc 123
- 4 E:\project\cProject\tt.exe 01\0123 abc 123
- 5 ---323

## 函数传参

- 函数的参数可以有多个
- 形式参数只有在函数被调用的过程中才实例化
- 传一段连续空间的数据时（如：数组），需要告知函数 空间首地址 + 空间大小

## 函数返回值

- 函数的返回值只有一个
- 需要返回多个值时，在形参中使用指针完成

## 一段简单程序的汇编（函数的栈帧）

- [汇编网址](#)

- 1 #include <stdio.h>
- 2 int func(int a, int b)
- 3 {
- 4 int num = a + b;
- 5 return num;
- 6 }
- 7 int main(int argc, char \*argv[])
- 8 {
- 9 int num1 = 100;
- 10 int num2 = 200;
- 11 int num = func(num1, num2);
- 12 return 0;
- 13 }

- 明确观点：
- 1.rbp, rsp, ecx, edx, eax, rdx都是cpu的寄存器
- 2.代码是存在代码区中
- 3.现在这些汇编是程序运行时，对栈区的操作
- 4.栈是从高地址向低地址分配
- 5.程序运行之前，rbp, rsp指向其他未知区域
- 6.rsp指向指向的内存，才是分配好了的，未指向的空间没有分配
- 7.进入程序 或者 调用函数之前，当前的栈帧会为main或者 被调用函数留出32B的空间

main部分：  
push rbp; 可以理解成 \*(rsp -= 8) = rbp; rsp--; 将上一个盒子的首地址 或者 上一个盒子中下一条指令的地址，提前放入main的开头。

mov rbp, rsp; 理解成 \*rbp = \*rsp; 用rbp记录main的栈帧的首地址

sub rsp, 48; 理解成 rsp -= 48; rsp 记录main栈的结尾地址，此时确定main的栈帧大小

mov 16[rbp], ecx;  
mov 24[rbp], rdx; 将main()接收的形参值分别放到离首地址+16B的内存与首地址+24B的内存中（预留的空间中）

call \_main 进入main函数里面

mov 4[rbp], 100;  
mov -8[rbp], 200; 将100和200放到main中离首地址4B和8B的内存中

mov ecx, -8[rbp];  
mov eax, -4[rbp]; 将100和200放入cpu中的

```
1 func(int, int):
2 push rbp
3 mov rbp, rsp
4 sub rsp, 16
5 mov QWORD PTR 16[rbp], ecx
6 mov QWORD PTR 24[rbp], edx
7 mov ecx, QWORD PTR 16[rbp]
8 mov eax, QWORD PTR 24[rbp]
9 add ecx, edx
10 mov QWORD PTR -4[rbp], eax
11 add rsp, 16
12 pop rbp
13 ret
14
15 main:
16 push rbp
17 mov rbp, rsp
18 sub rsp, 48
19 mov QWORD PTR 16[rbp], ecx
20 mov QWORD PTR 24[rbp], rdx
21 call _main
22 mov QWORD PTR -4[rbp], 100
23 mov ecx, QWORD PTR -8[rbp]
24 mov esp, QWORD PTR -4[rbp]
25 mov ecx, eax
26 call func@123
27 mov QWORD PTR -12[rbp], eax
28 mov ecx, 0
29 add rbp, 48
30 pop rbp
31 ret
```

main续：  
mov ecx, eax 将eax赋值给ecx

call func(int,int) 进入func函数，将下一条指令的地址传给func

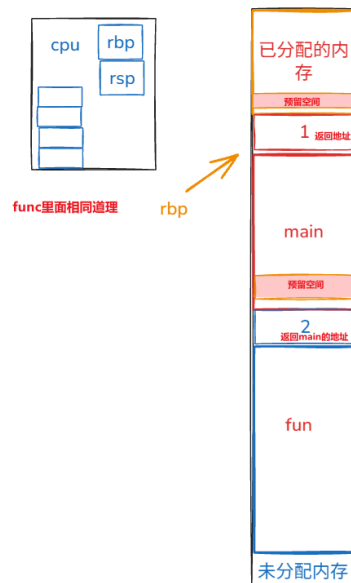
mov -12[rbp], eax 将结果放到rbp-12的内存上

mov ecx, 0 清零

add rsp, 48; 将rsp重新指向上一个盒子的底部

pop rbp; rbp回到上一个盒子

要移动画布，请按住(鼠标左键)或(空格)同时拖拽鼠标，或使用抓手工具。



## 14. 多文件编译

- 多模块编程原理（多文件）
  - C语言编译器
    - **每个源文件会单独编译成目标文件**，每个函数的调用，最终在二进制文件中只用函数名（本质是跳转的位置——在链接时，才能最终确认）
    - 多个目标文件链接成可执行文件
  - `nm .o`文件：查看二进制文件的符号表
    - `nm`结果中符号说明
      - `T`：全局 text(代码段) 符号，常见于对外可见的函数定义。
      - `t`：局部 text 符号，常见于只在本文件可见的静态函数。
      - `U`：未定义 (undefine) 符号，表示这里只是引用了它，真正定义在别处。
      - `__函数名`：编译器标准函数
  - 链接时，装配器 (ld.exe) 只关注符号表，不会检查函数的输入行为和返回值
- 多模块例子
  - 编译器只认函数名，编译阶段不会检查参数和返回值
    - 缺少定义：undefined reference to 'myAdd'
    - 重复定义：multiple definition of 'myAdd'
    - 没有声明的函数，编译器不会检查输入参数和返回值。如果输入参数少了，就使用空间的随机值；如果输入参数多了，就舍去。为了防止这个情况**在使用前声明这个函数，让编译器在编译器阶段报错**（所有函数，编译器会自动生成 跳转指令，提前声明函数的形式，编译器 评价依据）

```
1 /* fun.c */
2 #include <stdio.h>
3 void myAdd(int a, int b){
4 printf("a + b = %d\n", a + b);
5 }
6
7 /* main.c */
8 #include <stdio.h>
9
10 /* ---注释1--- */
11 // void myAdd(){
12 // printf("1111\n");
13 // }
14 /* ----- */
15
16 /* ---注释2--- */
17 // extern void myAdd(int,int);
18 /* ----- */
19
20 int main(){
```

```

21 printf("main...\n");
22 // a = 10; // 变量不声明, 编译器编译时会报 .\main.c:7:5: error: 'a'
undeclared (first use in this function)
23 myAdd(); //
24 return 0;
25 }

```

```

1 PS E:\project\cNew\day10\multi01> tree /f /a
2 E:.
3 fun.c
4 main.c
5 main.o
6
7 PS E:\project\cNew\day10\multi01> gcc -c -o main.o .\main.c
8 .\main.c: In function 'main':
9 .\main.c:6:5: warning: implicit declaration of function 'myAdd' [-
wimplicit-function-declaration]
10
11 6 | myAdd();
12 | ^~~~~
13
14 # 在main.o中,main是定义的全局函数, 有相对地址0000000000000000
15 # myAdd因为没有声明, 所有是未定义函数, 没有相对地址
16 PS E:\project\cNew\day10\multi01> nm .\main.o
17 0000000000000000 b .bss
18 0000000000000000 d .data
19 0000000000000000 p .pdata
20 0000000000000000 r .rdata
21 0000000000000000 r .rdata$zzz
22 0000000000000000 t .text
23 0000000000000000 r .xdata
24 U __main
25 0000000000000000 T main
26 U myAdd
27 U puts
28
29 # 当链接时, 依然有函数没有被提供, 那么会报 undefined reference to `myAdd'
30 -- 符号表少了缺少原材料
31 PS E:\project\cNew\day10\multi01> gcc .\main.o
32 E:/project/programming_environment/mingw64/bin/./lib/gcc/x86_64-w64-
mingw32/13.2.0/./././././././x86_64-w64-mingw32/bin/ld.exe:
33 .\main.o:main.c:(.text+0x1d): undefined reference to `myAdd'
34 collect2.exe: error: ld returned 1 exit status
35
36 # 链接时, 也不会检查函数是否缺少参数
37 # 能运行, 但结果不定
38 # 而有结果打印, 是因为调用函数后, 去栈上未初始化的a、b空间中取出随机值
39 # 添加声明
40 PS E:\project\cNew\day10\multi01> gcc .\fun.o .\main.o
41 PS E:\project\cNew\day10\multi01> .\a.exe
main....
a + b = -668533552

```

```

42
43 # 解开注释1,发现main.o中myAdd已经被定义,且分配了相对地址
44 # 在链接时,会报 multiple definition of `myAdd' —— 符号表中有重复,原材料多了
45 PS E:\project\cNew\day10\multi01> gcc -c -o main.o .\main.c
46 PS E:\project\cNew\day10\multi01> nm .\main.o
47 0000000000000000 b .bss
48 0000000000000000 d .data
49 0000000000000000 p .pdata
50 0000000000000000 r .rdata
51 0000000000000000 r .rdata$zzz
52 0000000000000000 t .text
53 0000000000000000 r .xdata
54 U __main
55 000000000000001e T main
56 0000000000000000 T myAdd
57 U puts
58 PS E:\project\cNew\day10\multi01> gcc .\fun.o .\main.o
59 E:/project/programming_environment/mingw64/bin/./lib/gcc/x86_64-w64-
mingw32/13.2.0/./../x86_64-w64-mingw32/bin/ld.exe:
.\main.o:main.c:(.text+0x0): multiple definition of `myAdd';
.\fun.o:fun.c:(.text+0x0): first defined here
60 collect2.exe: error: ld returned 1 exit status
61
62 # 解开注释2,此时编译阶段,会报 too few arguments to function 'myAdd' 参数
 不匹配
63 PS E:\project\cNew\day10\multi01> gcc .\main.c .\fun.c
64 .\main.c: In function 'main':
65 .\main.c:12:5: error: too few arguments to function 'myAdd'
66 12 | myAdd();
67 | ^~~~~
68 .\main.c:3:13: note: declared here
69 3 | extern void myAdd(int,int);
70 | ^~~~~

```

- 为了避免声明与函数定义不匹配,需要函数的实现者,维护接口的声明 —— 引出头文件 —— 来维护函数声明。为了避免调用者自己再次声明 —— 让调用者调用头文件

```

■ 1 /* fun.h */
 2 extern void myAdd(int a, int b);
 3
 4 /* fun.c */
 5 #include <stdio.h>
 6 void myAdd(int a, int b){
 7 printf("a + b = %d\n", a + b);
 8 }
 9
 10 /* main.c */
 11 #include <stdio.h>
 12 #include "fun.h"
 13
 14 int main(){
 15 printf("main...\n");
 16 myAdd(10,20);

```

```

17 return 0;
18 }

```

- 为了避免头文件多次被调用（头文件重复引用）—— 防御性编程 —— guard header

```

1 /* fun.h */
2 #ifndef __FUN_H_
3 #define __FUN_H_
4 extern void myAdd(int a, int b);
5 #endif
6
7 /* fun.c */
8 #include <stdio.h>
9 void myAdd(int a, int b){
10 printf("a + b = %d\n", a + b);
11 }
12
13 /* main.c */
14 #include <stdio.h>
15 #include "fun.h"
16
17 int main(){
18 printf("main...\n");
19 myAdd(10,20);
20 return 0;
21 }

```

- 为了区分自己定义的函数名与其他库中的同名函数，且自己定义的函数不对外使用，可以使用static关键字，编译器会优先使用本文件的函数
- #include <stdio.h> 与#include "xxx.h"区别
  - <>: 只在编译器提供的include目录下查找（实际开发中，规定寻找自定义接口）
  - "": 不光在编译器提供的头文件目录进行查找，也会在当前路径下查找（实际开发中，规定当前项目自己定义的接口）
- 多文件的定义
  - 头文件(.h文件)

```

1 // xxxx.h
2 #ifndef XXXX_H // 没有定义XXXX_H，执行里面的程序
3 #define XXXX_H // 定义XXXX_H，防止头文件重复包含(因为最后所以文件会
 // 被链接到一起)
4 ...
5 // 声明的函数 extern可以省略
6 extern fun();
7 ...
8
9 #endif

```

- 源文件(.c文件)

- ```
1 // xxxx.c
2 #include "xxxx.h"
3
4 /* 正常的程序 */
5 ...
6
```

- 其它文件调用时，应该包含该函数的头文件

- ```
1 // main.c
2 #include "xxxx.h"
3 ...
```

## 15. 浮点数

- 小数标准: IEEE745

## 16. 笔记中使用到的Linux命令

- Linux中对于字符串的操作
  - `strings` 文件名 过滤出字符串

- ```
1 [siguyuan@localhost a]$ cat main.c
2 #include <stdio.h>
3 #include "head.h"
4
5 int main()
6 {
7     int a = 20;
8     int b = 12;
9     printf("a = %d, b = %d\n", a, b);
10    printf("a + b = %d\n", add(a, b));
11    printf("a - b = %d\n", subtract(a, b));
12    printf("a * b = %d\n", multiply(a, b));
13    printf("a / b = %f\n", divide(a, b));
14    return 0;
15 }
16 [siguyuan@localhost a]$ strings main.c
17 #include <stdio.h>
18 #include "head.h"
19 int main()
20     int a = 20;
21     int b = 12;
22     printf("a = %d, b = %d\n", a, b);
23     printf("a + b = %d\n", add(a, b));
24     printf("a - b = %d\n", subtract(a, b));
25     printf("a * b = %d\n", multiply(a, b));
26     printf("a / b = %f\n", divide(a, b));
27     return 0;
```

- `hexdump -C 文件名` 查看文件在内存中的值

```

1  [siguyuan@localhost a]$ hexdump -C main.c
2  00000000  23 69 6e 63 6c 75 64 65  20 3c 73 74 64 69 6f 2e  |#include
   <stdio.|
3  00000010  68 3e 0a 23 69 6e 63 6c  75 64 65 20 22 68 65 61  |h>.#include
   "hea|
4  00000020  64 2e 68 22 0a 0a 69 6e  74 20 6d 61 69 6e 28 29  |d.h"..int
   main()|
5  00000030  0a 7b 0a 20 20 20 20 69  6e 74 20 61 20 3d 20 32  |.{.    int a
   = 2|
6  00000040  30 3b 0a 20 20 20 20 69  6e 74 20 62 20 3d 20 31  |0;.    int b
   = 1|
7  00000050  32 3b 0a 20 20 20 20 70  72 69 6e 74 66 28 22 61  |2;.
   printf("a|
8  00000060  20 3d 20 25 64 2c 20 62  20 3d 20 25 64 5c 6e 22  | = %d, b =
   %d\n"|
9  00000070  2c 20 61 2c 20 62 29 3b  0a 20 20 20 20 70 72 69  |, a, b);.
   prin|
10 00000080  6e 74 66 28 22 61 20 2b  20 62 20 3d 20 25 64 5c  |ntf("a + b =
   %d\|
11 00000090  6e 22 2c 20 61 64 64 28  61 2c 20 62 29 29 3b 0a  |n", add(a,
   b));.|
12 000000a0  20 20 20 20 70 72 69 6e  74 66 28 22 61 20 2d 20  |    printf("a
   - |
13 000000b0  62 20 3d 20 25 64 5c 6e  22 2c 20 73 75 62 74 72  |b = %d\n",
   subtr|
14 000000c0  61 63 74 28 61 2c 20 62  29 29 3b 0a 20 20 20 20  |act(a, b));.
   |
15 000000d0  70 72 69 6e 74 66 28 22  61 20 2a 20 62 20 3d 20  |printf("a * b
   = |
16 000000e0  25 64 5c 6e 22 2c 20 6d  75 6c 74 69 70 6c 79 28  |%d\n",
   multiply(|
17 000000f0  61 2c 20 62 29 29 3b 0a  20 20 20 20 70 72 69 6e  |a, b));.
   prin|
18 00000100  74 66 28 22 61 20 2f 20  62 20 3d 20 25 66 5c 6e  |tf("a / b =
   %f\n|
19 00000110  22 2c 20 64 69 76 69 64  65 28 61 2c 20 62 29 29  |", divide(a,
   b));|
20 00000120  3b 0a 20 20 20 20 72 65  74 75 72 6e 20 30 3b 0a  |;.    return
   0;.|
21 00000130  7d 0a                                     |}.|
22 00000132

```

- `echo $?` 命令行命令，返回上一次程序运行的状态 (false or true)
- `** objdump -S 二进制文件名`：将二进制文件反编译成汇编文件（C语言编译器需要带这个功能）**
- `nm .文件夹` 查看二进制文件的符号（C语言编译器一起需要带这个功能）

17. 杂项知识点

- 定长内存池（内存池中所有内存块都相同）
 - 运行场景：分配固定大小对象
 - 优点：减少内存碎片
 - 使用数组的方式（两种思路）：
 1. 用数组存可用空间的下标，使用的空间标记为-1，释放以后将（当前地址 - 首地址）/ 单个内存块大小的数值存入数组
 2. 数组的下标与内存池对应，在对应的下标下存对应内存块的状态
 - 使用链表的方式：
 - 需要管理结构体（内部有存下一个内存块的起始地址）、内存块

```
1 // 如果：面试题 —— 一般问这个就寄了，可以和面试官聊其他的
2 a+++a++;
3
4 // a + (++a)++
5 // a++ + a++
6
7 // 不回答a等于什么；
8 // 回答：不同编译器不同结果，（有歧义）实际开发不推荐这样写
```

- `_`变量名：一般是内部变量，外部不能调用、修改
- `__`变量名：一般是编译器的内部变量，只读
- 遍历法：
 - 从头到尾依次处理（确定区间，区间的映射范围是计算机能表示）
 - 循环，每次都要做什么事务，每次处理的元素（操作对象），都要做同样的事情
 - 循环：先循环体（循环规则），最后写循环退出条件
 - 寻找退出的条件，满足条件，退出
 - 退出的可能性，拆分处理 =》循环终止的位置，考虑结果的可能性（循环次数完、break退出）
- 记录1: `++*x` 与 `*x++`

```
1 #include <stdio.h>
2
3 int main(){
4     int num = 25;
5     int *x = &num;
6
7     printf("%d\n", *x); // 25
8     printf("%d\n", ++*x); // 26
9     printf("%d\n", *x); // 26
10    printf("%d\n", (*x)++); // 26
11    printf("%d\n", *x); // 27
12    printf("%d\n", *x++); // 27
13    printf("%d\n", *x); // x+1中的内容
14
15    return 0;
```

```
16     }
17
18     // 现象
19     /*
20     25
21     26
22     26
23     26
24     27
25     27
26     -1317012440
27     */
```

- printf中使用的内置变量名
 - `__FUNCTION__` → 打印当前函数名
 - `__FILE__` → 打印当前文件名
 - `__LINE__` → 打印当前行号
- printf有返回值，返回的是打印到标准输出（屏幕上）的字符个数
- 嵌入式关注的点 —— 编解码
 - 二进制编解码
 - 对一串二进制数，进行 分割 或者 组装
 - 常用于底层的通信协议
 - 文本编解码（符合编辑码）
 - 将 文本符号 与 底层的二进制序列 建立一个映射关系
 - 常用于 文本编码格式

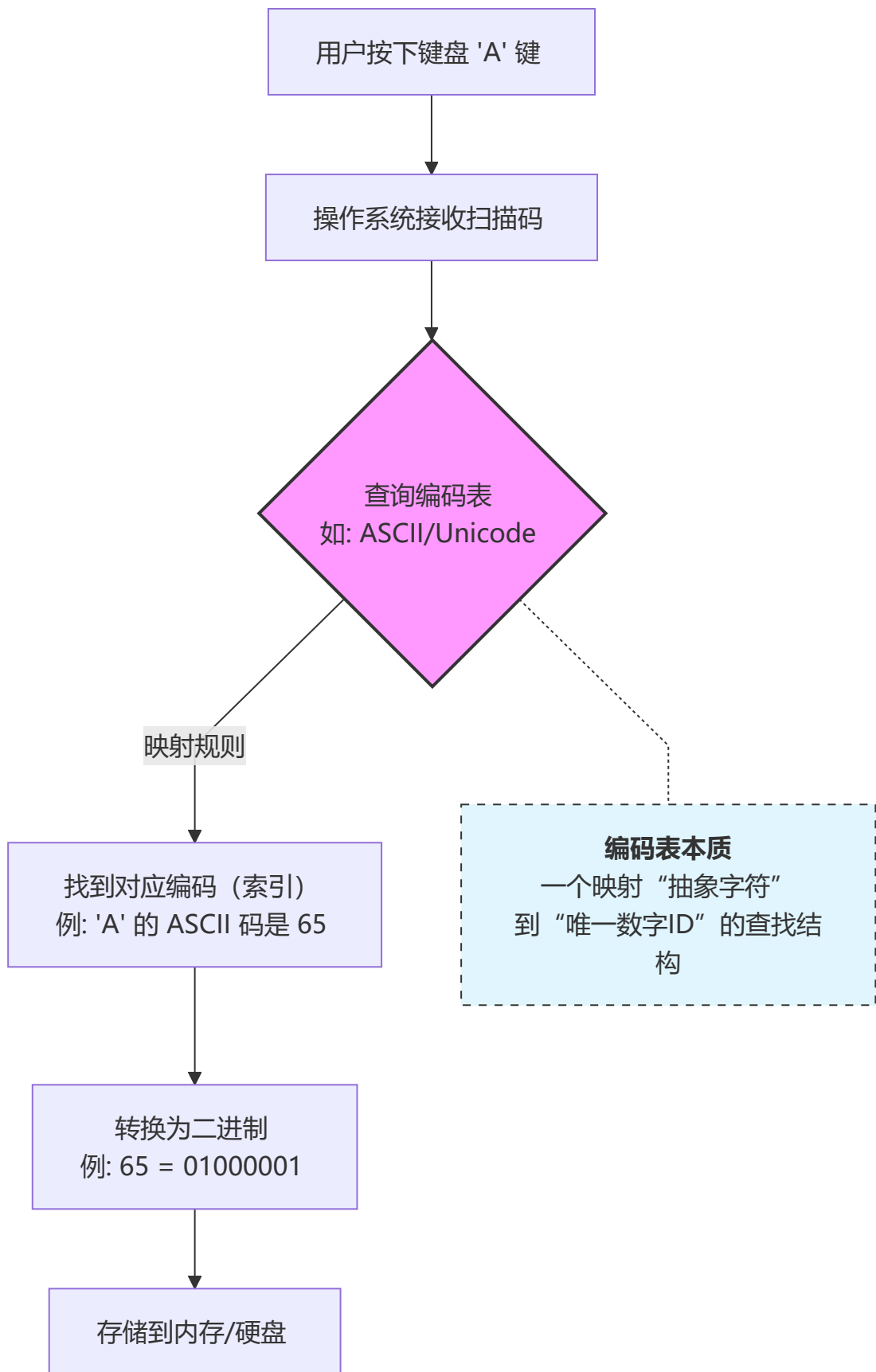
17.1 MinGW-w64目录说明

第一层目录	含义说明	主要内容
bin	可执行文件目录（最重要）	gcc、g++、ld、make、gdb、clang、python 等所有命令行工具都在这里
include	头文件目录（C/C++接口）	系统头文件、标准库头文件（如 <code>stdio.h</code> 、 <code>iostream</code> 等）
lib	库文件目录(C/C++接口的实现)	<code>.a</code> 、 <code>.lib</code> 静态库、 <code>.dll</code> 动态库、编译器运行时库
libexec	编译器内部工具目录	GCC 内部使用的程序（如 <code>cc1</code> 、 <code>cc1plus</code> 、 <code>collect2</code> 等），一般不需要手动操作
share	共享数据、文档、国际化文件	man 手册、locale（多语言）、gdb/python 脚本、cmake 模块、clang 头文件等

第一层目录	含义说明	主要内容
x86_64-w64-mingw32	目标平台专属目录（MinGW-w64 的核心）	针对 Windows 64位 的专属头文件、库文件、链接脚本等

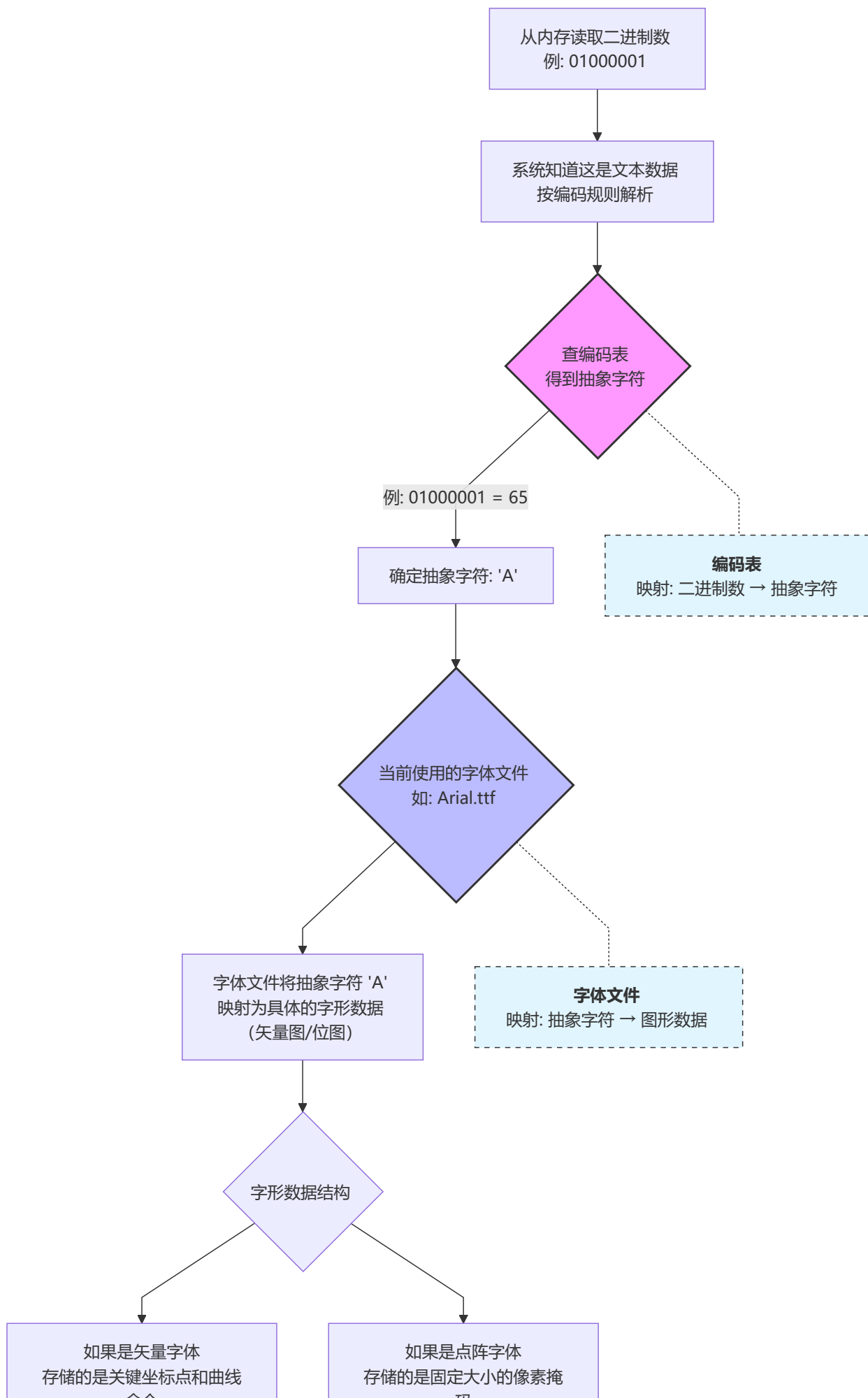
17.2 编码表（符号编解码）

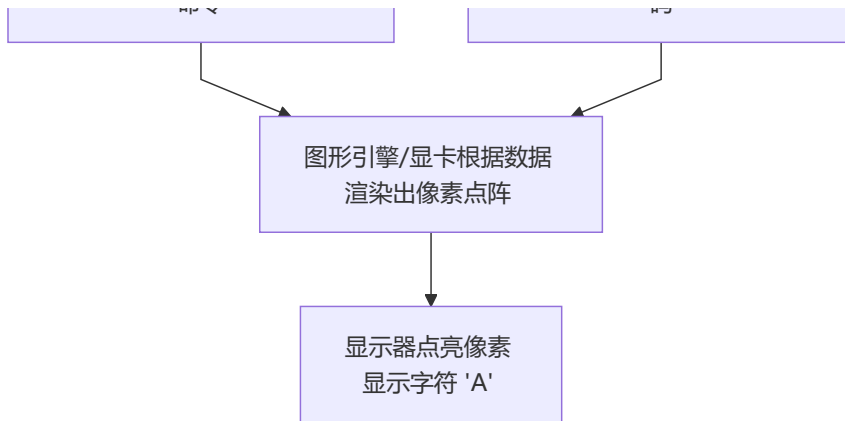
- 字符 转换到 内存的二进制数
 -



- 内存二进制数 转换到 字符

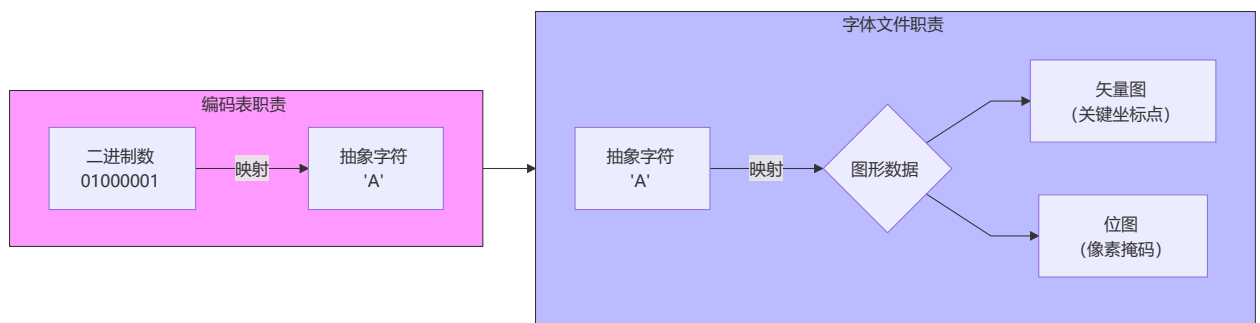
○





- 总的规则（抽象字符 本质还是一个数字 也叫做“码点”）

○



•

17.2.1 英语系国家的编码表 —— ASCII

- ASCII控制字符的编号范围是0-31和127 (0x00-0x1F和0x7F) ， 共33个字符

○

ASCII控制字符（共33个）

二进制	十进制	十六进制	缩写	Unicode 表示法	脱出字符 表示法	名称 / 意义
0000 0000	0	00	NUL	NUL	^@	空字符 (Null)
0000 0001	1	01	SOH	SOH	^A	标题开始
0000 0010	2	02	STX	STX	^B	本文开始
0000 0011	3	03	ETX	ETX	^C	本文结束
0000 0100	4	04	EOT	EOT	^D	传输结束
0000 0101	5	05	ENQ	ENQ	^E	请求
0000 0110	6	06	ACK	ACK	^F	确认回应
0000 0111	7	07	BEL	BEL	^G	响铃
0000 1000	8	08	BS	BS	^H	退格
0000 1001	9	09	HT	HT	^I	水平定位符号
0000 1010	10	0A	LF	LF	^J	换行键
0000 1011	11	0B	VT	VT	^K	垂直定位符号
0000 1100	12	0C	FF	F_F	^L	换页键
0000 1101	13	0D	CR	C_R	^M	CR (字符)
0000 1110	14	0E	SO	S_O	^N	取消变换 (Shift out)
0000 1111	15	0F	SI	S_I	^O	启用变换 (Shift in)
0001 0000	16	10	DLE	D_L_E	^P	跳出数据通讯
0001 0001	17	11	DC1	D_C_1	^Q	设备控制一 (XON 激活软件速度控制)
0001 0010	18	12	DC2	D_C_2	^R	设备控制二
0001 0011	19	13	DC3	D_C_3	^S	设备控制三 (XOFF 停用软件速度控制)
0001 0100	20	14	DC4	D_C_4	^T	设备控制四
0001 0101	21	15	NAK	N_A_K	^U	确认失败回应
0001 0110	22	16	SYN	S_Y_N	^V	同步用暂停
0001 0111	23	17	ETB	E_T_B	^W	区块传输结束
0001 1000	24	18	CAN	C_A_N	^X	取消
0001 1001	25	19	EM	E_M	^Y	连线介质中断
0001 1010	26	1A	SUB	S_U_B	^Z	替换
0001 1011	27	1B	ESC	E_S_C	^[	退出键
0001 1100	28	1C	FS	F_S	^\ ^_	文件分割符
0001 1101	29	1D	GS	G_S	^]	组群分隔符
0001 1110	30	1E	RS	R_S	^^	记录分隔符
0001 1111	31	1F	US	U_S	^_	单元分隔符
0111 1111	127	7F	DEL	D_E_L	^?	Delete字符

- 可显示字符编号范围是32-126 (0x20-0x7E) , 共95个字符。

o

ASCII可显示字符（共95个）

二进制	十进制	十六进制	图形	二进制	十进制	十六进制	图形	二进制	十进制	十六进制	图形
0010 0000	32	20	(space)	0100 0000	64	40	@	0110 0000	96	60	`
0010 0001	33	21	!	0100 0001	65	41	A	0110 0001	97	61	a
0010 0010	34	22	"	0100 0010	66	42	B	0110 0010	98	62	b
0010 0011	35	23	#	0100 0011	67	43	C	0110 0011	99	63	c
0010 0100	36	24	\$	0100 0100	68	44	D	0110 0100	100	64	d
0010 0101	37	25	%	0100 0101	69	45	E	0110 0101	101	65	e
0010 0110	38	26	&	0100 0110	70	46	F	0110 0110	102	66	f
0010 0111	39	27	'	0100 0111	71	47	G	0110 0111	103	67	g
0010 1000	40	28	(	0100 1000	72	48	H	0110 1000	104	68	h
0010 1001	41	29	)	0100 1001	73	49	I	0110 1001	105	69	i
0010 1010	42	2A	*	0100 1010	74	4A	J	0110 1010	106	6A	j
0010 1011	43	2B	+	0100 1011	75	4B	K	0110 1011	107	6B	k
0010 1100	44	2C	,	0100 1100	76	4C	L	0110 1100	108	6C	l
0010 1101	45	2D	-	0100 1101	77	4D	M	0110 1101	109	6D	m
0010 1110	46	2E	.	0100 1110	78	4E	N	0110 1110	110	6E	n
0010 1111	47	2F	/	0100 1111	79	4F	O	0110 1111	111	6F	o
0011 0000	48	30	0	0101 0000	80	50	P	0111 0000	112	70	p
0011 0001	49	31	1	0101 0001	81	51	Q	0111 0001	113	71	q
0011 0010	50	32	2	0101 0010	82	52	R	0111 0010	114	72	r
0011 0011	51	33	3	0101 0011	83	53	S	0111 0011	115	73	s
0011 0100	52	34	4	0101 0100	84	54	T	0111 0100	116	74	t
0011 0101	53	35	5	0101 0101	85	55	U	0111 0101	117	75	u
0011 0110	54	36	6	0101 0110	86	56	V	0111 0110	118	76	v
0011 0111	55	37	7	0101 0111	87	57	W	0111 0111	119	77	w
0011 1000	56	38	8	0101 1000	88	58	X	0111 1000	120	78	x
0011 1001	57	39	9	0101 1001	89	59	Y	0111 1001	121	79	y
0011 1010	58	3A	:	0101 1010	90	5A	Z	0111 1010	122	7A	z
0011 1011	59	3B	;	0101 1011	91	5B	[	0111 1011	123	7B	{
0011 1100	60	3C	<	0101 1100	92	5C	\	0111 1100	124	7C	
0011 1101	61	3D	=	0101 1101	93	5D	]	0111 1101	125	7D	}
0011 1110	62	3E	>	0101 1110	94	5E	^	0111 1110	126	7E	~
0011 1111	63	3F	?	0101 1111	95	5F	_				

- o 大写字母 优先级大于 小写字母（大写字母先被找到）
- o ‘字符’：助记符，编译器会去 映射表中找这个字符对应的二进制数
- 其余的0x80~0xFF 没有被使用

17.2.2 非英语系国家的编码表 —— 国标表（GB系列）、万国表（Unicode表）

17.2.2.1 国标表（国标码）

- 利用ASCII剩余的 0x80~0xFFH，每个国家自己编制了对应的编码表
- 中文 的编码表—— GBK —— 2个字节标识一个字符（2个字节都不会使用0x00~0x7F这个区间的值）
- 问题：没有统一的编码，使用别的国家的计算机时，出现乱码情况。

- 乱码情况，文件的生成时格式与使用时格式不同导致的。

- 如：源文件（使用vscode的编码格式） ——> 内存中存储的就是该编码格式映射的二进制数 ——> 运行时的环境（Windows的终端 默认使用的都是 GBK格式）

- ```
1 # cmd终端查看方式
2 chcp
3 # 输出 活动代码页：936 代表使用的是GBK的编码格式 活动代码页：65001 代表使用的是utf-8的编码格式
```

■

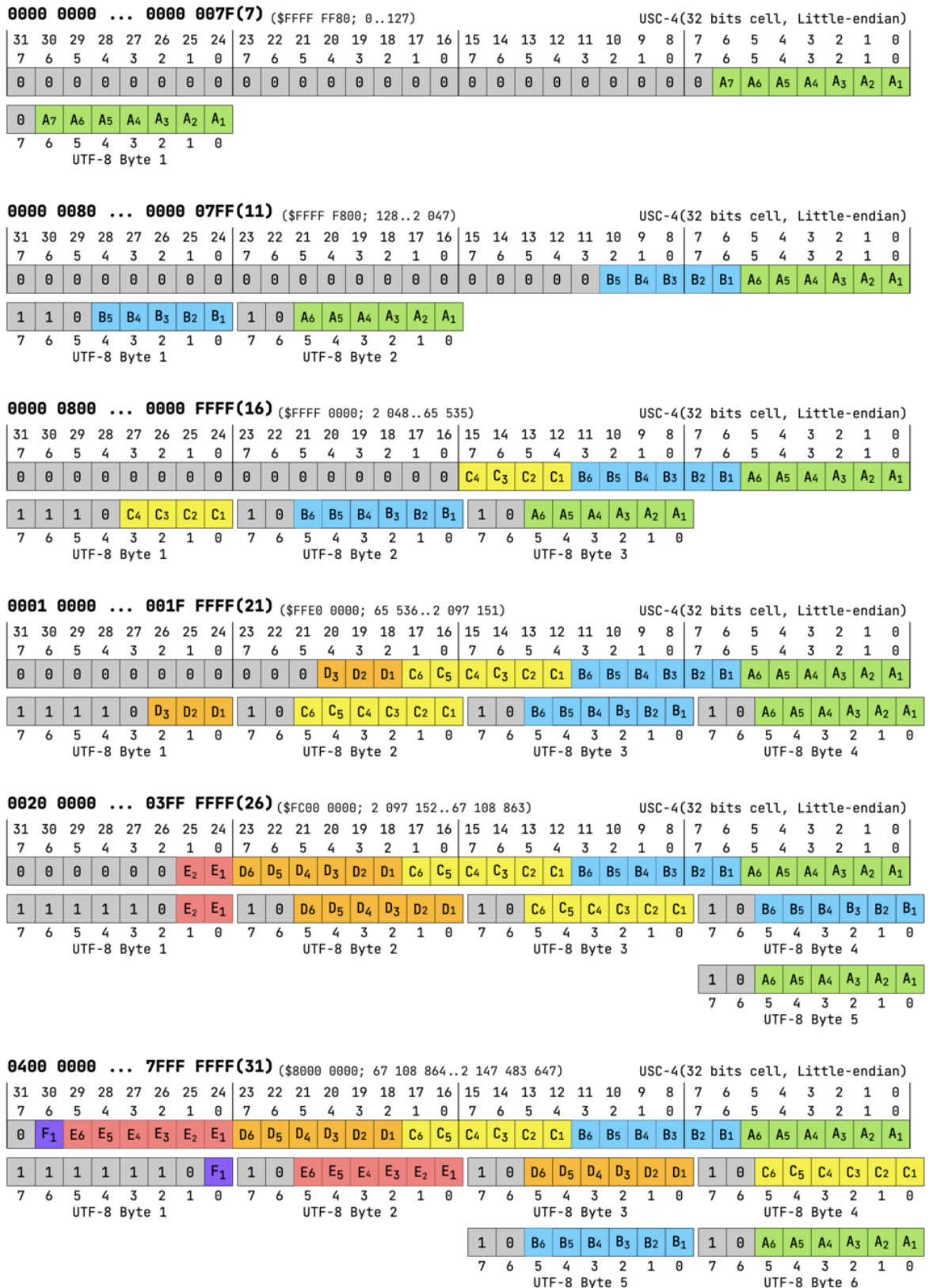
```
1 # powershell终端查看编码方式和版本方式
2 # 查看版本 Major 对应是版本
3 > $PSVersionTable.PSVersion
4
5 Major Minor Patch PreReleaseLabel BuildLabel
6 -----
7 7 5 0
8
9 # 查看编码格式 936表示GBK编码 65001表示utf-8编码
10 # 输出编码格式
11 > [Console]::OutputEncoding
12 EncodingName : Chinese Simplified (GB2312)
13 WebName : gb2312
14 HeaderName : gb2312
15 BodyName : gb2312
16 Preamble :
17 WindowsCodePage :
18 IsBrowserDisplay :
19 IsBrowserSave :
20 IsMailNewsDisplay :
21 IsMailNewsSave :
22 IsSingleByte : False
23 EncoderFallback : System.Text.InternalEncoderBestFitFallback
24 DecoderFallback : System.Text.InternalDecoderBestFitFallback
25 IsReadOnly : False
26 CodePage : 936
27
28 # 输入编码格式
29 > [Console]::InputEncoding
30 EncodingName : Chinese Simplified (GB2312)
31 WebName : gb2312
32 HeaderName : gb2312
33 BodyName : gb2312
34 Preamble :
35 WindowsCodePage :
36 IsBrowserDisplay :
37 IsBrowserSave :
38 IsMailNewsDisplay :
39 IsMailNewsSave :
40 IsSingleByte : False
41 EncoderFallback : System.Text.InternalEncoderBestFitFallback
```

|    |                 |   |                                            |
|----|-----------------|---|--------------------------------------------|
| 42 | DecoderFallback | : | System.Text.InternalDecoderBestFitFallback |
| 43 | IsReadOnly      | : | True                                       |
| 44 | CodePage        | : | 936                                        |

#### 17.2.2.2 万国表 (万国码)

- unicode编码表 (utf-16 —— 使用16bit 2B)
  - 统一全球的各个国家的编码格式
  - 使用2个字节 表示一个字符
  - 等长编码
  - 弊端: 英语系国家的字符占用内存增大
- utf-8编码方式
  - 变长编码
  - 英语系国家使用1B表示字符
  - 中文 使用 3B表示
  -

# UTF-8 Encoding Scheme



- 每一对的第一行属于Unicode编码的内存表示，第二行属于utf-8的内存表示
  - utf-8怎么转unicode:
    - 第一个字节的 灰色区域为标志位（有多少个1 就是多少字节， 0是为了与实际数据区分 没有1就是 只有了1个字节）
    - 后续字节前面2位必定是（1 0 —— 使用了0x80~0xFF的数） —— 能自校验



- ③ Input Operands (输入操作数)
  - C 语言变量作为输入传递给汇编代码。
  - 格式为: "限制符" (C语言表达式)。
  - 常用的限制符: "r" (放入寄存器)、"m" (直接使用内存地址)、"i" (立即数)。
- ④ Clobbered Registers (破坏描述符)
  - 告诉编译器, 你在汇编代码中悄悄修改了哪些寄存器或内存, 让编译器提前做好备份, 避免寄存器冲突。
  - 例如: "eax", "memory" ("memory" 表示汇编修改了内存, 强制编译器重新从内存读取变量值, 起到 Memory Barrier 的作用)。

- 例子

- 两个整数相加 (最基础的  $R=A+B$ )

```

1 #include <stdio.h>
2
3 int main() {
4 int src1 = 10, src2 = 20, dst;
5
6 // 实现 dst = src1 + src2
7 asm volatile (
8 "addl %2, %0" // 汇编指令: %0 = %0 + %2
9 : "=r" (dst) // %0: 输出, 绑定到 dst, r 表示寄存器
10 : "0" (src1), // %1: 输入, "0" 表示复用 %0 相同的寄存器 (把
src1 读进 %0)
11 : "r" (src2) // %2: 输入, 绑定到 src2, r 表示寄存器
12 : // 没有修改其他额外寄存器
13);
14
15 printf("Result: %d\n", dst); // 输出 30
16 return 0;
17 }
```

- Linux 内核风格的原子自增 (Atomic Increment)

```

1 static inline void atomic_inc(int *v) {
2 asm volatile (
3 "lock; incl %0" // lock 前缀保证原子性, incl 递增内存值
4 : "+m" (*v) // %0: 既是输入又是输出 ("+"), 直接操作内存 ("m")
5 : // 无纯输入
6 : "cc", "memory" // "cc" 表示修改了 CPU 标志位 (Condition Code)
7);
8 }
```

## 18. 注脚

---

1. 代码检测: vscode中使用clangd插件(LLVM提供的客户端), 同时mingGW中bin目录下有clangd.exe (LLVM的服务端)。clangd将客户端捕获的输入, 发送给服务端进行检查。 [↩](#)

2. 打开文件名, 读取文件中的内容, 展开到当前源码位置处。头文件的寻找方式有3种: `<>`: 在编译器提供的头文件目录下寻找、`"`": 主要在当前路径下寻找、`gcc -I<path>` : 额外添加寻找路径, 使用gcc的-I参数 [↩](#)

3. 补码实际含义 :最高位为负权值 与 其余正权值 组成的二进制数

如: -100的补码1001 1100 实际表示:  $-1 * 2^7 + 1 * 2^4 + 1 * 2^3 + 1 * 2^2 = -100$  [↩](#)

4. 快速方法

发现: 负数的最高**负权值** + 其余的正权值 (全部有效, 为1) = -1

负数的补码: (负数绝对值 - 1) 的原码 取反

-100 :  $100 - 1 = 99$  99的原码: 0110 0011 取反为-100的补码 1001 1100 [↩](#)

5.

```
// <stdint.h> 或 <cstdint> 内部实现逻辑 (因平台而异)

#ifdef __LP64__ // 64位 Unix/Linux/macOS
 typedef long int64_t;
#elif defined(_WIN64) // Windows 64位
 typedef long long int64_t;
#elif defined(__SIZEOF_LONG__) && __SIZEOF_LONG__ == 8
 typedef long int64_t;
#elif defined(__SIZEOF_LONG_LONG__) && __SIZEOF_LONG_LONG__ == 8
 typedef long long int64_t;
#else
 // 极少数情况下可能用其他方式, 或编译失败
#endif
```

[↩](#)

6. C语言, 初始化动作对于已经别管理的空间只会执行一次, 无论全局、静态 (这二者的生命周期都是程序结束)、局部变量 (每次进入函数都会开辟新的空间, 只是名字不变, 当前函数结束就释放, 所以初始化动作只执行了一次, 下一次进入改函数都是新的空间) [↩](#)

7. 源码文件为文本文件。

文本文件: 按照一系列特定符号编码 显示出符号。

二进制文件: 需要特定的解码规则, 才能显示。

[↩](#)